GPS-denied navigation: solution test report

Test run on 2026-10-23. Branch `claude/new-session-peh13r`, folder `gpsdenied-eval/`. Every number comes from a logged run under `results/`; this document is generated by `harness/report_build.py`.

1. Executive summary

What was tested. Six GPS-denied navigation solutions (S1-S6) and three paper-only items (S7) on the same synthetic NIDAR-like video, the same reference map and the same metric code, plus an ArduPilot SITL loop. The real NIDAR video, GPU, Jetson and the fallback datasets (MAFS, MUN-FRL, ALTO) were **not available**, so every result is on synthetic data, on CPU, with a map that is the same imagery the video was rendered from. Accuracy numbers are therefore **optimistic** and the ranking is about relative behaviour, not absolute field accuracy.

Ranking (owner weights). 1. S6 our composite (Jetson baseline) (8.29), 2. S5 SWA-PF (7.78), 3. S3 tile-matching (Shuhaieva) (4.53), then S1 koksalan/gnss-denied-nav (4.25), S2 uav-geolocalization (CAEVL + VIO) (2.38), S4 hmakelin/gisnav (0.79). The sensitivity check keeps the same winner under every weight change, so a single overall winner is stated.

Rank	Solution	Weighted score (0-10)	Accuracy (30%)	Robustness (20%)	Real-time and edge fit (20%)	SITL outcome (15%)	Integration, documentation and licence (15%)
1	S6 our composite (Jetson baseline)	**8.29**	9.9	9.2	5.8	9.3	6.1
2	S5 SWA-PF	**7.78**	10.0	10.0	4.7	7.8	4.5
3	S3 tile-matching (Shuhaieva)	**4.53**	6.0	3.8	3.1	5.0	4.0
4	S1 koksalan/gnss-denied-nav	**4.25**	6.9	3.8	0.0	4.8	4.8
5	S2 uav-geolocalization (CAEVL + VIO)	**2.38**	0.0	0.1	10.0	0.0	2.5
6	S4 hmakelin/gisnav	**0.79**	0.0	0.0	0.0	0.0	5.3

Best for each purpose.

Purpose	Best	Basis
Highest accuracy (lowest accepted-fix median, fix rate over 20 %)	S3	common 1 Hz set, synthetic NIDAR-like
Highest coverage at 50 m (recall@50)	S5	recall over all frames
Lowest latency under the constrained replay	S2	1-2 CPU cores, CPU only (no Jetson)
Best autopilot integration (SITL outcome score)	S6	SIM-2..SIM-7, 3 seeds

Purpose	Best	Basis
Best for Jetson-class deployment (edge-fit score)	S2	latency, throughput, memory; NOT measured on a Jetson
Most robust over feature-poor terrain and weather	S5	precision, blur/haze windows, worst terrain class
Easiest to adopt (setup effort)	S5	minutes + 10 per failed step

Recommendation for the NIDAR system. Build the NIDAR baseline on the **retrieve → match → PnP → consistency gate → tracking** structure that S6 implements, with S6's permissive-licence parts (MegaLoc, XFeat, OpenCV; confirm the LighterGlue weight licence), fed by the compass and barometer hints. On the synthetic data it was the only complete pipeline: 92 % of frames fixed, 0.4 m median, 100 % precision, a 100 % mission-completion rate in SIM-2, SIM-4 and SIM-7, and a 0.9 s step on a 2-thread CPU. **Treat this as a design direction, not a verdict:** (1) S6 is our own composite, written and tuned (gate thresholds, tile size, crop) on the very frames it is scored on, with no held-out set; (2) video and map come from the same imagery, so the real appearance gap is untested; (3) nothing was measured on a Jetson, ONNX/TensorRT export of the matcher failed on CPU, and in the 5 Hz real-time replay it processed only 21 % of the frames on two threads and 13 % on one core; (4) it needs a heading and height hint, which were noise-free here. S5 (particle filter on semantic labels) was the second most accurate and the most robust to blur and haze (93.5 % fixed), but it needs an odometry source, converges only after about 10 s, takes several seconds per step on this CPU at 20,000 particles (the first step 37 s) and its labels here came from a substitute map, so it fits a GPU companion computer, not a CPU-only edge board. S1's gate and tracking ideas are already inside S6; S1 itself retrieved poorly with pretrained weights (22 % of frames, all in the first 44 s). S3 is a good reference for the plain two-stage idea but has no rotation handling and no gating; S2 (VIO plus map fixes) failed with our under-trained encoder and a VIO substitute, so it is neither supported nor refuted by this test; S4 could not be run. Whichever solution is chosen, the autopilot side needs a velocity or odometry aid: even ideal 5 m-noise fixes cannot bridge a 20 s gap with our position-only EKF configuration (SIM-3 failed for every row, including the control).

What could not be tested. The real NIDAR video and its telemetry (we used a synthetic NIDAR-like flight); any field or non-synthetic imagery with a seasonal or lighting gap to the map; the fallback datasets MAFS 200 m, MUN-FRL and ALTO (unreachable) and the UAV-VisLoc reproduction; any GPU run and any Jetson run (no Jetson number in this report is measured); TensorRT; OpenVINS inside S2 (built but not run: the Docker image export ran out of disk); S4 (GISNav) in any form: the stack could not be stood up here (6.9 GB image, PX4/Gazebo/QGC build, GPU and X11, 50 GB); S5 on the MAFS dataset and the cross-area test; the YFS90 item (no code exists) and the STHN item (thermal imagery only); a fully closed-loop SITL in which each solution's output is recomputed from the simulated vehicle's own camera view (we used Mode B with precomputed estimates); take-off with GPS already denied (SIM-6 cuts GPS at the start of cruise); roll/pitch/attitude errors in the hints (they are noise-free).

2. Test setup

2.1 Hardware and software

Item	Value
Machine	cloud container, 4 CPU cores (Intel Xeon @ 2.1 GHz), 15 GB RAM, no GPU, no Jetson, about 30 GB disk allowance shared by everything
OS / Python	Ubuntu 24.04 container; Python 3.13 (3.9 / 3.11 / 3.12 per solution where the solution needed it); every solution in its own virtualenv (`solutions/<sol>/env`), versions in `solutions/<sol>/lock.txt`
Torch	CPU wheels only (torch 1.10 for S5, 2.x for the others)
Autopilot	ArduCopter 4.5.7 SITL built from source (`Copter-4.5.7`), run at 8x speed-up; pymavlink 2.4.50
Repository commits	`solutions/<sol>/COMMIT.txt` (S1 572b9ff, S2 9983dca, S3 e63334e, S4 f2c7264, S5 e8a9217; S7 YFS90 968eaf8)
Random seeds	replay seed 0 for every solution; SITL seeds 0, 1, 2 (outlier injection and drops use the seed; SITL's own physics is deterministic, so seeds mainly matter for SIM-5)

CPU timing caveat: while the solution agents were working, the machine was shared by up to six heavy jobs, so latencies in the per-solution chapters were measured under load. The latency table in section 3.5 comes from **serial re-runs on an otherwise idle machine** (commands in the appendix).

2.2 Inputs used and why (runbook section 3)

Input	Used	Branch / reason
NIDAR video	**not supplied**: no file under `data/nidar/`, in the session uploads, or anywhere on disk (search for "nidar")	runbook branch 3. Per your instruction we **generated a synthetic NIDAR-like flight** instead; every NIDAR line in this report is therefore "synthetic NIDAR-like", not real NIDAR footage
Flight	ArduCopter SITL flew A→B→C→D at 200 m AGL, 12 m/s: A→B 500 m north, B→C 500 m east, C→D 250 m west and 400 m south; cruise 152.8 s, 1,464 m	truth lat/lon/yaw from SITL's true state at 60 Hz (`truth60.csv`), 5 Hz subset `truth.csv`
Telemetry	SITL IMU (`imu.csv`), barometer/attitude hints from truth	the solution hints are noise-free (a limit on realism)
Reference map	Esri World_Imagery, zoom 19 (0.283 m/px) over Pune, India (about 1.8 × 1.8 km); GeoTIFF `data/nidar/map.tif` (EPSG:3857)	benchmark use only; never redistributed (the map, frames and video are git-ignored; report figures contain no imagery)
Camera	gimbal-stabilised **nadir** pinhole, 1280×800 (1.02 MP) grayscale, global shutter (every frame is one instant, no rolling-shutter skew), 60 fps, 60° horizontal FOV, f = 1108.5 px, 0.18 m/px at 200 m AGL	frames rendered by warping the map through the exact ground-plane homography
Weather	minor Gaussian blur (σ up to 3 px) at 21-31 s and 95-104 s; close haze (transmission down to 0.4, low-frequency cloud pattern) at 49-69 s, 95-107 s and 125-134 s; sensor noise and vignetting	schedule in `data/nidar/weather.json`, windows marked on every error-over-time plot
DEM	not used by any solution that ran	S4 would have used one
Fallback datasets MAFS	**not run**: MAFS only on Baidu Pan (unreachable from here, no open mirror found), MUN-FRL / ALTO links not reachable or too	all "fallback dataset" test IDs read "not run: dataset unreachable"

Input	Used	Branch / reason
200 m, MUN-FRL, ALTO	large; UAV-VisLoc exists as a 17.7 GB archive on Hugging Face, too large for the disk allowance	

2.3 Deviations from the runbook (all of them)

1. **Synthetic video, not NIDAR** (above). The reference map is the very imagery the frames were rendered from, so there is no seasonal, lighting or resolution gap between query and map. This makes matching **much easier than in the field** and several solutions return implausibly small errors (for example S3 verified fixes at 0.07 m); we checked this against the metric code and the data alignment and it is genuine, a consequence of identical pixels.
2. **Data defect found and fixed during the run.** The first render used a body-fixed camera on a quadcopter that pitches −30° in cruise, so frames were strongly oblique (runbook: nadir or near-nadir). We re-rendered with a stabilised nadir camera and **re-ran every primary test**. The oblique results are kept in `results/<sol>/nidar_oblique/` and discussed as a stress result (S5: 106.8 m median error without rectification; S6: nadir assumption failed completely; S2: near chance).
3. **1 Hz subsampling.** Several tests ran on every 5th frame (1 Hz, 153 frames) because CPU time per frame is 5-35 s for S1, S3 and S5; each result names its frame count. The master comparison uses the **same 153 frames for every solution**.
4. **CPU only, no Jetson.** The constrained replay is an approximation: the process is pinned to 1-2 cores with `taskset`. No number in this report is a Jetson measurement.
5. **Degraded or substitute components**, each flagged where it appears: S1 uses pretrained DINOv2-base (the author's fine-tuned weights are unpublished); S2's VIO layer is our substitute (OpenVINS built but could not be run) and its CAEVL encoder is under-trained; S5's semantic map is our substitute from the repo's SegFormer.
6. **SITL is Mode B (video in the loop), not closed loop.** The vehicle flies the A→B→C→D mission in SITL; the solutions' estimates were precomputed offline on the recorded video. At each recorded frame position the bridge passes the solution's *error vector for that frame* applied to the SITL vehicle's true position as a `GPS_INPUT` fix. A solution's output is never recomputed from the SITL vehicle's own view, and its compute latency is not modelled except in SIM-4 (added 200 / 500 ms). SIM-6 (cold start) starts the GPS cut at the beginning of cruise instead of before take-off, because there are no ground-level frames.
7. **Fix feed rate** follows each solution's run: 5 Hz (S6, S2, S1 tracking run) or 1 Hz (S3, S5), without modelling the seconds of CPU time per fix.
8. **Not run** (reasons in the chapters): S4 (stack could not be stood up: 6.9 GB image pull ran out of disk, PX4/Gazebo/QGC needs a GPU, X11 and 50 GB), S1/S2/S3/S5/S6 fallback datasets, S2 with OpenVINS, TensorRT anywhere, GPU runs, Jetson runs, S7 items other than Bearing-UAV.

2.4 Test status matrix (every runbook test ID has a result or a recorded reason)

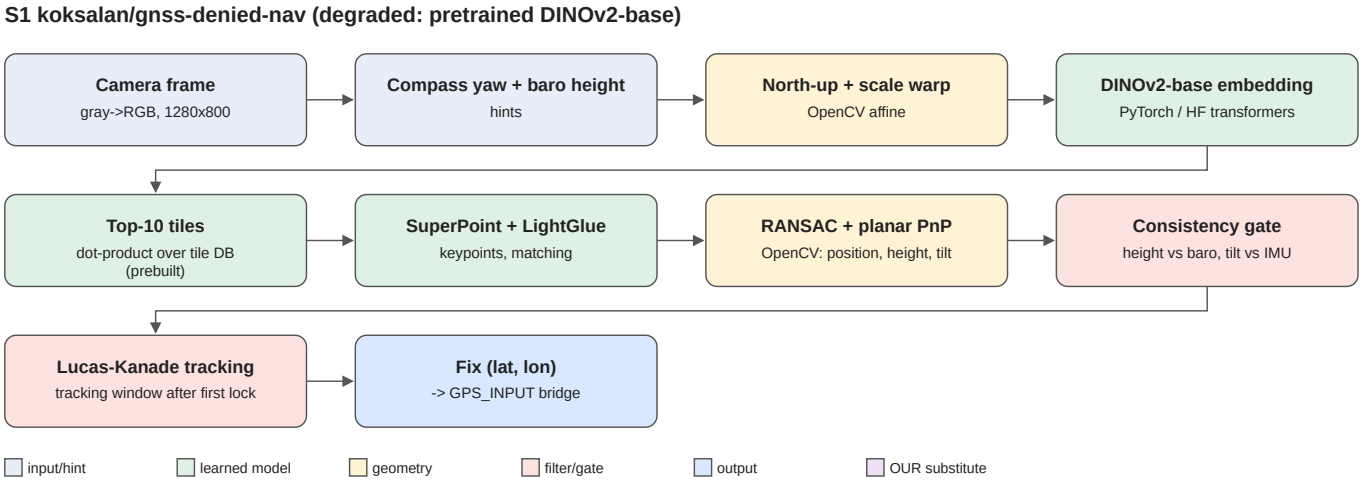
"done" = run and logged under `results/`; frame counts are in the per-solution chapters. "serial" = re-run alone on the idle machine (section 3.5).

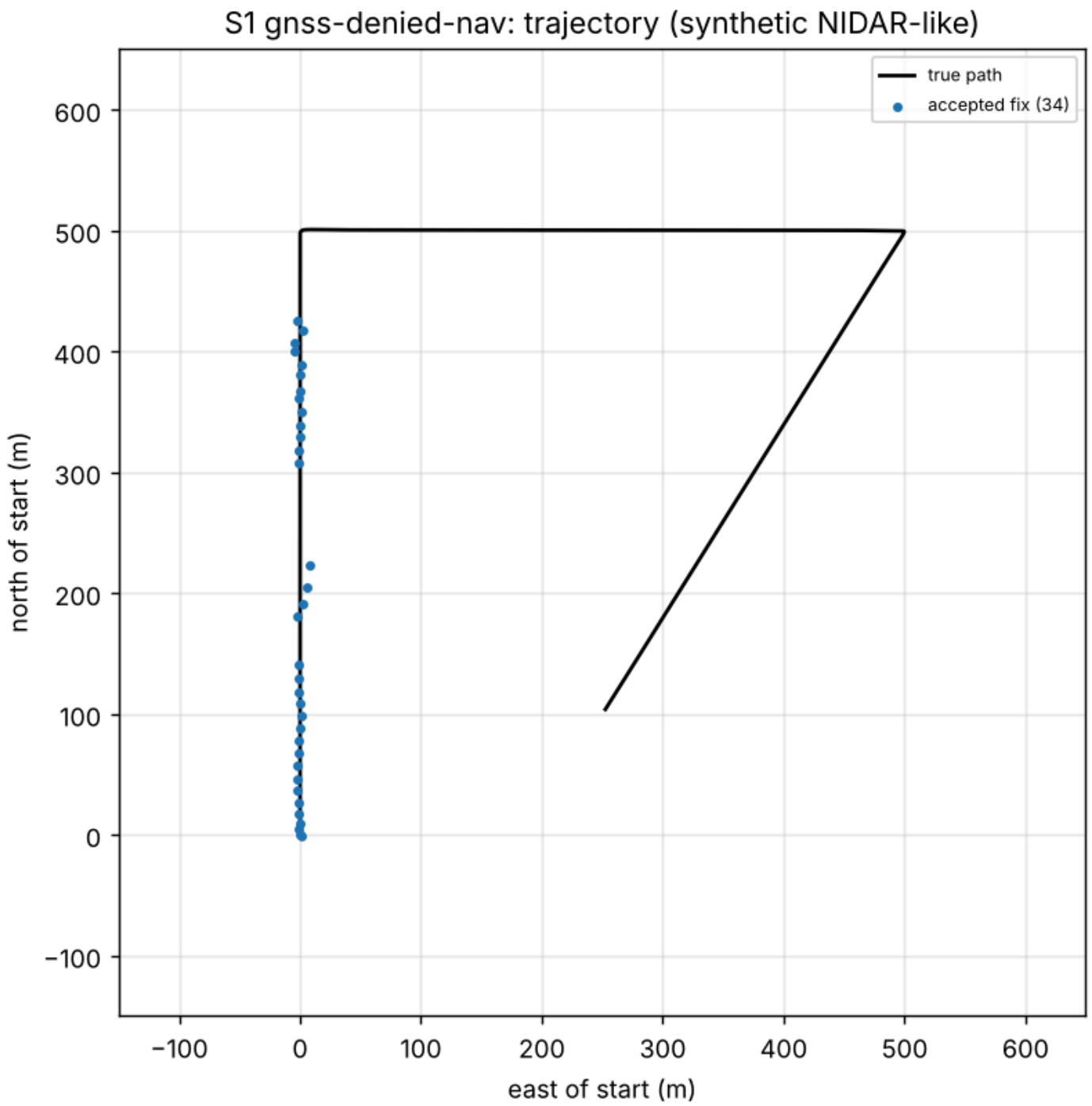
Solution	Done	Not run, with reason
S1	T1 global search (1 Hz, 153 frames), T2 tracking (1 Hz and full 5 Hz), T3 real-time (serial), T4 constrained (ONNX export attempts, 1-core replay), T5 gate off, T6 no hints (weak: 77 frames, all on the first leg), T8 SITL (SIM-0..7, 3 seeds), T9 failure injection (first 60 s)	T1 at 5 Hz (about 7 h CPU: 1 Hz used); T7 fallback datasets (UAV-VisLoc is a 17.7 GB archive; MAFS/MUN-FRL unreachable); the repo's own visual-odometry gap filling was excluded from the adapter
S2	T1 VIO only (substitute), T2 retrieval only, T3 fused causal, T4 align/ekf/causal, T5 consensus off, T6 corridor on/off, T7 tile size 60/120/240 m (1 Hz), T8 real-time and constrained (serial), T10 SITL (fused and VIO-only rows)	T9 MUN-FRL (unreachable, no mirror); OpenVINS itself (built, Docker image export out of disk); GPU run
S3	T1 as published, T2 verified vs fallback, T3 no scale normalisation, T4 retrieval depth 1/5/10, T5 terrain, T6 real-time and constrained (serial), T8 our Kalman extension, T9 SITL (as published and with de-rotation)	T7 fallback datasets and UAV-VisLoc subset 06 (17.7 GB archive, MAFS unreachable)
S4	none	T1 PX4 SITL demo (6.9 GB gisnav image pull: no space left on device; PX4 + Gazebo + QGC build; GPU, X11, 50 GB required), T2, T3, T6, T7 (depend on the stack), T4 not applicable, T5 dataset unreachable, T6 SITL: no estimates exist
S5	T2 NIDAR replay (substitute semantic map), T3 particle sweep 1k/5k/20k/40k, T4 initialisation (3 seeds, 40 frames), T5 map resolution, T6 scene breakdown, T7 constrained (serial; 2,000 particles, 2 cores), T9 SITL (1 Hz fixes)	T1 MAFS reproduction and T8 cross-area (MAFS only on Baidu Pan, 3 attempts for a mirror)
S6	T1 offline (5 Hz), T2 retriever swap, T3 matcher swap, T4 offline with tracking and real-time (serial), T5 PyTorch/ONNX variants (CPU), T7 SITL	T5 TensorRT (no GPU: `pip install tensorrt` fails; LighterGlue ONNX export fails with an IndexError), T6 fallback dataset (unreachable); every Jetson number
S7	YFS90 documentation review, Bearing-UAV (weights run on 153 synthetic frames), STHN (not applicable: thermal only)	YFS90 code does not exist; STHN dataset not downloaded
SITL scenarios	SIM-0..SIM-7 (SIM-4 as 4a/4b), 3 seeds, rows S1, S2 (+VIO-only), S3 (+de-rotation extension), S5, S6, plus an ideal-fix control	S4 (no estimates); SIM-6 is a cold start at the start of cruise, not before take-off

S1 koksalan/gnss-denied-nav

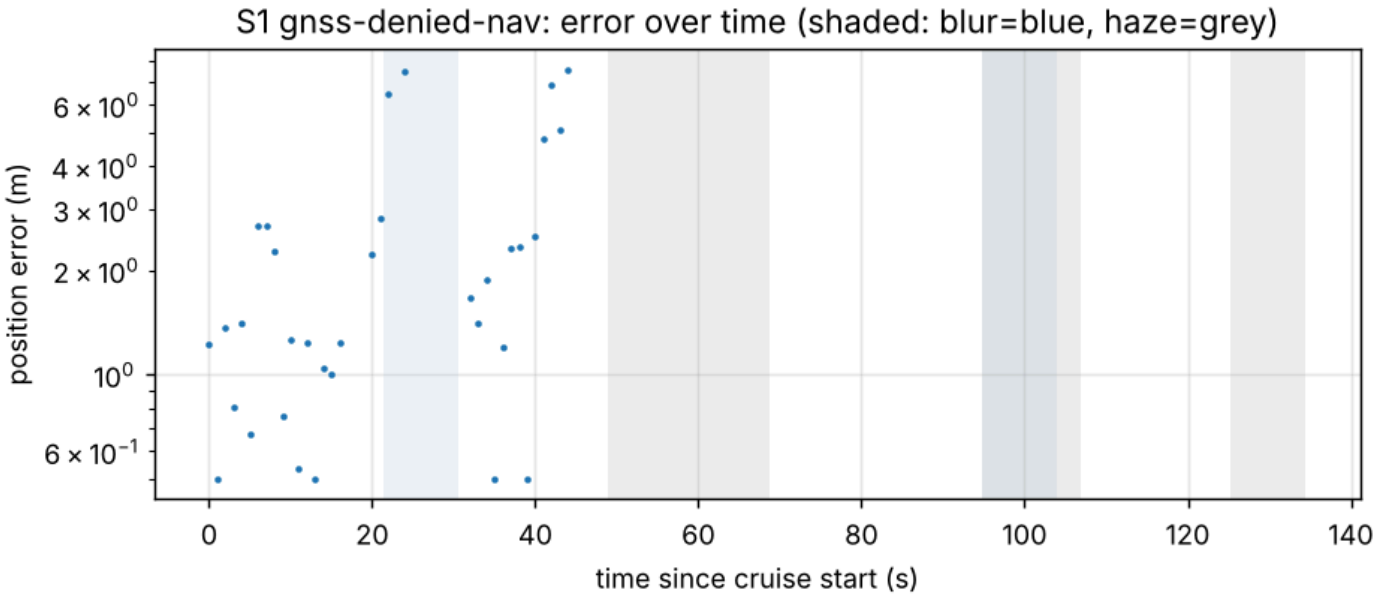
> **Degraded:** the author's fine-tuned DINOv2 weights are not published; pretrained `facebook/dinov2-base` was used. Query keypoints capped at 1,024 and patch 140 m (author: 2,048 and 250 m).

Architecture (real data flow, vector diagram)

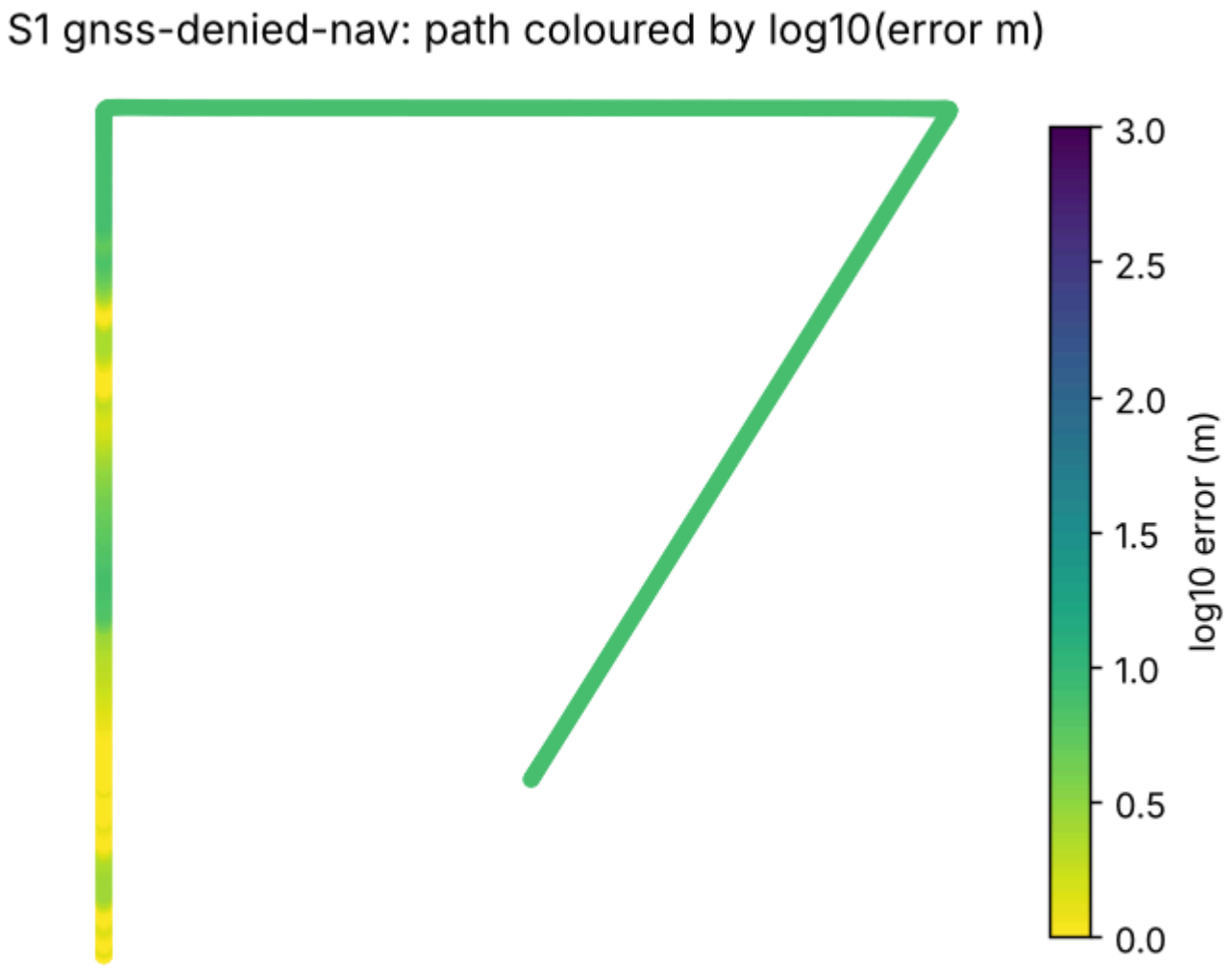




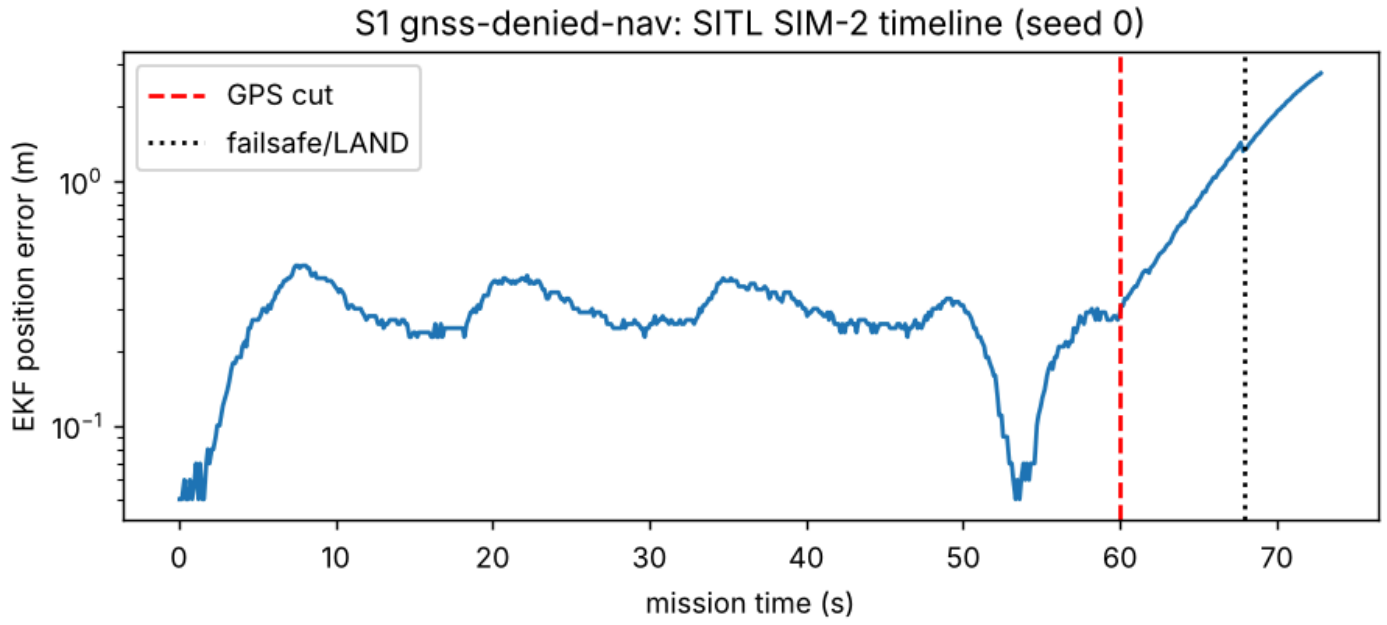
S1 koksalan/gnss-denied-nav: trajectory (dataset: synthetic NIDAR-like nadir; run S1-T1-20261023-s0; 1 Hz common frames or full run as named)



S1 koksalan/gnss-denied-nav: position error over time, blur (blue) and haze (grey) windows shaded



S1 koksalan/gnss-denied-nav: path coloured by error



S1 koksalan/gnss-denied-nav: SITL SIM-2 timeline, seed 0 (GPS cut at 60 s)

Analysis written by the solution's test agent (from the code actually run)

S1 analysis: koksalan/gnss-denied-nav on synthetic NIDAR-like data

All numbers below come from runs under `results/s1/nidar/<run-id>/` (harness `metrics.summarize`, same metric code as the other solutions). Data: **synthetic NIDAR-like** nadir render (gimbal-stabilised, 200 m AGL, 12 m/s, 1280×800 gray, 0.18 m/px), reference map Esri z19 0.283 m/px. **The video was rendered from the same map, so the appearance gap is optimistic.** Weights are the **pretrained (not fine-tuned) DINOv2-base: every S1 result is DEGRADED** (the author's fine-tuned weights are not published). CPU only, 2 threads, box shared with other agents (latencies are inflated and noisy; the lead's alone-run numbers supersede them). Query keypoints limited to 1024 (repo default 2048) to make CPU runs feasible.

1. How it works (from the code that ran)

`adapter.py` → `gdnav.localizer.Localizer.localize` per frame: 1. **Prior geometry**: the frame is rotated north-up by the compass yaw (`query.query_warp`, `heading_sign * yaw + offset`) and scaled to a fixed ground size from barometer height / focal length; a circular mask of 140 m diameter (inscribed circle of our frame) is cut. Without heading and height the method cannot start (it assumes north-up and known scale). 2. **Retrieval** (global mode only): DINOv2-base (CLS + GeM pooled patch tokens, 1536-d, L2-normalised) of the 224 px north-up patch; dot product against the 1225 pre-embedded map tiles (50 m stride); top-10 tiles. (Exact search done by a torch matmul; FAISS is not used in the localizer.) 3. **Fine matching**: SuperPoint keypoints of the warped query (up to 2048; we used 1024) are matched with

LightGlue against a 450 m window of keypoints precomputed for the whole map (`MapFeatures`, kept in RAM), for each candidate; RANSAC fits a similarity transform (rejects rotation >15 deg or scale change >25 %), the candidate with most inliers wins. 4. **Pose**: inlier matches go to a homography RANSAC (4 px) and planar PnP (IPPE + LM refinement) on the ground plane: camera position (lat/lon), height above ground and off-nadir tilt. 5. **Confidence**: fix is trusted with >= 25 inliers. 6. **Consistency gate** (`sim/visual_gps.py::consistency_gate`, copied verbatim into the adapter): needs >= 50 inliers (>= 80 when no fix accepted for 5 s, "degraded"), |PnP height - baro| <= 8 m, |PnP tilt - IMU tilt| <= 4 deg (tilt skipped in degraded mode). 7. **Tracking mode**: once a fix is accepted, the next frame skips retrieval and matches only one 600 m window around the previous fix (`track_window_m`); a gate rejection keeps the prior, a "genuinely lost" frame (< 25 inliers) clears it and the next frame searches globally again. 8. Not in the adapter (out of scope): visual odometry between fixes (LK optical flow, `odometry.py`), ArduPilot GPS_INPUT bridge (`sim/visual_gps.py`), learned confidence model, detection/planning. The repo's "Lucas-Kanade fills gaps" part is therefore **not evaluated**; between fixes our adapter reports no position.

2. Data-flow stages (for the diagram)

camera frame (gray→RGB) + compass yaw + baro height → **north-up/scale warp (OpenCV affine, circular mask)** → **DINOv2-base embedding (PyTorch, HF transformers)** → **tile-DB top-10 (dot product; DB built offline from map tiles, 1225 × 1536 float)** → **SuperPoint (LightGlue pkg) on query** → **LightGlue match vs precomputed map keypoints window (x10 candidates, or x1 in tracking)** → **RANSAC similarity (OpenCV)** → **homography RANSAC + planar PnP IPPE/LM (OpenCV)** → **consistency gate (baro height, IMU tilt, inlier count)** → accepted fix lat/lon/height → (repo, not run here) EKF3 via MAVLink GPS_INPUT. Tracking feedback: accepted fix → prior window for next frame.

3. Results (synthetic NIDAR-like, nadir)

Fix rate counts accepted fixes over ALL frames; precision = accepted fixes within 50 m (all accepted fixes were within 8 m in every run, so precision is 100 % and false fixes = 0 everywhere).

Test	run id	frames	accepted	fix rate	median err	p95	max	false fixes >50 m	latency p50 (loaded CPU)
S1-T1 global search, no prior, 1 Hz subsample	S1-T1-20261023-s0	153 (stride 5)	34	22.2 %	1.40 m	7.05 m	7.6 m	0	34.0 s
S1-T2 tracking after first lock	S1-T2-20261023-s0	153	45	29.4 %	1.50 m	5.06 m	5.9 m	0	track 9.7 s / global 18.4 s
S1-T2full tracking, full 5 Hz (**SITL file**)	S1-T2full-20261023-s0	764	221	28.9 %	1.56 m	4.01 m	7.4 m	0	track 5.4 s / global 9.1 s

Test	run id	frames	accepted	fix rate	median err	p95	max	false fixes >50 m	latency p50 (loaded CPU)
S1-T5 tracking, gate OFF	S1-T5-20261023-s0	153	46	30.1 %	1.53 m	5.43 m	5.9 m	0	-
S1-T6a no baro, no heading (nominal 200 m, heading 0)	S1-T6a-20261023-s0	77 (stride 10)	16	20.8 %	1.66 m	4.30 m	6.6 m	0	12.3 s
S1-T6b baro kept, heading removed	S1-T6b-20261023-s0	77	16	20.8 %	1.69 m	4.46 m	5.3 m	0	10.2 s
S1-T9 failure injection, early windows (tracking)	S1-T9-20261023-s0	61 (t<=60 s, 3 dropped by the injector)	26	42.6 %	1.34 m	4.87 m	5.9 m	0	11.0 s
S1-T9late injection after lock is lost anyway	S1-T9late-20261023-s0	153	35	22.9 %	1.30 m	3.67 m	5.9 m	0	13.2 s

Notes on each test: * **T1 (global, 1 Hz)**: the headline: **only 22 % of frames get a fix**, all inside the first 44 s (leg A→B, plus a few frames). Median 1.4 m on the fixes it makes (optimistic: same map). Time to first fix 0 s. T1 at 5 Hz (764 frames x ~35 s each) was not run: ~7 h of CPU. As agreed in the brief we used 1 Hz; **the 5 Hz estimates.csv for SITL is `S1-T2full` (tracking enabled, 5 Hz), not a no-prior run**. In tracking mode the lead should expect position only until t = 44.2 s; afterwards S1 outputs nothing (accepted = False) for the rest of the flight. * **T2 tracking**: a tracking frame costs about half of a global frame (5.4 vs 9.1 s on the quieter box, 9.7 vs 18.4 s in the loaded run): no DINOv2, one LightGlue call instead of ten. Fix rate rises 22 → 29 % (the 1 s gaps in haze are not bridged; lock is lost at t = 45 s and never regained). * **T5 gate off**: no change in false fixes (0 → 0) because there were no wrong matches to reject; the gate rejected one correct fix (t = 45 s, 33 inliers < 50, degraded rule 80). On this data the gate is neutral to slightly harmful; its value (author: rejected all 16 fixes > 50 m off) cannot be shown because the failure mode here is "no match", not "wrong match". Inlier counts of correct fixes are 127-172, far above the 25 vs 50 thresholds, so the 25-inlier rule alone already separates them. * **T6 no hints**: fix rate 16/77 (20.8 %) for both variants versus 19/77 (24.7 %) for T1 at the same even-second frames (3 fewer fixes, error unchanged). Because the baseline only localises on the first leg where the true heading is ~0 deg (north) and the height is the nominal 200 m, **the test cannot show the dependence on hints**: it only shows the method tolerates heading errors < 15 deg and height errors of a few metres, and it silently depends on those hints elsewhere (heading error > 15 deg is rejected by the similarity gate). Without the barometer the height/tilt part of the gate is not available (only the inlier count). * **T9 failure injection** (tracking mode, 1 Hz; windows 3-9 s wrong heading +90 deg, 12-20 s 50 % random frame drop, 33-38 s low light (gain 0.12, gamma 1.8, noise), 40-45 s synthetic Gaussian blur sigma 8 px; video blur 21-31 s, haze 49-69 s): * wrong heading +90 deg: lock lost at the first corrupted frame (t = 3 s), no false fix accepted (rotation > 15 deg is rejected); global search re-acquired at the first correct frame (t = 9 s): **recovery time 0-1 frame (1 s)**. * 50 % frame drop: **no effect**: the last fix stays as prior and tracking resumed at the next frame (recovery 0 s; fixes 12, 16...). * low light: lock lost at once (t = 33), no fix during the 5 s window, re-acquired globally at t = 38 (first clean frame):

recovery 1 s after the window. * blur sigma 8: lock lost (t = 40), at t = 45 a correct but weak fix (37 inliers, 2.9 m) was rejected by the degraded gate (needs 80) and lock was never regained before the natural haze (49 s): **no recovery within 15 s** (here it is the retrieval, see below). * natural blur 21-31 s: T2 (tracking) got fixes on 9 of 18 frames inside the video blur windows, T1 (global) on 2 of 18 (all haze frames: 0 of 41 in both). * In T9late (injections at 35-45 s etc.) lock was lost at the blur (35 s) and never regained. * **T3 real-time, T4 constrained**: implemented, not run (lead runs alone). **T4 export attempt** (single process CPU, 2 threads, loaded box; results/s1/nidar/S1-T4-20261023-s0/t4.json): * DINOv2-base retriever: torch fp32 944 ms → ONNX Runtime fp32 602 ms (max abs diff 4e-7, 346 MB) → ORT dynamic INT8 214 ms (90 MB) **but INT8 embeddings cosine to fp32 only 0.58: unusable** (the author uses "sensitivity-aware" INT8, not reproduced; not tried). * SuperPoint: only the dense CNN exported (800×1280 input): torch 2.59 s → ORT 1.44 s; NMS/top-k/descriptor sampling remain python. No TensorRT / Jetson path (author states none). * LightGlue: TorchScript ONNX export fails twice (`IndexError: list index out of range` in torch.onnx symbolic helper, opset 16 and 17, with adaptive depth/width disabled); `torch.onnx.export(dynamo=True, opset 18)` works (after installing onnxscript), 46.6 MB, fixed depth (no early exit): torch 2.40 s → ORT 2.12 s for 512 × 1024 keypoints (lightglue_retry.json). Adaptive pruning is lost in the export. * Overall, one tracking frame ~ SuperPoint 3 s + LightGlue 2-3 s on 2 CPU threads: ~6 s per fix at best, ~1/30 of the 5 Hz frame rate; real-time is not reachable on this CPU without a large model/resolution cut. * **T7 fallback datasets**: not run. Attempt: GitHub UAV-VisLoc readme reachable (links to Baidu, blocked); HF search found mirrors `haiduonghuynhle/UAV_VisLoc_dataset` (one 17.7 GB zip, AGPL tag) and `Demyd/UAV_VisLoc_dataset` (empty), `VERYBC/XIAN_Visloc` (not checked): too large for the disk allowance, so nothing fetched. A single flight could perhaps be range-extracted from the zip; not attempted. * **T8 SITL**: lead.

Plausibility checks (results/s1/alignment_check.txt): shifting the truth by +-0.1 s makes the error worse (1.56 → 1.8-1.9 m), so time alignment is right; mean bias N 0.01 m, E -0.66 m; fixes come from image matching (PnP height 196.7 m vs baro 197.1 m), truth is only used by the driver for hints (baro, yaw) and by `metrics.py`. The 1-2 m error is credible for same-map rendering with 0.5 m/px matching; real-imagery appearance gap will make it much worse (the author's own median is 24 m).

4. Failure analysis (frames in results/s1/figures/failures/t.png: north-up query | expected tile at truth | top-1 retrieved tile; stats in failures.json)

t (s)	condition	what happened	cause
25	video blur 21-31 s (peak 3 px)	no fix in global mode; the true tile has retrieval rank 23/1225, top-10	blur removes texture → DINOv2 descriptor drifts; retrieval miss

t (s)	condition	what happened	cause
		closest tile 311 m away	
52	haze 49-69 s	no fix; true tile rank 15 (top-10 nearest 162 m)	haze/low contrast; retrieval miss
60	haze	true tile IS rank 1 (8 m) but 0 inliers	haze contrast kills SuperPoint keypoints/LightGlue matches
75	clear air, forest/ river	true tile rank 1 (12 m), 0 inliers	vegetation/water texture, gray query vs colour map → few repeatable keypoints (see t75.png)
100	blur 95-104 + haze 95-107	true tile rank 212 (nearest top-10 53 m)	combined degradation; retrieval miss
130	haze 125-134	true tile rank 11 (just outside top-10), top-1 887 m away	haze; the fine-tuned retriever is exactly what is missing
145	clear	true tile rank 4 (26 m) but 0 inliers	matching failure on low-texture terrain (second half of the route)
Summary of causes: (a) pretrained retrieval misses the right tile in the 10 candidates (blur/haze, ranks 11-212); (b) even with the right tile at rank 1, the SuperPoint+LightGlue stage finds 0-6 inliers on forest/water/haze; (c) after the lock is lost at t = 45 s the system never regained it in the remaining 108 s. By terrain: 0 accepted fixes after t = 44.2 s over ~108 s / ~1.2 km of path.			
Frames with < 25 inliers in the gate counted as "genuinely lost" and cleared the tracking prior, so one haze frame forces an expensive global search that itself usually fails (no gap bridging; the repo's LK odometry would be needed).			

5. Comparison with the author's claims

claim (README)	ours	verdict
median 24 m, 97 % within 100 m on held-out real UAV-VisLoc flights, whole-map search without prior (fine-tuned DINOv2)	median 1.4 m on the fixes made, but only 22 % of frames get a fix (global); the README's own *pretrained DINOv2-B + LightGlue* row is 30 m median / 69 % / 75 % (fix rate numbers)	**not reproduced / partly consistent**: our weights are the author's "pretrained" baseline, which the author also reports as much weaker (top-1 retrieval 18 %). Our accuracy is optimistic (same map); our coverage is much lower than 69 % because of blur/haze, a 140 m footprint (the author's 250 m patch has more context) and gray-vs-colour imagery
6.3 m median error with GNSS jammed (SITL, Gazebo, EKF3 fusion), 840 m without	not run by us (SITL by the lead)	not tested
VO halves worst-case error (86 → 32 m)	VO not run (adapter excludes it)	not tested
consistency gate rejected all fixes > 50 m	no wrong fixes occurred, gate neutral/slightly harmful (1 correct fix	cannot be confirmed on our data

claim (README)	ours	verdict
off	rejected)	
fast path: 24 fixes/s	0.1-0.2 fixes/s on 2 CPU threads (GPU claimed)	not reproducible on CPU; ~100-200x slower
no Jetson/TensorRT	confirmed; ONNX export works for DINOv2 and (dynamo) LightGlue, SuperPoint partial; dynamic INT8 breaks the descriptor	consistent
Differences: synthetic data, nadir stabilised 200 m, different camera (FOV 60 deg), pretrained weights, CPU.		

6. Verdict

- Strengths: very good accuracy when a fix exists (1-2 m here; PnP gives height and tilt that can be cross-checked with baro/IMU, a nice sanity gate); drift-free absolute fixes; clean modular code, matches its documentation; tracking mode halves the cost; heading and frame-drop faults are handled gracefully (no false fixes at all in any test); extensive training/evaluation scripts.
- Weaknesses: with off-the-shelf DINOv2 only ~1 frame in 4 gets a fix and nothing after t = 44 s; heavy dependency on compass heading (+-15 deg) and baro height; no published weights (cannot be used as delivered); heavy compute (needs GPU; 9-35 s per frame on 2 CPU threads); nothing between fixes without the VO part; thresholds tuned on one flight; SuperPoint weights are non-commercial; requires EPSG:4326 GeoTIFF.
- Use when: a GPU-equipped companion computer, a fine-tuned retriever for the operating area/season, reliable compass + barometer, fixed-wing/multicopter at moderate altitude over textured terrain, and a flight controller that accepts a second GPS. Not when: Jetson-class hardware without porting, low-texture terrain (forest, water, haze), no compass, or the weights cannot be trained.
- Risks observed: lock loss is silent and permanent in haze (no recovery for 108 s); gate rule "degraded" demands 80 inliers, which rejects good fixes after a gap; a wrong heading hint (>15 deg) blocks all fixes; prior gets cleared by any weak frame; memory ~2.4-3 GB; build 15 min on CPU for 1.9 km^2 (scales with area); harness-side: frame sorting by string.

7. Files

`adapter.py` (Adapter + MapShim + gate), `driver.py` (T1..T9 driver, injection, subsampling, realtime mode), `t4_constrained.py`, `t4_lightglue_retry.py`, `failure_figs.py`, `run_all.sh` (queue used), `camera.json`, `lock.txt`, `SETUP.md`. Oblique-camera calibration

runs of the discarded first data set: results/s1/nidar_oblique/ (body-fixed 30-degree-oblique camera: stress case only, 12-frame calibrations: fix on 2/12 frames, errors 1.5-3.5 m).

8. Lead commands (run alone, serial, from /home/user/Nexmesh/gpsdenied-eval)

```
E=solutions/s1_gnss_denied_nav
# S1-T3 real time (5 Hz, 153 s wall, drops frames while busy)
OMP_NUM_THREADS=2 $E/env/bin/python $E/driver.py --id S1-T3 --mode realtime --
config '{"query_kp":1024,"tracking":true}'
# S1-T4 constrained: ONNX variant timings (set threads / taskset for the
constrained profile)
OMP_NUM_THREADS=2 taskset -c 0,1 $E/env/bin/python $E/t4_constrained.py --id S1-
T4 --threads 2
# and constrained replay of the pipeline itself
OMP_NUM_THREADS=1 taskset -c 0 $E/env/bin/python $E/driver.py --id S1-T4rt --
mode realtime --config '{"query_kp":1024,"tracking":true}'
```

SITL estimates: `results/s1/nidar/S1-T2full-20261023-s0/estimates.csv` (5 Hz, 764 rows; accepted only for t <= 44.2 s). The tile index cache must exist (`cache/`, ~135 MB, git-ignored, on this machine); first start otherwise takes 15 min.

Setup account

S1 setup: koksalan/gnss-denied-nav (commit 572b9ffe..., see COMMIT.txt)

CPU: Intel Xeon @ 2.10 GHz, 4 cores shared (we used 2 threads, `nice`), 15 GB RAM, no GPU. Python 3.12 (uv venv; system python is 3.13). Wall-clock from clone to first working output (frame 0 localised to 1.0 m): about 25 min (of which 15 min is the one-off tile index build). Total agent time including all tests: roughly 9 h (CPU-bound). Failed or changed steps: 6 (listed below).

Commands that worked

```
export UV_NO_CACHE=1                        # disk is tight (cleaned 2 GB uv
cache afterwards)
cd solutions/s1_gnss_denied_nav
uv venv --python 3.12 env && . env/bin/activate
uv pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu     # torch 2.14.1+cpu
```

```
uv pip install "transformers>=4.45" faiss-cpu rasterio opencv-python-headless
numpy scipy pandas matplotlib tqdm \
   kornia timm onnx onnxruntime scikit-learn joblib pymavlink imageio "lightglue
@ git+https://github.com/cvg/LightGlue.git"
uv pip install onnxscript                  # only for the LightGlue ONNX
(dynamo) export attempt
uv pip freeze > lock.txt                   # 74 packages
# first run builds + caches the tile DB and map keypoints (15 min), later runs
start in ~20-40 s:
OMP_NUM_THREADS=2 nice python solutions/s1_gnss_denied_nav/driver.py --id S1-T1
--stride 5 --config '{"query_kp":1024}'
```

Skipped on purpose (out of scope): ultralytics/YOLO, stable-baselines3, gymnasium, fastapi/uvicorn (detection, RL planner, ground station). Not needed for localisation.

Weights (DEGRADED)

- Retriever: the repo has NO released fine-tuned weights (`outputs/` is git-ignored; README only shows training commands that need UAV-VisLoc, hosted on Baidu). Training was not feasible: no VisLoc data on disk (a 17.7 GB HF mirror `haiduonghuynhle/UAV_VisLoc_dataset` exists but exceeds the disk allowance; free disk was ~2.7-13 GB) and CPU fine-tuning of DINOv2-B would take far longer than 45 min.
- So: **pretrained `facebook/dinov2-base` (HF, 86.6 M params, 346 MB fp32 ONNX / ~330 MB HF cache), pool `cls+gem`, no fine-tune. All S1 results are flagged DEGRADED.** The author's own table (README) gives pretrained-DINOv2-B top-1 retrieval of only 18.2 % vs the fine-tuned model; we reproduce that weakness.
- SuperPoint `superpoint_v1.pth` (5 MB) and LightGlue `superpoint_lightglue` (45 MB) are downloaded by the `lightglue` package from github.com/cvg/LightGlue releases.

Index build (map data/nidar/map_esri_z19.png), recorded in cache/build_*.json

- Map resampled once from Web-Mercator to a linear lat/lon grid in RAM (the repo requires an EPSG:4326 GeoTIFF; `MapShim` in adapter.py replaces `SatelliteMap`). Native 0.283 m/px kept; the repo itself resamples to 0.625 m/px (retrieval tiles, 224 px) and 0.5 m/px (SuperPoint keypoint map) and warps each camera frame to the same scale, so no resampling to the camera GSD (0.18 m/px) is needed. Ratio map/camera = 1.57; camera → 0.5 m/px is a 2.77x down-scale.
- 1225 tiles (140 m patch, 50 m stride), 1536-d descriptors (cls+gem), descriptor DB 7.5 MB. Build time 685 s (DINOv2-B on CPU, 0.56 s/tile under load). The onboard localizer does exact dot-product search with `torch` (`db @ q`), NOT FAISS; FAISS (`IndexFlatIP`,

same exact search) appears only in `gdnav/evaluation.py` and `scripts/eval_int8.py`. README's "FAISS index" is therefore only partly accurate.
- SuperPoint keypoints of the whole map precomputed once: 4970×4970 px image at 0.5 m/px, 47 958 keypoints (up to 2600 per 1024 px tile), 124 MB in RAM, build 202 s.
- Peak RSS of a replay process ~2.4-3 GB.

Camera config (camera.json, repo format of configs/camera_visloc*.json)

`focal_px` 1108.51, `heading_sign` +1, `heading_offset_deg` 0 (the video has image-up = heading; verified: matches succeed at 133-172 inliers with 1 m error on the first leg; the repo's VisLoc configs use sign -1 for their dataset). `patch_m` = 140 m (the repo default 250 m would exceed the inscribed circle of our 1280×800 frame at 200 m: 800 px x 0.18 m = 144 m).

Failures / deviations (each counted)

1. `SatelliteMap` needs an EPSG:4326 GeoTIFF → wrote an in-memory shim (no GeoTIFF re-reads, as README recommends).
2. Repo default patch 250 m does not fit our camera → 140 m.
3. CPU cost: stock config (top-10 rerank, 2048 query keypoints, 450 m windows) took 35-60 s/frame at the beginning (contended box; SuperPoint upsamples the 280 px query to 1024 px; ten LightGlue runs ~3 s each) → `query_kp=1024` for all runs (calibration runs in results/s1/nidar_oblique/ and S1-calib-nadir; on oblique-data calibration frames 2048 vs 1024 keypoints gave 1.0 vs 1.5 m on frame 0 and the 2048 run died before finishing, so equivalence is only weakly checked). Lower `search_m=250` broke correct matches (inliers 11 vs 37), rejected.
4. harness/run_replay.py sorts frame names as strings (100.0 before 20.2) → driver sorts numerically; yaw hint is in degrees (adapter has `yaw_unit`).
5. First data set (body-fixed, 30 deg oblique camera) replaced by the lead with a nadir render after ~3 h of my runs; those runs were stopped and not used (calibration logs kept in results/s1/nidar_oblique/).
6. A calibration process (2048 keypoints) died (probably OOM while other agents were running); rerun not needed.

Reproducibility verdict

Setup with the repo as published: reproducible for localisation code; **results of the author are not reproducible** because weights, training data and dataset links are not published (Baidu only), and no pinned requirements (requirements.txt has only lower bounds such as `torch>=2.4`; `lightglue` from git HEAD). Seeds: no stochastic component in inference

except RANSAC (OpenCV, default seeding); `--seed 0` set for numpy/torch, T9 frame-drop uses numpy rng seed 0.

Licences

component	licence
koksalan/gnss-denied-nav	MIT
DINOv2 (facebook/dinov2-base)	Apache-2.0
LightGlue (cvg) code	Apache-2.0; its SuperPoint-LightGlue weights follow the SuperPoint licence
SuperPoint (Magic Leap) weights	Magic Leap research licence (non-commercial)
PyTorch / torchvision	BSD-3
transformers	Apache-2.0
faiss-cpu	MIT
OpenCV (headless)	Apache-2.0
ONNX Runtime / onnx	MIT / Apache-2.0
rasterio, kornia, timm	BSD-3 / Apache-2.0 / Apache-2.0
UAV-VisLoc dataset (not used)	CC BY-NC-SA (per dataset page; verify)
Esri World Imagery map (our benchmark map)	Esri terms, benchmark only, not redistributed
Note: the SuperPoint non-commercial licence is a blocker for commercial deployment.	

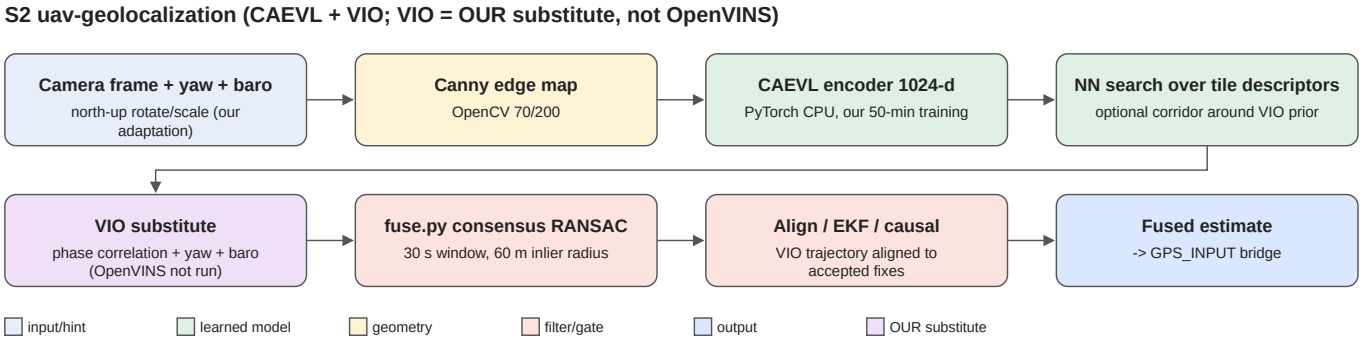
Documentation rubric (0-3)

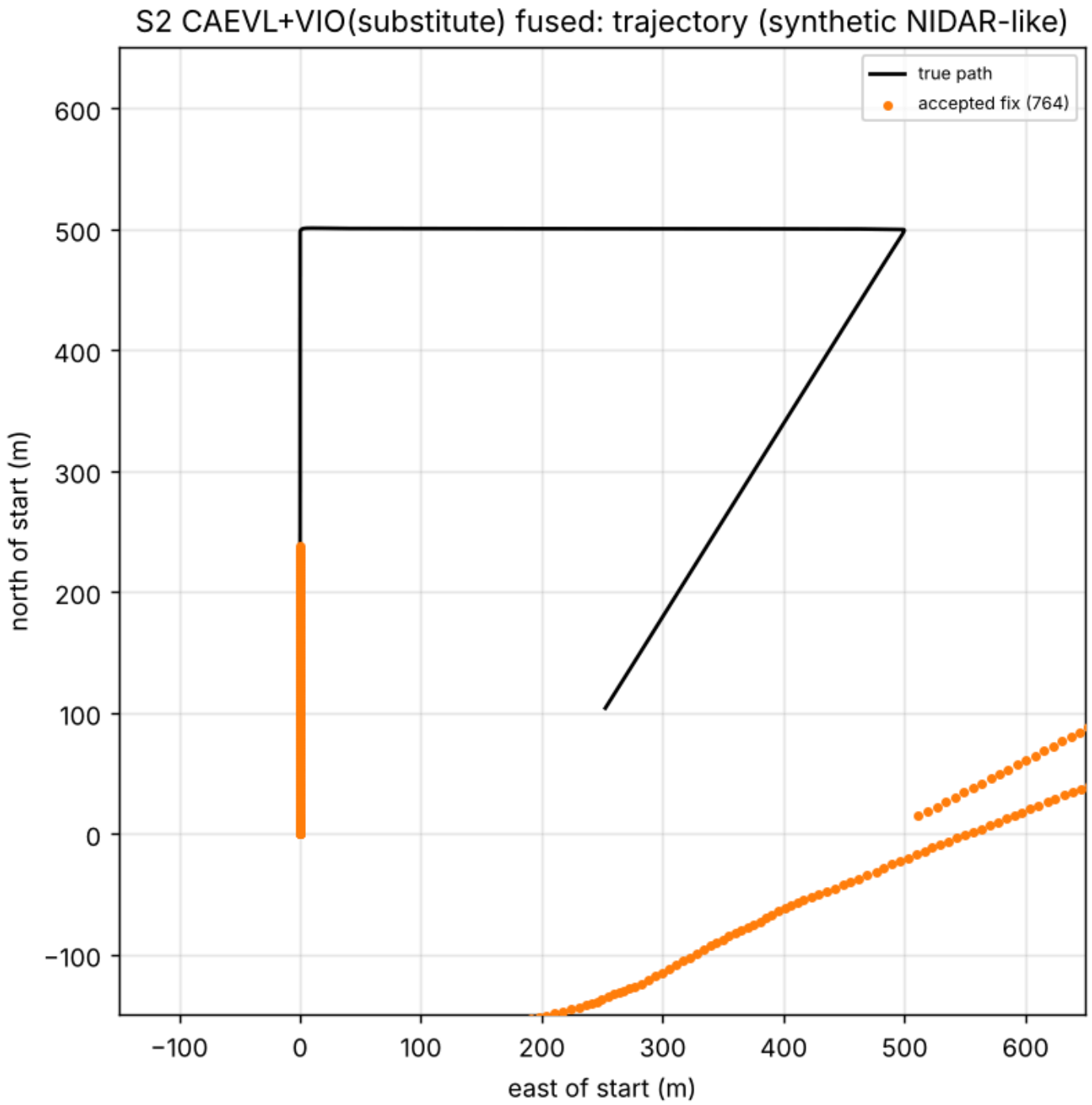
- Install steps work as written: **1** (requirements.txt has no versions/CPU path; README has no install section for the localizer; GeoTIFF/EPSG:4326 requirement implicit; weights absent).
- Config explained: **2** (camera json + LocalizerConfig docstrings; thresholds explained in code comments, but `patch_m` vs camera footprint is not documented).
- Results reproducible: **1** (scripts exist, but data/weights unobtainable).
- Algorithm described: **3** (README + module docstrings are detailed and match the code).

S2 uav-geolocalization (CAEVL + VIO)

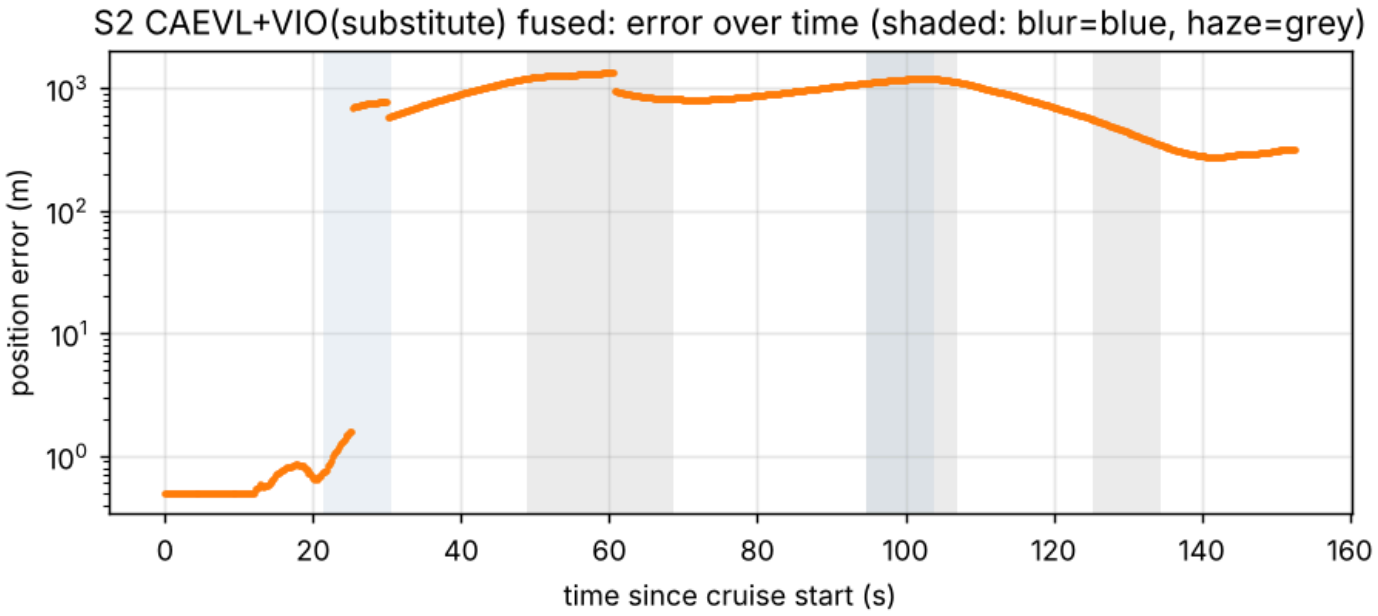
> **Substitute inputs:** the VIO layer is **our substitute** (phase correlation + compass yaw + barometer), not OpenVINS (it built but could not be run within the disk limit). The CAEVL encoder is ours, trained about 50 minutes on a CPU for 2 epochs on 1,800 tiles (the authors: up to 20 epochs on ~250k tiles on a GPU).

Architecture (real data flow, vector diagram)

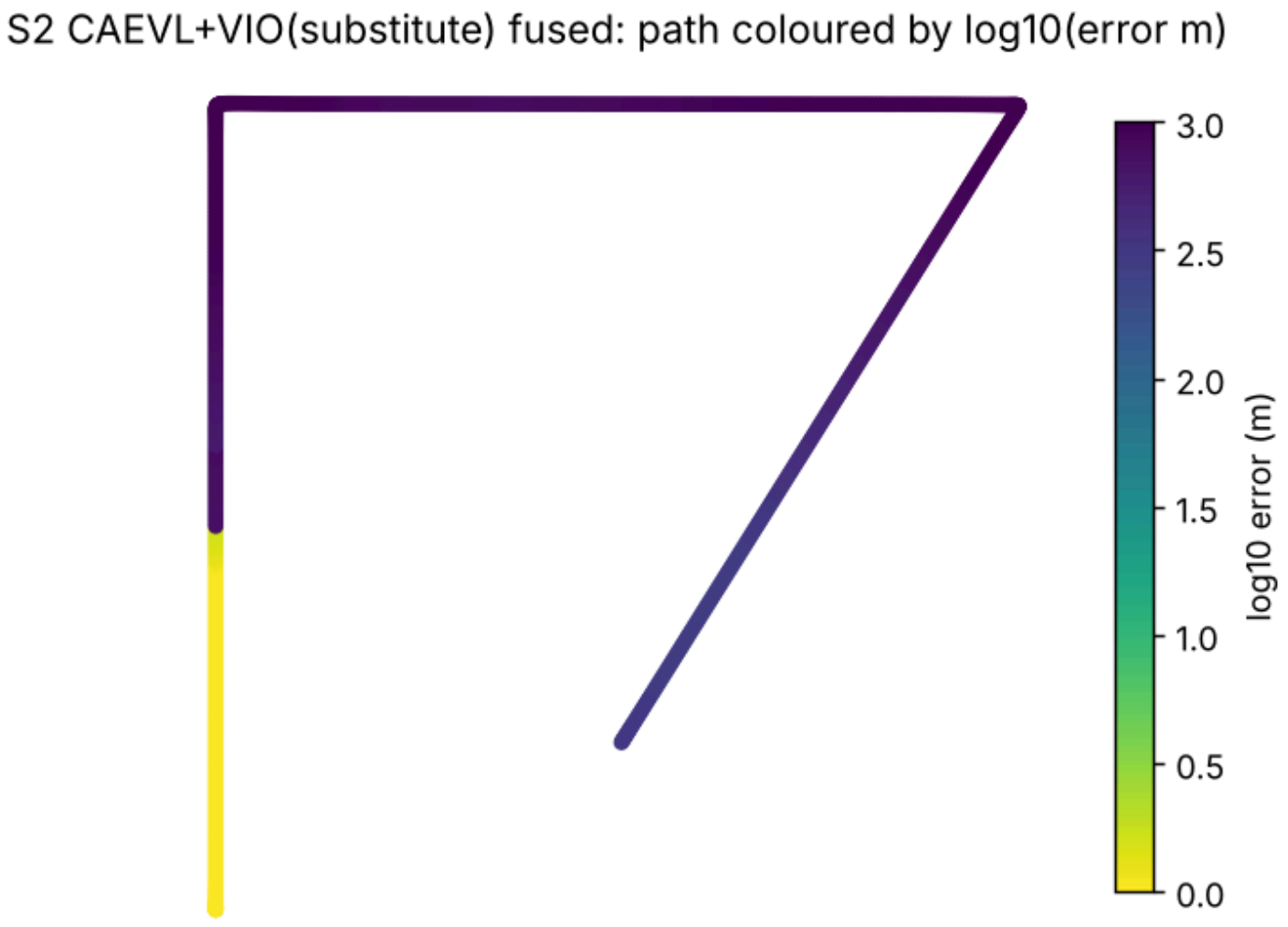




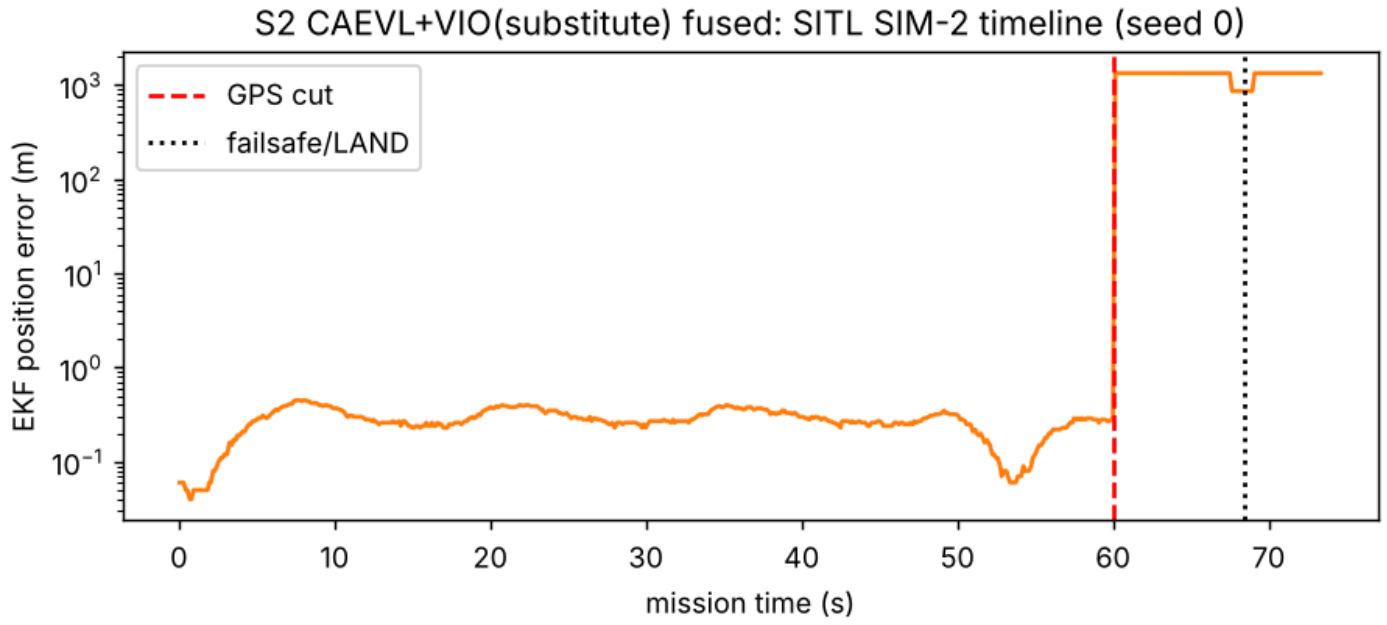
S2 uav-geolocalization (CAEVL + VIO): trajectory (dataset: synthetic NIDAR-like nadir; run S2-T3-20261023-s0; 1 Hz common frames or full run as named)



S2 uav-geolocalization (CAEVL + VIO): position error over time, blur (blue) and haze (grey) windows shaded



S2 uav-geolocalization (CAEVL + VIO): path coloured by error



S2 uav-geolocalization (CAEVL + VIO): SITL SIM-2 timeline, seed 0 (GPS cut at 60 s)

Analysis written by the solution's test agent (from the code actually run)

S2 analysis (uav-geolocalization: CAEVL + VIO + fuse.py)

Data label: **synthetic NIDAR-like** (map and frames from the same Esri imagery; optimistic). **VIO layer = OUR-VIO-SUBSTITUTE, not OpenVINS** (OpenVINS builds, was never run; see SETUP.md). **Primary data = gimbal-stabilised NADIR render (data/nidar). The earlier body-fixed 30-degree-oblique render is kept as a stress result (appendix, results/s2/nidar_oblique/). CAEVL encoder = our 2-epoch CPU-trained model (about 50 min), far weaker than the authors' GPU model.** The central result must be read with both caveats.

How it works (from the code run)

Stages: 1. Map prep (offline): north-up tiles cut from the reference map at a fixed ground size (60/120/240 m, stride half a tile), resized to 256×256 gray. 2. Edge descriptor: OpenCV Canny (70/200) on the tile → CNN encoder of CAEVL (`caevl.ae` encoder, 37.5 M params, 1024-d, PyTorch CPU) → L2-normalised descriptor. Computed once per tile and stored as a float32 matrix (3.7 MB for 900 tiles). 3. Training (offline, reference imagery only): stage 1 autoencoder reconstructs Canny maps; stage 2 VICReg pulls together a clean tile and a randomly degraded copy (rotation, zoom, blur, vignette, lens mask) with variance/covariance terms to avoid collapse. 4. Query: camera frame → (our adaptation) rotate to north-up by compass yaw and scale by baro altitude (the nadir data needs no tilt correction; for the oblique data a full ground-plane rectification through IMU-estimated roll/pitch was used) → same Canny + encoder → cosine nearest neighbour (matrix product; `faiss` not needed at

this size) → tile centre = absolute fix (corrected for the camera-axis offset). 5. VIO (relative motion): author uses OpenVINS (MSCKF, ROS 2). We used OUR-VIO-SUBSTITUTE: north-up ground images + `cv2.phaseCorrelate` between consecutive frames + compass yaw + baro height. 6. Fusion (`fuse.py` causal): keep a trailing 30 s buffer of fixes with the VIO position at each fix time; RANSAC on two-fix similarity transforms (inlier radius 60 m, at least 3 agree) keeps only mutually consistent fixes; all accepted fixes ever seen are fitted with a least-squares similarity (scale, yaw, offset) once they span 300 m; output = transform applied to the current VIO position. Modes `align` (offline windowed refit) and `ekf` (4-state filter, chi-square gate) are alternatives. The retrieval score is never used. 7. Optional corridor search: restrict candidate tiles to a disc around the motion prior.

Results (primary: nadir data; runs under results/s2/nidar//)

Test	Result
S2-T1 VIO only (substitute, 764 frames at 5 Hz, path 1464 m)	final error 7.9 m, RMSE 5.0 m, 5.4 m per km of path (0.54 %). T1b degraded (3 deg yaw bias, -4 % scale): final 29.0 m, RMSE 36.5 m, 19.8 m/km. Uses noise-free truth yaw and baro altitude, so this is optimistic; real OpenVINS drift unknown.
S2-T2 retrieval only (120 m tiles, 900 tiles, 764 frames, full map)	recall@50 m 3.8 %, @100 m 4.6 %, @250 m 9.2 %; median error 656 m, p95 1146 m; true tile median rank 64 of 900; true place in top-20 within 100 m: 43.5 %. Clear weather 6.8 % @100 m; haze 0 %, blur 0 %, blur+haze 0 %. Near chance.
S2-T3 fused causal, full-map search (estimates.csv for SITL)	fused RMSE 798 m vs VIO alone 5.0 m (whole flight); after warm-up (t >= 25.4 s) 874 m vs 5.5 m. Final 316 m vs 7.9 m. Accepted at arrival 84 fixes, 7.1 % precise (raw precision 4.6 %); fitted scale 2.95, yaw -30 deg (wrong). Fusion is far worse than VIO alone.
S2-T3c fused with a 200 m corridor around the motion prior	fused RMSE 116 m (whole flight; 129 m after warm-up at 29 s) vs 5.0 m VIO. Raw fixes in the corridor are 31.8 % within 100 m and consensus accepts 301 fixes with 36.9 % precision, but 69 % wrong is still enough to corrupt the fitted transform.
S2-T4 fuse.py batch modes on the T3 logs	causal as shipped (with its non-causal pre-filter) RMSE 804 m; align (offline) 576 m; ekf 837 m; VIO alone 5 m. The offline bound does not rescue it.
S2-T5 consensus off	RMSE 580 m (fit yaw -17 deg, scale 2.88) vs 799 m with consensus on; consensus helps slightly but >95 % of fixes are wrong and wrong fixes agree with each other. Streaming port reproduces the adapter to 2e-8 m.
S2-T6 corridor off vs on (T2 run, prior = VIO substitute)	off: recall@100 4.6 %, median 656 m, 900 candidates, 0.08 ms. 200 m radius: recall@100 35.6 % (@50 24.1 %), median 121 m, 35 candidates, 0.03 ms. 300 m radius: 18.6 % (@50 13.6 %), median 198 m. The gain is mostly the geometric prior (median error is about the corridor scale), not better descriptors; encode (about 190 ms on the loaded 2-thread box) dominates, search time is negligible.
S2-T7 tile size at 1 Hz (153 frames), stride T/2	60 m: 3721 tiles, recall@50 2.6 %, @100 4.6 %; 120 m: 900 tiles, 3.9 % / 4.6 %; 240 m: 196 tiles, 0 % / 0 % (stride 120 m caps accuracy near 85 m and one 240 m query is mostly black corners). All near chance, no tile size is meaningfully better.
S2-T3x sanity (synthetic fixes with 30 m noise, NOT CAEVL output)	The fusion layer cannot help a 5 m VIO: with the substitute VIO, 40 % good fixes still gives RMSE 5 vs 5 m (-9 %) and 10-20 % good fixes make it worse (168 m, 20 m). With a degraded VIO (RMSE 37 m): 20 % good → 12 m (67 % lower), 40 % good → 18 m (50 % lower), 10 % good → 125 m (worse). Consensus off: 344-608 m at every rate.
S2-T8 real-time / constrained	implemented (`run_s2.py t8`), NOT run (lead runs).

Test	Result
S2-T9 fallback dataset	not run: MUN-FRL unreachable, no open mirror known.
S2-T10 SITL	not run by S2.

Does fusion beat VIO alone? Is the 36 % reduction reproduced?

No, and not reproduced. On nadir data our VIO substitute is already very good (RMSE 5 m, 0.5 % of path), so there is little drift for fixes to remove, and our under-trained encoder (4.6 % of raw fixes right) feeds the fusion mostly wrong fixes, so fused RMSE is about 160x worse (798 m vs 5 m). Author claim: causal RMSE 617 → 392 m (-36 %) with a real OpenVINS that has a 34-40 % scale error, on MUN-FRL where 18 % of fixes are right. Our VIO layer is not comparable with that, so relative gain is judged only as fused vs VIO-alone here: negative. The sanity test shows the consensus + similarity-fit layer helps (50-67 % lower RMSE) only when the VIO is poor (tens of metres) and 20-40 % of fixes are good; it fails at 10 %. Claimed retrieval R@1 @100 m 48-53 % (ViLD), 17.7 % (MUN-FRL): ours 4.6 %. We cannot separate method quality from our training budget (50 min CPU, 1800 tiles, versus GPU, about 250 k tiles): that is the main confound. 13.3 Hz on CPU: not measured (lead T8); single encode about 190 ms on our loaded 2 threads.

Failure analysis (T2/T3, 120 m model; crops query | expected | matched in results/s2/figures/failures/)

By condition (recall_by_condition.csv): clear 512 frames 6.8 % recall@100; haze 160 frames 0 %; blur 46 frames 0 %; blur+haze 46 frames 0 %. Top-1 error and true-tile rank: - t=10 s (clear, take-off leg): 1090 m, rank 2, score 0.995 (confident, wrong; nearly right ranking). - t=24 s (blur): 920 m, rank 313; edges wiped by blur. - t=40 s (clear): 302 m, rank 218. - t=58 s (haze 0.6): 657 m, rank 21. - t=74 s (clear, over river/bridge, low texture): 650 m, rank 100. - t=98 s (blur + haze): 892 m, rank 240. - t=128 s (haze 0.55): 745 m, rank 320. - t=145 s (clear, last leg): 957 m, rank 46. Causes: under-trained encoder; scores uninformative (wrong matches score 0.98-0.995, as the README says); repetitive dense rooftops; blur/haze suppress Canny edges; stride 60 m caps accuracy at about 42 m. Fusion failures: everything after warm-up (t > 25 s) follows a wrong transform; error 300-1350 m.

Verdict

Strengths: tiny index (0.8-15 MB), encoder-bound runtime with negligible search, appearance-invariant concept, honest docs, design that ignores the retrieval score; a corridor prior is a cheap large gain (4.6 → 35.6 % recall@100). Weaknesses: needs a well-trained encoder per area and altitude (no released weights; a CPU hour is not enough); fuse.py is a batch tool tuned to one flight (its "causal" CLI applies a non-causal pre-filter; VIO-alone baseline is aligned with fixes from the whole flight; accepted fixes are re-

appended every step); needs yaw, altitude (and attitude); degrades under blur/haze; consensus latches onto consistent wrong matches and cannot improve a good VIO. Use when a GPU-trained encoder exists for the terrain and altitude, fix hit rate is at least 20 %, and VIO drift is large. Do not use as shipped on this data.

Risks observed

Harness: `run_replay.run` sorted frames as strings (breaks temporal adapters; S2 uses its own driver); hint yaw is in degrees (adapter `yaw_unit='deg'`). Disk/memory pressure (stage-2 training 4-8 GB RAM; docker export ENOSPC). Under-trained model must not be read as the method ceiling.

Appendix: oblique stress result (results/s2/nidar_oblique/, "body-fixed 30-deg-oblique camera")

Before the dataset fix the camera was body-fixed with 30 deg nose-down pitch. S2 handled it by IMU tilt estimation (complementary filter, median error 0.1-0.2 deg, transients to 13 deg) plus ground-plane rectification. Results then: VIO substitute RMSE 61 m (final 19 m; the error was a 40-60 m offset from the take-off tilt transient); retrieval recall@100 2.0 %, median 688 m; corridor 200 m 23.7 %; fused RMSE 790 m (fuse.py batch causal 1057, align 434, ekf 790) vs 61 m VIO; consensus off 675 m; tile size 60/120/240: 4.6/2.0/2.0 %; sanity with good fixes at 20-40 % gave 49-59 % lower RMSE than that 61 m VIO. Conclusion identical: the weak encoder breaks fusion; a poorer VIO (as in the oblique case) is where fusion has room to help. Old figures: results/s2/figures_oblique/.

Lead commands for T8 (run alone; outputs in results/s2/nidar/S2-T8--20261023-s0/)

```
cd solutions/s2_uav_geolocalization && . env/bin/activate
OMP_NUM_THREADS=4 python run_s2.py t8 --mode realtime --threads 4
OMP_NUM_THREADS=1 taskset -c 0 python run_s2.py t8 --mode constrained --cores 0
--threads 1
```

Main SITL file: results/s2/nidar/S2-T3-20261023-s0/estimates.csv (VIO-only until t=25.4 s, then fused; fused is worse than the VIO-only track in S2-T1-20261023-s0/estimates.csv, which is the better input for SITL).

Setup account

S2 setup log (uav-geolocalization = CAEVL + OpenVINS + fuse.py)

Primary results (results/s2/nidar/) are on the gimbal-stabilised nadir data; the earlier body-fixed oblique results are in results/s2/nidar_oblique/ (stress result). All results: **synthetic NIDAR-like** data (map and frames both from the same Esri imagery, so the appearance gap is optimistic). **The VIO layer in every result is `OUR-VIO-SUBSTITUTE`, NOT OpenVINS** (see "OpenVINS" below).

Components, commits, licences

Component	Source	Commit	Licence
uav-geolocalization (fusion tools, report)	NijatAbbasov06/uav-geolocalization	9983dca3e9e14df22d33173cfab6f5b3ca7e88a8	none shown on repo page / no LICENSE file
CAEVL fork	NijatAbbasov06/caevl	43a28b9ec0f31d0bf09e69bfb01802bf6cd19140	no LICENSE file in fork (README says forks keep upstream licence; upstream Tristan-Amadei/caevl @266ba3a also has no LICENSE file) → treat as all rights reserved, internal evaluation only
OpenVINS fork (branch alto_integration)	NijatAbbasov06/open_vins	71eedce9614904fbc9e972fa030b5c7fb06ee413	**GPLv3** (LICENSE file). Isolated in `openvins_gpl/` (Dockerfile, ROS bag script, config), never imported by other code
torch 2.14.1+cpu, torchvision, timm, faiss-cpu, opencv-headless, numpy …	PyPI / download.pytorch.org/whl/cpu	see lock.txt	BSD / Apache / MIT
Weights	trained by us on the NIDAR reference map (no released checkpoint exists in either repo)	`weights/stagetwo.pth` 362 MB (not committed)	n/a
Esri World_Imagery map	benchmark only		not redistributable

CPU: Intel Xeon @ 2.10 GHz, 4 cores shared with other agents (torch/OpenCV limited to 2 threads, `nice`). No GPU: a GPU run is impossible here.

Commands that worked

```
git clone <caevl fork> caevl_src ; git clone -b alto_integration <open_vins fork> openvins_gpl/src   # both clone fine via the git proxy (curl/gh API 403, git works)
python3 -m venv env && . env/bin/activate
pip install --index-url https://download.pytorch.org/whl/cpu torch torchvision
pip install numpy opencv-python-headless pyyaml tifffile tqdm timm einops faiss-cpu loguru scikit-learn Pillow scipy pandas matplotlib gdown   # gdown is imported but missing from caevl requirements.txt
python make_train_tiles.py 1800                      # 1800 train + 240 val reference tiles (60/120/240 m, random positions, 256 px)
cd caevl_src
```

```
python caevl/ae/train.py -c ../configs/ae_nidar.yml            # stage 1 (1
epoch)
python caevl/ft_stage/train.py -c ../configs/stagetwo_nidar.yml  # stage 2
(epoch 1), then again with --load-weights (epoch 2)
cd .. ; S2_TILT=on is needed only to reproduce the oblique runs.
python run_s2.py t1 ; python retrieval_eval.py --tag S2-T2 --tile 120 --
corridors 200,300 --vio <T1 vio_track.csv>
python run_s2.py t3 ; python run_s2.py t3 --tag S2-T3c --corridor 200 ; python
post.py t4|t5|t9 ; python make_figs.py
retrieval_eval.py --tag S2-T7-tile{60,120,240} --tile {60,120,240} --step 5
```

Wall clock from clone to first working output (a retrieval number): about 95 min, dominated by CPU training and a loaded machine.

CAEVL training (CPU, budget < 60 min, spent about 50 min)

Reference-only training as in the README: stage 1 autoencoder on Canny edge maps (172.5 M params, encoder 37.5 M, latent 1024), stage 2 VICReg on clean/degraded tile pairs. Data: 1800 tiles from the NIDAR map (mixed 60/120/240 m footprints so one model serves the T7 sweep), val 240. Seed 0. | step | epochs | batch | wall | train loss | val loss | |---|---|---|---|---|---|---| | stage 1 (L2, no DINOv2 perceptual loss) | 1 | 32 | 16.5 min | 0.220 | 0.170 | | stage 2 VICReg | 1 | 32 | 16.0 min | 24.5 | 19.0 | | stage 2 VICReg (resumed) | +1 | 32 | 16.0 min | 21.3 | 17.97 | Total about 49 min. The authors trained on GPU for up to 20 epochs on ~250k tiles; we used 2 epochs-equivalent on 1800 tiles, so this encoder is severely under-trained (see ANALYSIS.md: retrieval is near chance). Logs: results/s2/training/. Stage-1 checkpoint was deleted for disk (resume of stage 2 needed `load_weights_backbone: False`).

Tile database

Own script (`s2core.Retriever`; the repo's `munfrl_map_tiles.py` is MUN-FRL-filename specific and `caevl` tile tools target IGN data). North-up tiles, stride = tile/2, 256 px, descriptors L2-normalised, computed once. | tile | stride | tiles | index | build time (2 threads, loaded box) | |---|---|---|---|---| | 60 m | 30 m | 3721 | 15.2 MB | 285 s | | 120 m (default) | 60 m | 900 | 3.7 MB | 68-85 s | | 240 m | 120 m | 196 | 0.8 MB | 16 s |

Adaptations needed (not in the author's code)

1. **Camera tilt (oblique data only).** The first dataset version was body-fixed, pitched -30 deg in cruise. For that data `s2lib.query_crop_att` resampled frames onto a north-up ground plane through full attitude (roll/pitch from our own complementary filter on imu.csv, `ahrs.py`, set `S2_TILT=on`); those results are in results/s2/nidar_oblique/. The

current primary data (data/nidar) is a gimbal-stabilised nadir render, so roll=pitch=0 (default), only compass yaw (hint, in DEGREES) and baro altitude are used.
2. **Streaming fusion.** fuse.py is a batch CLI. `s2core.StreamFuser` re-implements the loop of `causal_mode` step by step, importing `causal_consensus`, `fit_similarity`, `rotz` unchanged from repo/tools/fuse.py. It deliberately omits the CLI's whole-flight consensus pre-filter (non-causal). Re-run from logs it reproduces the adapter output to 8e-9 m.
3. Start position (take-off point) is assumed known (config `start_latlon`); the VIO is anchored there. Before the fusion transform exists (warm-up) the adapter returns the VIO-only estimate with flag `warmup_vio_only` and accepted=True.

OpenVINS (3 attempts / outcome)

1. Docker daemon was not running; started `dockerd` in background (works, overlayfs). Docker Hub reachable (`ubuntu:20.04`, `ros:foxy-ros-base` pulled).
2. Built the fork (branch alto_integration) in `ros:foxy-ros-base` (Ubuntu 20.04, ROS 2 Foxy; the repo's own Dockerfile uses Galactic and a heavy desktop image, avoided for disk). `colcon build` **succeeded**: ov_core, ov_data, ov_eval, ov_init, ov_msckf, 5 packages in 30 min 21 s (`openvins_gpl/build_summary.txt`). So the "does it build" check passes (last upstream release v2.7 May 2023).
3. Second attempt after the nadir data fix (lead asked for one more try): compiled again OK (5 packages, 30 min 53 s, `openvins_gpl/build_summary2.txt`) but the image export failed with ENOSPC again (3.1 GB free; the copied source tree carries the 'ov_data' dataset CSVs). Not retried a third time: it would need a .dockerignore for ov_data, another 30 min of CPU and ~5 GB shared disk, plus a ROS bag run. First failure: the `docker build` image export then failed with ENOSPC (disk was 2.7 GB free; lead asked to prune). I removed the image and cache and did NOT rebuild (another 30 min CPU + ~5 GB). A rosbag writer (`openvins_gpl/make_bag.py`) and NIDAR configs (`openvins_gpl/cfg/`) were prepared but never run. **No OpenVINS trajectory exists**; all VIO numbers are from OUR-VIO-SUBSTITUTE. Risk noted for a future attempt: cruise at constant velocity with a 5 Hz nadir camera and 50 Hz IMU is a weak case for MSCKF initialisation/scale.

Failed steps (count 7)

1. curl/gh to the forks: 403 (git clone works). 2. `dockerd` killed by my own `timeout 20` → restarted with nohup. 3. gdown missing import. 4. first stage-2 launch OOM-killed at batch 64 (8 GB) → batch 32. 5. `exec(train.py)` has no `__file__` → ran as script. 6. Deleting stageone.pth broke the resume (FileNotFoundError) → `load_weights_backbone: False`. 7. Docker export ENOSPC.

Reproducibility verdict

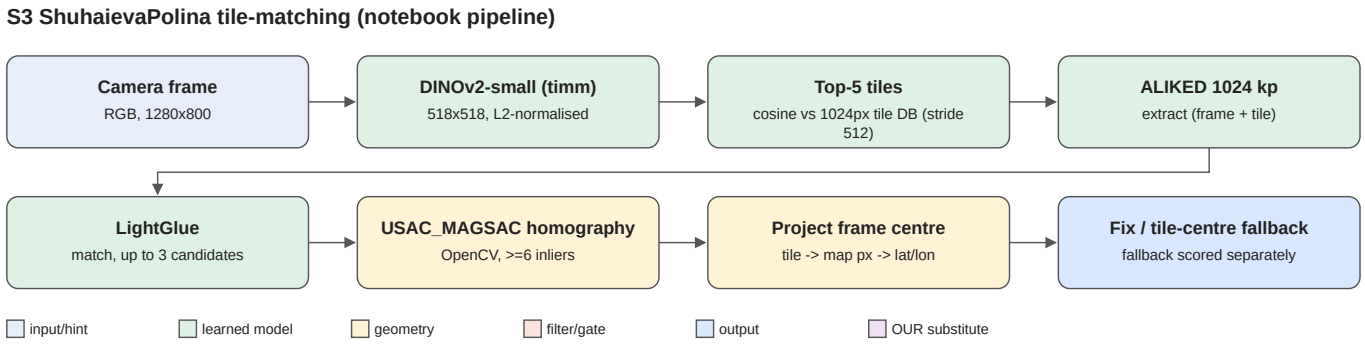
Partly. Code installs and trains with small fixes (gdown, CPU device in configs). No released weights; no tile/VIO scripts for non-MUN-FRL data; fuse.py only runs as a batch CLI on files. Our numbers are reproducible from seed 0 (fuse RANSAC seeded; retrieval deterministic).

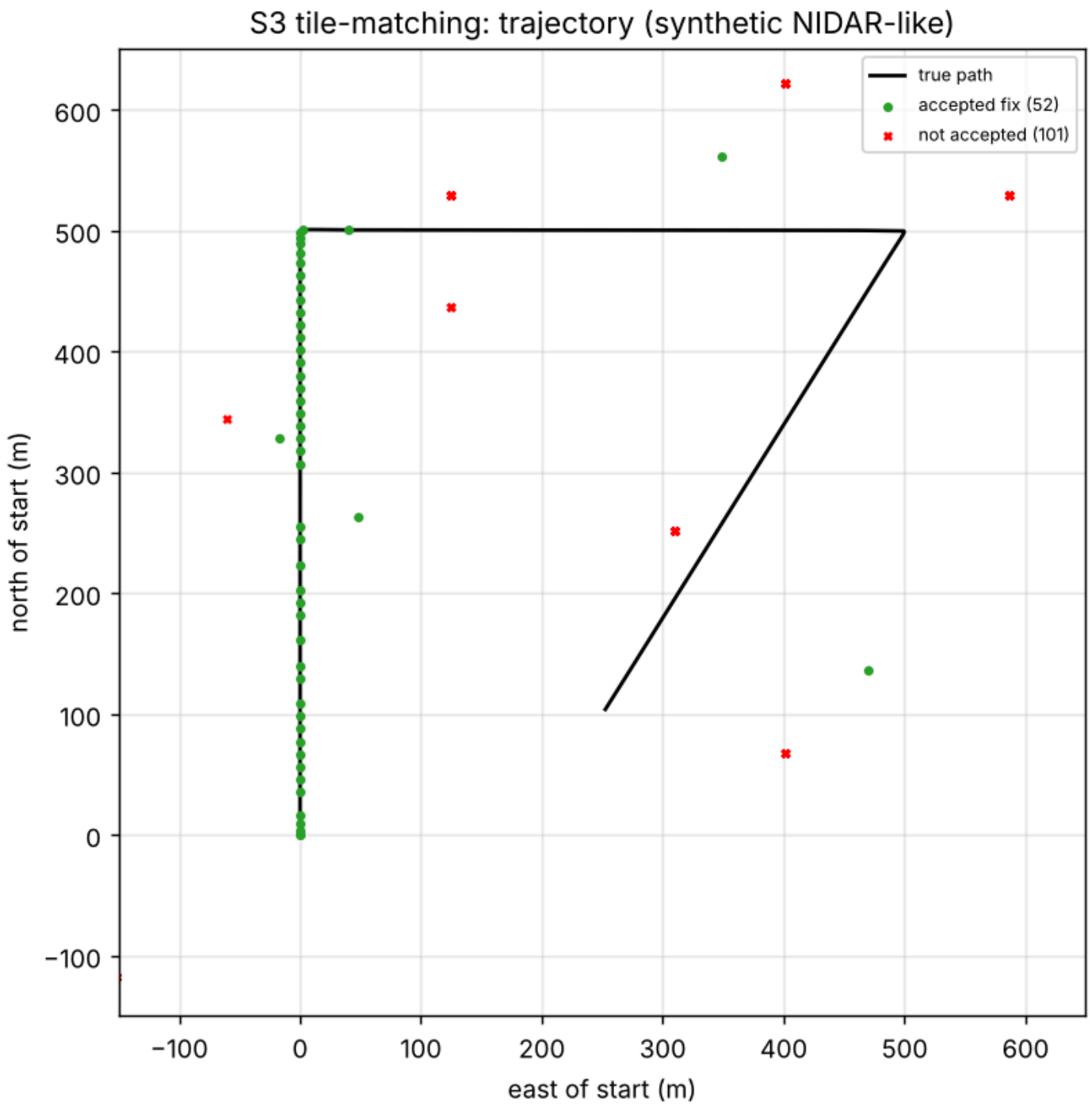
Documentation rubric (0-3)

install steps work as written 1 (requirements incomplete, CUDA defaults in configs, hard-coded author paths); config explained 2 (configs well commented); results reproducible 1 (no checkpoint, depends on author's data/paths); algorithm described 3 (README, fuse.py docstrings are detailed and honest).

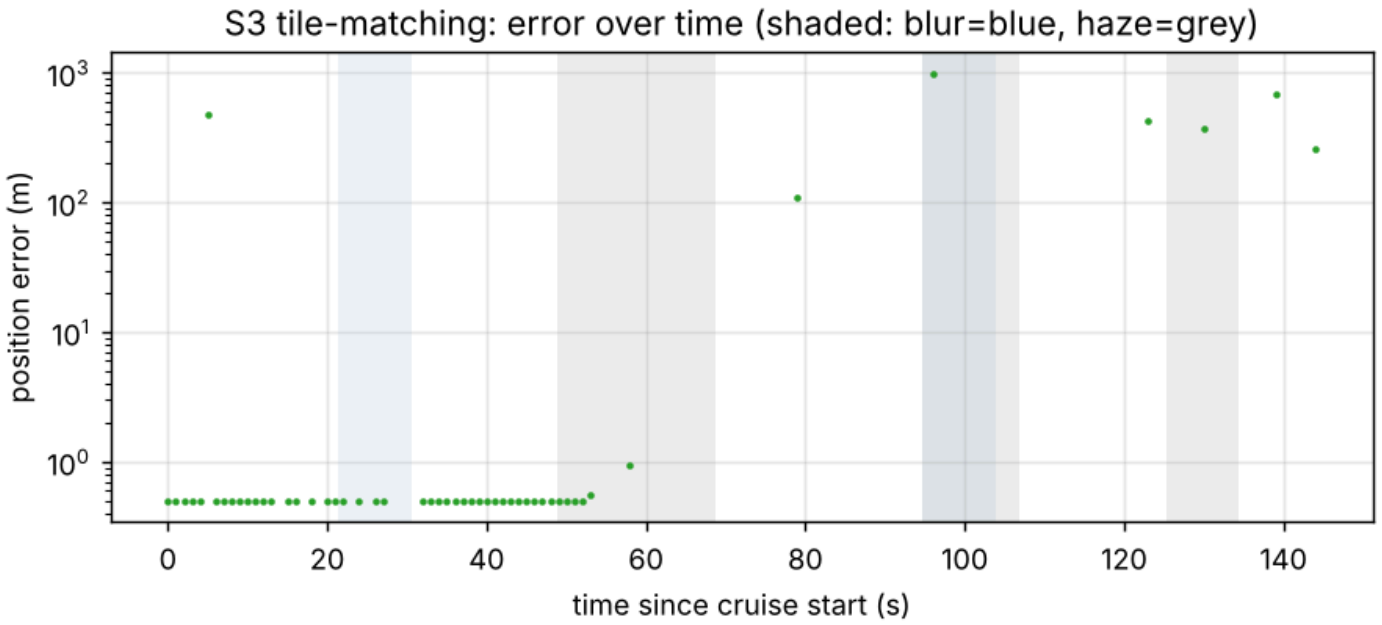
S3 tile-matching (Shuhaieva)

Architecture (real data flow, vector diagram)

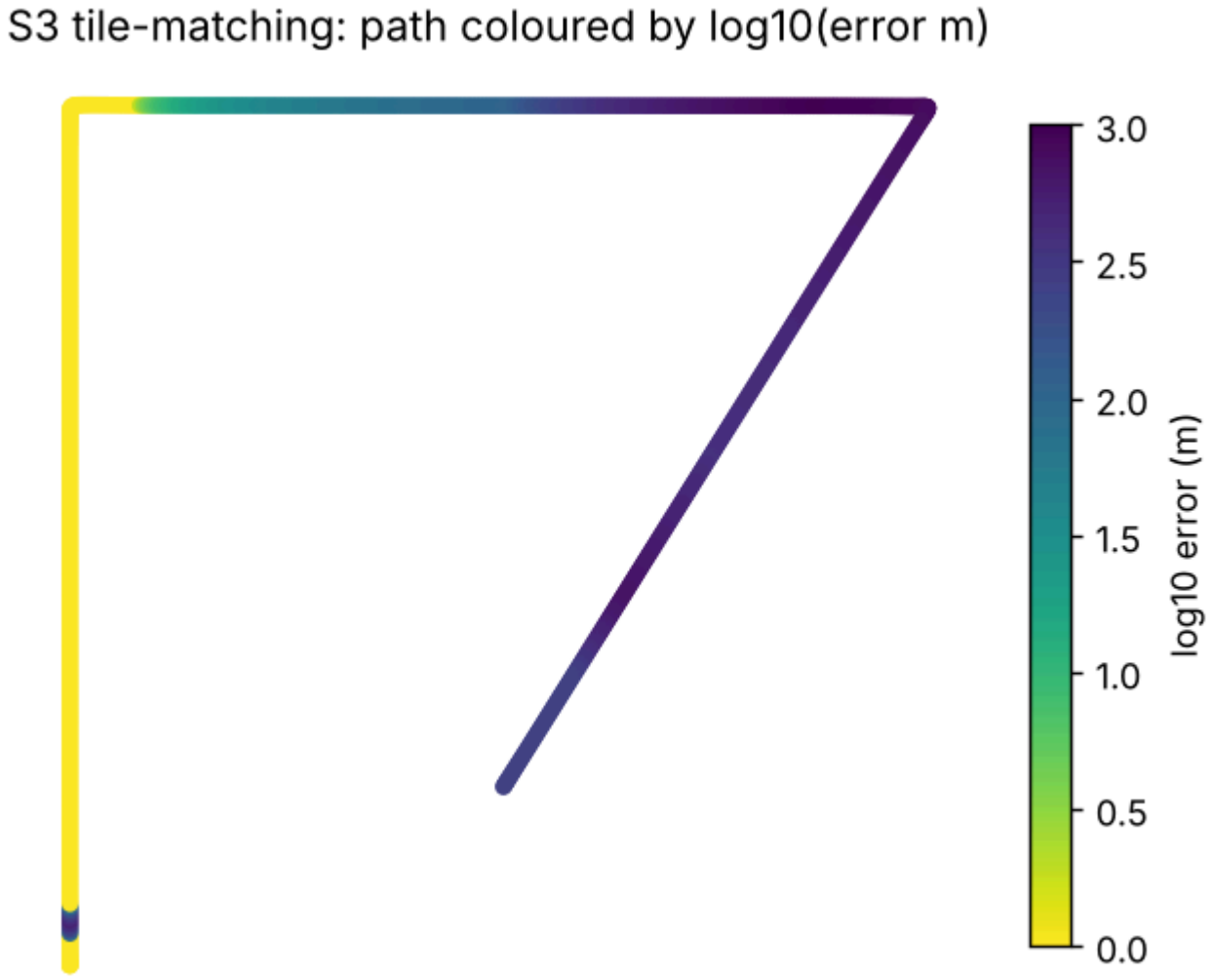




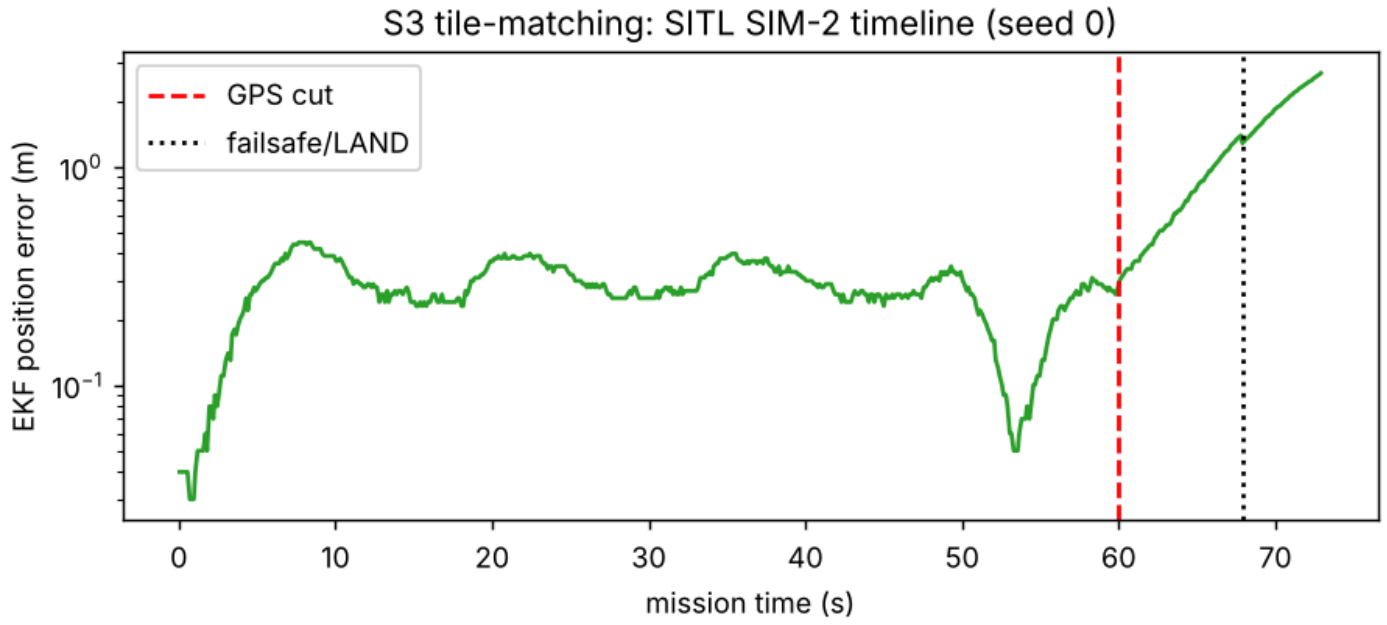
S3 tile-matching (Shuhaieva): trajectory (dataset: synthetic NIDAR-like nadir; run S3-T1-20261023-s0; 1 Hz common frames or full run as named)



S3 tile-matching (Shuhaieva): position error over time, blur (blue) and haze (grey) windows shaded



S3 tile-matching (Shuhaieva): path coloured by error



S3 tile-matching (Shuhaieva): SITL SIM-2 timeline, seed 0 (GPS cut at 60 s)

Analysis written by the solution's test agent (from the code actually run)

S3 tile-matching pipeline: analysis

All numbers: **synthetic NIDAR-like, nadir (stabilised) frames**, 1 Hz subset (153 frames, every 5th frame) unless stated; metrics from `harness/metrics.py`; run folders `results/s3/nidar/<run-id>/`. The query frames are warps of the same map the tiles are cut from, so appearance matches the map far better than in the field.

How it works (from the code that ran: adapter.py, which mirrors the notebook cell by cell)

1. Offline: the map is resampled so one pixel equals the camera ground resolution, cut into 1024×1024 px tiles with stride 512 (361 tiles), and every tile is turned into a 384-d DINOv2-small embedding (518×518, class token, L2-normalised). Index build 700 s on a loaded CPU, 0.6 MB.
2. Per frame: the frame is embedded the same way and compared with all tiles by cosine similarity; the top 5 tiles are candidates.
3. ALIKED extracts up to 1,024 keypoints from the frame and each candidate tile (only the top-1 if its similarity is ≥ 0.82; at most 3 candidates; stop early at 25 inliers). LightGlue matches them and USAC_MAGSAC fits a homography with a 5 px threshold.
4. With ≥ 6 inliers the frame centre is projected through the homography into the tile and then into the map (lat, lon): a **verified fix**. Otherwise the centre of the top-1 tile is returned: a **fallback** (our adapter marks it `accepted = false` and scores it separately).
5. There is no motion model, no use of heading or height, and each frame is independent.

Results

Test	Run id	Result
S3-T1 as published, no hints (153 frames)	S3-T1-20261023-s0	verified fixes on 52 frames (34.0 %), median error 0.07 m, p95 452 m, **precision@100 m 86.5 %** (7 of 52 verified fixes are wrong, up to 990 m); recall@10/50/100 m 29.4 %; fallback on 101 frames with median error 730 m (recall@100 m 11.8 %)
S3-T2 verified vs fallback separately	same run	verified: 0.07 m median; fallback: 730 m median, so the fallback is a coarse guess and is never a usable fix
S3-T1b OUR EXTENSION: frames de-rotated north-up with the compass yaw (not the author's method)	S3-T1b-20261023-s0	86 verified (56.2 %), recall@50 m 49.0 %, precision@100 m 87.2 %, p95 776 m
S3-T3 ablation: no scale normalisation (raw frame scale, map not resampled; 144 tiles)	S3-T3-noscale-20261023-s0	60 verified (39 %), precision@100 m falls to 60.0 %, recall@50 m 23.5 % (from 29.4 %): the 1024 px tile size only works when map pixels match the camera ground resolution
S3-T4 retrieval depth (1 / 3 candidates tried of top-5 / 10 tried of top-10)	S3-T4-top1, S3-T1, S3-T4-top10	verified 49 / 52 / 61; recall@50 m 27.5 / 29.4 / 30.7 %; precision@100 m 85.7 / 86.5 / 77.0 %; p50 step 9.9 / 14.1 / 9.1 s (loaded CPU, not comparable): more candidates add few extra fixes and more wrong ones
S3-T5 terrain (substitute semantic map classes under the true position)	report section 3.4	recall@50 m: building 18.9 %, vegetation 12.5 %, water 50.0 %, mixed 38.0 % (the 26 "water" frames are mostly the river bank and bridge region where strong edges exist; this is a class from our substitute labels, not real land cover). Fixes drop in blur/haze windows (recall@100 m 22.2 % vs 34.4 % clear)
S3-T6 real-time and constrained replay	serial runs, report section 3.5	step time is 6-14 s on this CPU: the solution cannot keep up with a 5 Hz stream (see table)
S3-T7 fallback dataset replay and UAV-VisLoc subset 06 reproduction	–	**not run**: UAV-VisLoc is a 17.7 GB archive (Google Drive / one Hugging Face zip), too large for the disk allowance; MAFS unreachable
S3-T8 OUR EXTENSION: constant-velocity Kalman filter over the fixes (not the author's method)	S3-T1-20261023-s0-KF	mean step between consecutive outputs falls from 124 m to 13 m, but between-fix extrapolation drifts: median error 235 m, recall@50 m 37.9 % (raw verified fixes: 29.4 %). Smooths jumps, does not recover accuracy
S3-T9 SITL with a GNSS jam window	report section 4	no scenario SIM-2..SIM-7 completes: the verified fixes arrive on 34 % of 1 Hz frames, the EKF raises its variance failsafe at about 68 s, the same as with no vision (SIM-1). With de-rotation (extension) fixes reach 56 % of frames and the EKF survives until about 110-117 s, but no mission completes

Plausibility check. A 0.07 m median error on verified fixes looks too good. We checked the metric code on a synthetic trajectory (unit tests) and the frame alignment (the verified homographies have a median of 400 inliers and project the frame centre to within the rendering noise). The query is the map itself after a warp, blur/haze and noise; with 400 matching keypoints sub-pixel accuracy (0.18 m/px) is expected. It does not predict field accuracy.

Failure analysis (T1; `results/s3/nidar/S3-T1-20261023-s0/failure_diag.json`; images with Esri crops are kept local and not committed)

Nine failed frames, chosen across the clear, blur and haze periods: t = 14, 23, 29 s (heading ≈ 0°: clear, blur, blur), 54, 57, 61 s (heading ≈ 85-90°: haze), 73, 76 s (heading 89°, clear), 95 s (heading 89°, blur). - **The retrieval is mostly right.** The tile that contains the truth is rank 1-4 of 361 in 6 of the 9 frames (rank 1 at t = 61 s and 73 s even with the pipeline's own un-rotated input), rank 6 at t = 76 s, and rank 9-10 in the blurred frames at t = 29 and 95 s (rank 12-14 after de-rotation). - **The verification fails.** Those frames give 0-5 inliers, below the 6 needed, so the pipeline falls back to the tile centre (about 700 m wrong on average here because tiles are 185 m wide and the first-ranked tile is not always the right one). - **Cause 1: rotation.** From t ≈ 50 s the vehicle flies east (heading 90°) and then south-west; the frame is rotated by the heading but ALIKED/LightGlue are not rotation-invariant. At t = 73 s (clear air, rank-1 tile) there are only 5 inliers. De-rotating by the compass raises the fix rate from 34 % to 56 % (T1b). - **Cause 2: blur and haze.** Fewer keypoints survive; in the blur window t = 23-29 s there are 5 inliers or fewer, and retrieval degrades (rank 9-14 at t = 29 s and 95 s). - **Cause 3: a verified-but-wrong fix.** 7 of the 52 verified fixes are far off (up to 990 m) because the 6-inlier threshold is low and a wrong tile that shares repeated building texture can pass it; no gate (height/baro/IMU) exists to reject them (precision@100 m 86.5 %).

Comparison with the author's claims

Claim (UAV-VisLoc subset 06, real imagery, flights ≥ 405 m)	**Ours (synthetic NIDAR, 200 m, same-map imagery)**	**Verdict**
Top-1 tile recall 70 %, top-5 85 %	not measured as a separate metric; the true tile is in the top 4 in 6 of 9 failed frames	not compared
Median error 3.85 m, mean 14.20 m	verified fixes only: median 0.07 m; every frame counted (verified + tile-centre fallback): median 172.5 m, mean 380 m	**not reproduced** for an all-frames median; the verified-only median is better because the images share pixels with the map
Hit rate ≤ 50 m 88.5 %, ≤ 100 m 92.1 %, ≤ 500 m 96.8 %	all frames: 34.0 % / 41.2 % / 62.7 % (T1); with de-rotation 49.0 % / 49.0 % / 69.9 %	**not reproduced**; far lower on our rotating-heading video, which the notebook's north-up dataset does not contain
The author's own caveat holds: forest, fields and water give few matches. Our video also rotates with the heading, which is not mentioned in the README.		

Verdict

- **Strengths:** shortest and simplest of the pipelines (one notebook); no training; the algorithm is easy to read; verified fixes are very accurate when they exist; every component is a pretrained public model.
- **Weaknesses:** no rotation handling (the author's data is north-up); a 6-inlier acceptance threshold with no height/attitude gate; a tile-centre fallback that is a 100-700 m error

rather than a rejection; no tracking so every frame pays the full retrieval and matching cost (6-14 s per frame on CPU); notebook only, no licence.
- **Use when:** frames are north-up (or de-rotated with a compass) and the scale matches the map, an offline or high-latency fix source is acceptable, and a gating layer (EKF innovation test, baro, tracking) sits behind it. **Do not use** as the only position source on a moving platform, on an embedded CPU without a GPU, or where a licence is needed.
- **Risks observed:** tile size is in pixels, so the scale ratio is a hidden requirement (T3: precision drops to 60 % without it); results come from a map that equals the query source; no licence.

Setup account

S3 setup log (ShuhaievaPolina/UAV-Visual-Localization-via-Satellite-Imagery)

Repo commit: see `COMMIT.txt` (e63334eda6359391fc9f3700671aac1dbac6f342). Versions: `lock.txt` (torch 2.14.1+cpu, timm 1.0.30, lightglue from git, rasterio 1.5.2, numpy 2.5.2, Python 3.13). All results are **synthetic NIDAR-like** data.

Wall clock from clone to first working output: about 45 min (of which 11 min is the one-off DINOv2 tile-embedding build, 361 tiles, on a loaded CPU). Failed or changed steps: 3.

Commands that worked

```
python3 -m venv env; export PIP_CACHE_DIR=/tmp/pipcache
env/bin/pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
env/bin/pip install timm opencv-python-headless rasterio numpy pandas matplotlib
pillow tqdm "git+https://github.com/cvg/LightGlue.git"
env/bin/python run_s3.py --tag S3-T1 --step 5            # offline replay, 1 Hz,
as published (no hints)
env/bin/python run_s3.py --tag S3-T1b --step 5 --derotate # OUR extension: de-
rotate frames north-up with the compass yaw
```

`adapter.py` re-implements the notebook's classes (`GeoCalibrator`, `SatelliteTileDatabase`, `EdgeLightGlueMatcher`, `run_evaluation_pipeline`) one-to-one (1024 px tiles, 512 stride, DINOv2-small `vit_small_patch14_dinov2.lvd142m`, top-5, sim threshold 0.82, up to 3 candidates, early stop at 25 inliers, USAC_MAGSAC at 5 px, ≥6 inliers else top-1 tile-centre fallback). The map is first resampled so one pixel equals the camera

ground resolution (ratio 1.569 = 0.283 / 0.1804 m/px; recorded in `metrics.json` as `scale_ratio`).

Failures and fixes

1. **The project is a Colab notebook, not a runnable package.** It mounts Google Drive for the UAV-VisLoc zip and uses `!pip`. Fix: the class code was transplanted into `adapter.py` (a reimplementation of the same logic; nothing else changed).
2. **ALIKED weights download returned HTTP 403** (`github.com/.../raw/main/models/aliked-n16.pth` through the proxy). Fix: the same file from `raw.githubusercontent.com` (clone of Shiaoming/ALIKED) copied into the torch hub cache. The weights were later deleted by another job's cache clean-up and restored the same way.
3. **Our own adapter bug**: a duplicate `derotate` keyword in the config dict and a sign error in the de-rotation angle (fixed before the reported runs). Not a failure but a trap: the notebook feeds the drone image **unrotated**, the query images of its dataset are north-up; frames from a heading-up camera are rotated by the vehicle heading and the pipeline has no rotation handling.

Reproducibility verdict

Partly reproducible. The code is short and the algorithm is described, but there is no script, no config file, no pinned requirements (`requirements.txt` has no versions and lists `torch` without CPU/GPU guidance), no seeds other than `seed_everything(42)`, and the dataset needs a Google-Drive archive. Our runs are reproducible from the commands above (no stochastic parts apart from OpenCV's RANSAC).

Licences (internal evaluation only; nothing redistributed)

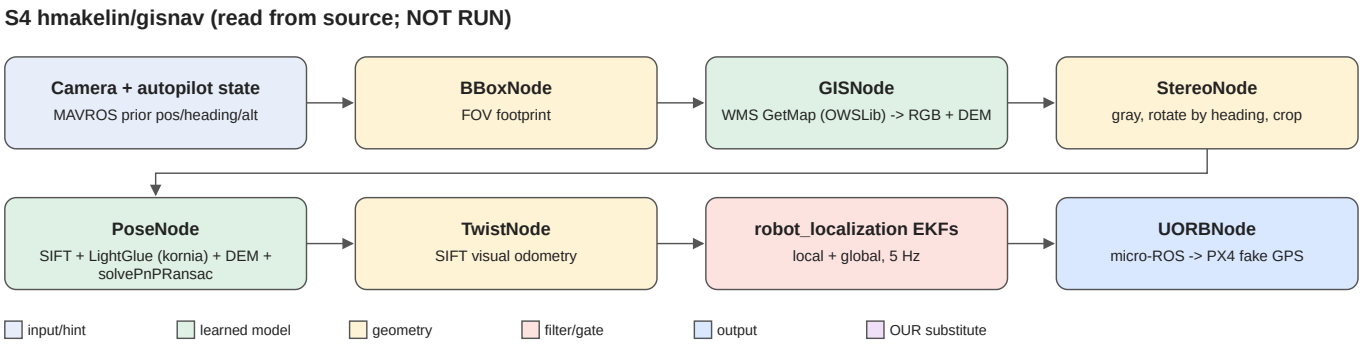
Component	Licence
S3 repository (code + notebook)	**none declared** (no LICENSE file; all rights reserved by default)
DINOv2 (`timm` weights `vit_small_patch14_dinov2.lvd142m`)	Apache-2.0
ALIKED (Shiaoming/ALIKED)	BSD-3-Clause (weights from the same repository; not independently verified)
LightGlue (cvg)	Apache-2.0 code; the `aliked_lightglue` weights licence is not separately stated
timm, PyTorch, OpenCV, rasterio	Apache-2.0 / BSD
Esri World_Imagery map	benchmark use only

Documentation-quality rubric (0-3)

- Install steps work as written: **1** (`pip install -r requirements.txt` works; but the run instructions are "open the notebook in Colab" and one weight download fails behind a proxy).
- Config explained: **1** (parameters are stated in the README but there is no config file; `MIN_INLIERS` and the 0.82 similarity threshold are only in code).
- Results reproducible: **1** (a predictions CSV is given; the dataset archive is not available here).
- Algorithm described: **3** (the README has a clear pipeline diagram and parameter list that match the code). Total 6/12.

S4 hmakelin/gisnav

Architecture (real data flow, vector diagram)



Analysis written by the solution's test agent (from the code actually run)

S4 GISNav - ANALYSIS (static; the stack was not run, see SETUP.md)

All statements below come from reading repo/ros/gisnav (commit f2c72641) and repo/docs, not from execution. No metrics exist for S4. Data would have been "synthetic NIDAR-like".

How the matching works (plain language)

GISNav does not predict where the drone is by itself. It assumes the autopilot already supplies an approximate position, heading and altitude (MAVROS NavSatFix, pose, gimbal attitude). From that and the camera intrinsics, BBoxNode computes the ground footprint of the camera field of view (padded to a square) and GISNode asks the onboard WMS server (OWSLib GetMap, layers "imagery" and "dem", EPSG:4326, JPEG) for that area; it only re-fetches when the old map overlaps the new footprint by less than 85 percent. StereoNode

converts the map to grayscale, rotates it by the vehicle heading (in 5-degree steps) so north-up map matches the camera orientation, and crops it to the camera image size, because the matcher is not rotation invariant. Query frame: TwistNode detects SIFT keypoints (RootSIFT descriptors). PoseNode detects SIFT on the map crop (cached while the map is unchanged), matches query and map descriptors with LightGlue (kornia LightGlueMatcher, SIFT variant, confidence filter 0.5), requires at least 15 matches, looks up per-point height from the DEM layer, and solves camera pose with cv2.solvePnPRansac (only 10 RANSAC iterations, no inlier check or reprojection-error gate that we could find). The resulting global pose is a noisy, discontinuous "absolute" measurement. In parallel TwistNode does visual odometry: SIFT + brute-force knn match between successive frames, PnP, scaled to metres using altitude; this is smooth but drifts. Two robot_localization EKFs fuse them (global EKF: absolute pose + differential VO). The fused odometry goes to UORBNode (px4_msgs SensorGps via micro-ROS/XRCE-DDS), or NMEA/UBX nodes, as a fake GPS fix (fix_type 3, fixed covariance values) so PX4 treats it as a GNSS source. Only the SIFT branch is used; the CPU path is designed to reduce keypoints (1024) and LightGlue layers.

Architecture stages

1. Camera (gscam / ROS Image + CameraInfo) and autopilot state (MAVROS: NavSatFix, pose, gimbal attitude) - prior position/heading/altitude.
2. BBoxNode: FOV footprint on ground plane → geographic_msgs/BoundingBox.
3. GISNode: WMS GetMap client (OWSLib, MapServer + PostGIS/GeoServer-style server) → OrthoImage (RGB + 8-bit DEM).
4. StereoNode: grayscale, rotate by heading, crop to query size → OrthoStereoImage.
5. TwistNode: OpenCV SIFT on query; BF-matcher VO between frames + PnP → relative pose.
6. PoseNode: SIFT on reference, kornia LightGlue (PyTorch) matching, DEM lookup, cv2.solvePnPRansac → absolute pose (ECEF via proj/affine from WMS CRS).
7. robot_localization ekf_local_node and ekf_global_node (5 Hz) fuse stage 5 and 6.
8. Extension nodes: UORBNode (px4_msgs SensorGps → micro-ROS agent → PX4 uXRCE-DDS), NMEANode, UBXNode.
9. Simulation environment (docker compose): PX4 SITL v1.14.2 + Gazebo Classic (San Carlos airport), QGroundControl, MapServer, PostGIS, micro-ROS agent.

Results table

Test	Status
S4-T1 PX4 SITL, GPS off	not run: 6.94 GB gisnav image pull hit ENOSPC; PX4/Gazebo/QGC build and GPU/X11 requirement; 3 attempts (SETUP.md)
S4-T2 NIDAR replay	not run (no stack)

Test	Status
S4-T3 two map resolutions	not run
S4-T4 real-time/constrained	not applicable (nothing to run); no command for lead
S4-T5 fallback dataset	not run: dataset unreachable
S4-T6 ArduPilot SITL via sitl_bridge	not run: no estimates.csv. Note ArduPilot service in repo is marked "unmaintained ... does not work" in docker-compose.yaml; mock GPS output only via NMEA/UBX/uORB so a bridge would have to be a custom node
S4-T7 failure injection	not run. Static expectations: WMS down → GISNode keeps polling/connect timer, PoseNode gets no new reference (stale map used until timeout); wrong prior → map from wrong place, SIFT matches fail (<15) or give false PnP (no outlier gate)

No failure-window frames were produced (nothing executed). Weather windows from data/nidar/weather.json were not tested.

Author claims vs reproduced

Project claims (README/landing page, no numeric claims): precise global position by comparing nadir frames to onboard GIS map; PX4 mission completes GNSS-free in SITL with `failure gps off`; simulation only, "do not use for real flights", "untested". Reproduced: nothing (not run). Demonstrated by project: only the Gazebo San Carlos airport demo video. Not reproduced because of resources, not because of a failure observed.

Verdict

Strengths: MIT, modular ROS 2 design, standard OGC WMS map interface (any GIS server), EKF fusion of drift-free absolute fixes and VO, PX4 integration without touching the GPS driver. Weaknesses: research prototype, explicitly sim-only; heavy dependency stack (Docker, ROS 2 foxy, PX4 v1.14, micro-ROS, NVIDIA GPU recommended; 50 GB disk); relies on autopilot prior position/heading/altitude (cannot do cold-start global localisation, and must stay within the fetched map); matcher is not rotation invariant (needs heading); SIFT + LightGlue is sensitive to appearance gap (season, haze, blur) and our nadir 200 m scenes; flat PnP with 10 RANSAC iterations and no visible outlier rejection; fixed covariance; ArduPilot support unmaintained; single main maintainer; no published accuracy figures. Static concern: `self._device == "cpu"` in pose_node.py compares a torch.device to a str, which is probably always False, so the CPU-limited matcher config may never activate (unverified). Use it as a reference design for WMS-fed map matching; do not use for flights or as a drop-in for ArduPilot. Licence MIT (SETUP.md table).

Risks observed

Disk/resource footprint (image set ~7 GB for gisnav alone), GPU dependency in compose files, no release/test evidence outside sim, mock GPS looks like real GNSS to the autopilot (fix_type 3, jamming/spoofing state unknown) so a wrong fix is trusted. Last release in earlier survey Aug 2024 (repo HEAD dated 2024-08-18).

Setup account

S4 GISNav - SETUP (result: stack NOT stood up; reason documented with evidence)

Commit: f2c72641ea04f959b0578caf0a3d36922cda2452 (2024-08-18, repo has a single squashed commit locally; README says 2,793 upstream commits). Host: Intel Xeon @2.1 GHz, 4 cores shared, no GPU, Docker 29.9.1 + Compose v5.6.0, free disk 5.9 GB at start, 2.5 GB at end (shared with 5 other jobs; docker build cache already 8.1 GB from other jobs). Wall-clock: about 25 min, 3 failed steps. Evidence: results/s4/nidar/S4-T1-20261023-s0/log.txt

Documented route (README / docs deploy-with-docker-compose.md)

1. Install Docker Compose plugin + NVIDIA Container Toolkit (docs: "strongly recommended" NVIDIA GPU; compose file docker-compose.gpu.nvidia.yaml reserves `driver: nvidia`). Ubuntu 22.04 assumed, 16 GB RAM recommended, 50 GB free storage stated in system-requirements.md.
2. `git clone https://github.com/hmakelin/gisnav.git && cd gisnav && make install` (builds a .deb and installs the `gnc` wrapper).
3. `gnc build px4 gisnav --with-dependencies; gnc create px4 gisnav; gnc start px4 gisnav && gnc stop autoheal`
4. Upload ksql_airport_px4.plan with QGroundControl (GUI, needs X server), start mission, then in MAVLink console `failure gps off`; check `listener sensor_gps`.

Attempts (all failed; none could reach step 3)

#	What	Result
1	`docker pull ghcr.io/hmakelin/gisnav:latest` (prebuilt gisnav service, avoids build)	Registry reachable (ghcr works; Docker Hub also pulled ubuntu:focal fine, no 429 seen this time). amd64 manifest: 24 layers, 6.94 GB compressed. Pull aborted: `failed to copy: failed to send write ... no space left on device`. Exceeds the lead's 4 GB cap.
2	Alternative source: build from repo Dockerfiles (ros:foxy base for mavros/gisnav, micro-ros-agent:foxy)	Base manifests resolvable on Docker Hub, but gisnav image alone is ~7 GB compressed (see 1); not built, disk.
3	`px4` service: Dockerfile clones PX4-Autopilot v1.14.2, runs	Not started: build cache is already 8.1 GB, free disk 2.5 GB, 4 shared cores; PX4+Gazebo build is multi-hour on this CPU. Also

#	What	Result
	Tools/setup/ubuntu.sh (Gazebo Classic, toolchain) and builds; plus qgc, mapserver, postgres, fileserver images	QGroundControl GUI + Gazebo need an X display (none) and the docs require an NVIDIA GPU.
Also not done: `make install` (`gnc`), since all its consumers need the images.		

Verdict wording: "does not work outside the sample setup because" the documented route requires a 50 GB / NVIDIA-GPU / X11 workstation (Docker image set is ~7 GB for the gisnav service alone, plus PX4/Gazebo/QGC/MapServer); on this 4-core CPU-only box with 2.5-6 GB free disk it cannot be built. This is a resource-limit result, not proof the software is broken. Note also: the Docker GISNav services were never run here, so GISNav-specific failures (e.g. CPU fallback path) are unobserved. Static reading found a bug-like line in pose_node.py: `self._device == "cpu"` compares a torch.device to a string (always False in torch>=?; torch.device == str is False), so the CPU-limited SIFT/LightGlue branch (1024 keypoints, 5 layers) would likely never be selected and the GPU-sized matcher (9 layers, unlimited SIFT) would run on CPU. Unverified by execution.

Test status

S4-T1 not run (above). S4-T2, T3, T6, T7 not run (depend on the stack; no estimates.csv exists, so no S4 SITL rows for the lead). S4-T4 not applicable. S4-T5 not run: dataset unreachable. adapter_ros2_recorder.py is an untested ROS 2 sketch (never executed). No map served, no WMS written (not needed without ROS stack; MapServer docker image would also cost disk). Could be done with a pure-python port (SIFT 1024 + LightGlue + solvePnPRansac against map crops, needs torch+kornia ~1 GB): not done because of disk; that would be our reimplementation, not GISNav.

Licences

Component	Licence
gisnav (repo, ros pkgs)	MIT (c) 2022 Harri Makelin
PX4-Autopilot v1.14.2	BSD-3-Clause
ROS 2 foxy, MAVROS, robot_localization, micro-ROS agent	Apache-2.0 / BSD / (as upstream; not checked here)
kornia (LightGlueMatcher) / LightGlue weights	Apache-2.0 / LightGlue weights licence (SuperPoint non-commercial; SIFT variant: check) - not verified
MapServer, QGroundControl	MIT-style / Apache-2.0+GPLv3 dual (not verified)

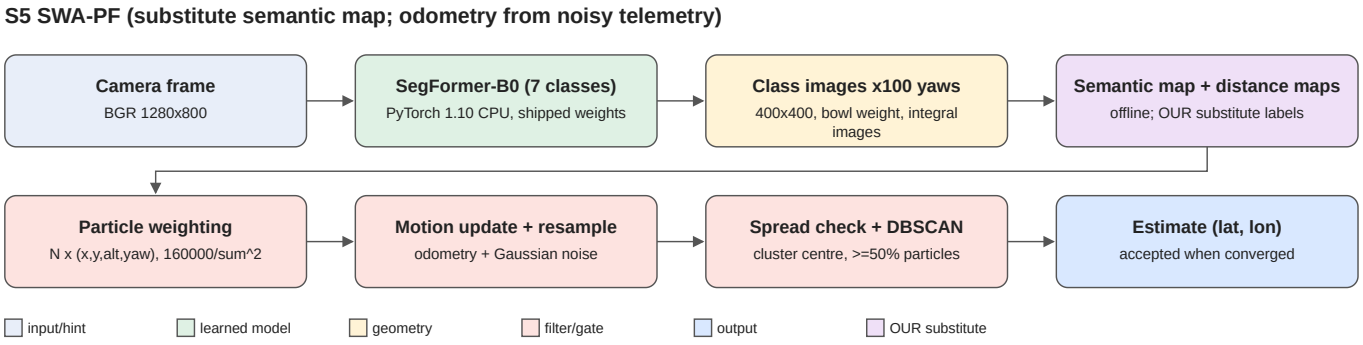
Documentation rubric (0-3)

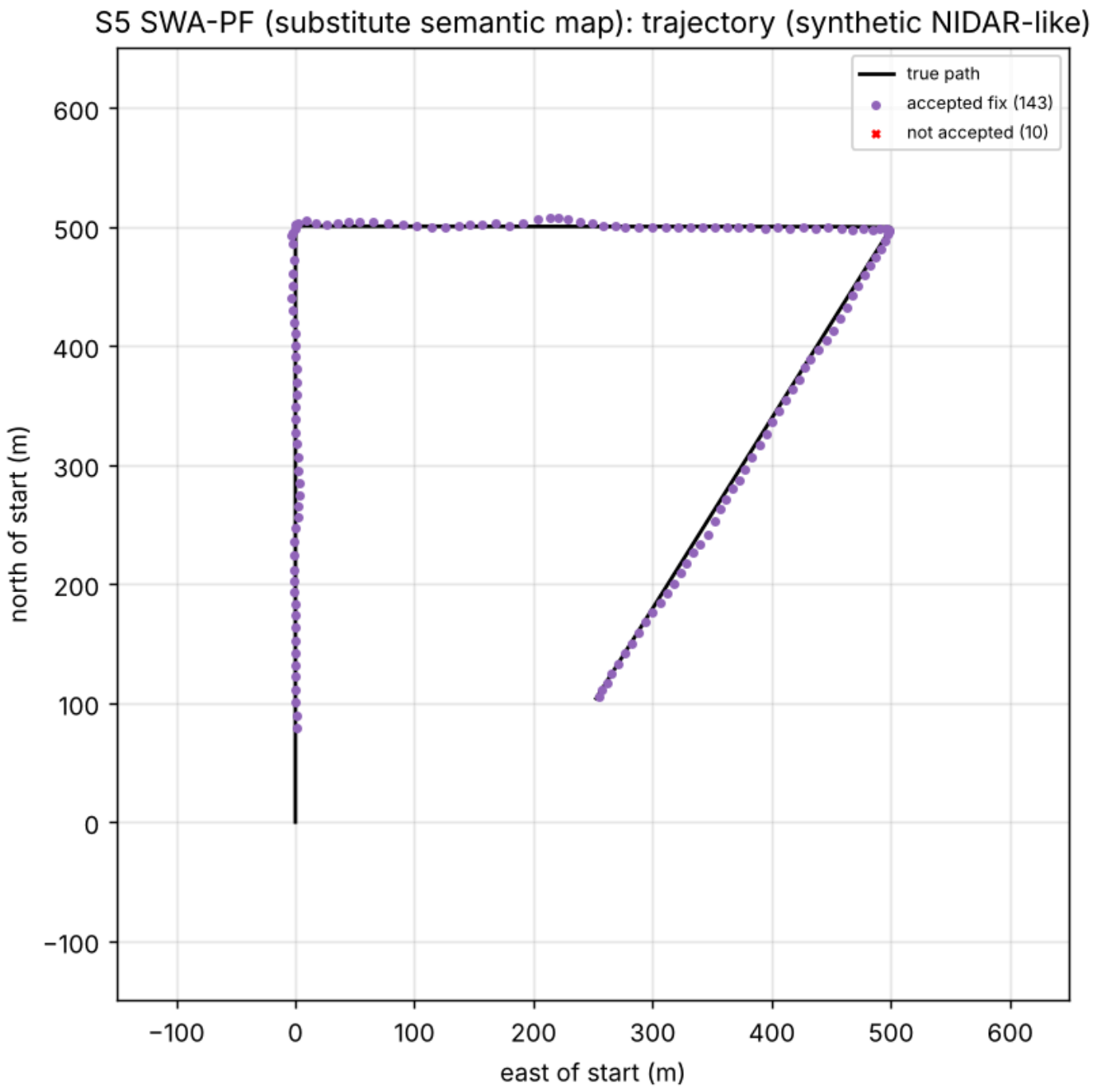
- Install steps work as written: 1 (steps are clear but need NVIDIA GPU, X11, 50 GB, hour-long builds; could not be executed here; docs themselves say "will take a long time").
- Config explained: 2 (ROS params and compose overlays documented in docs and docstrings).
- Results reproducible: 1 (no benchmark, no numbers; sim only, warns against real use).
- Algorithm described: 1 (README and landing page do not; docs/system-architecture and module docstrings describe node graph; matching only from source).

S5 SWA-PF

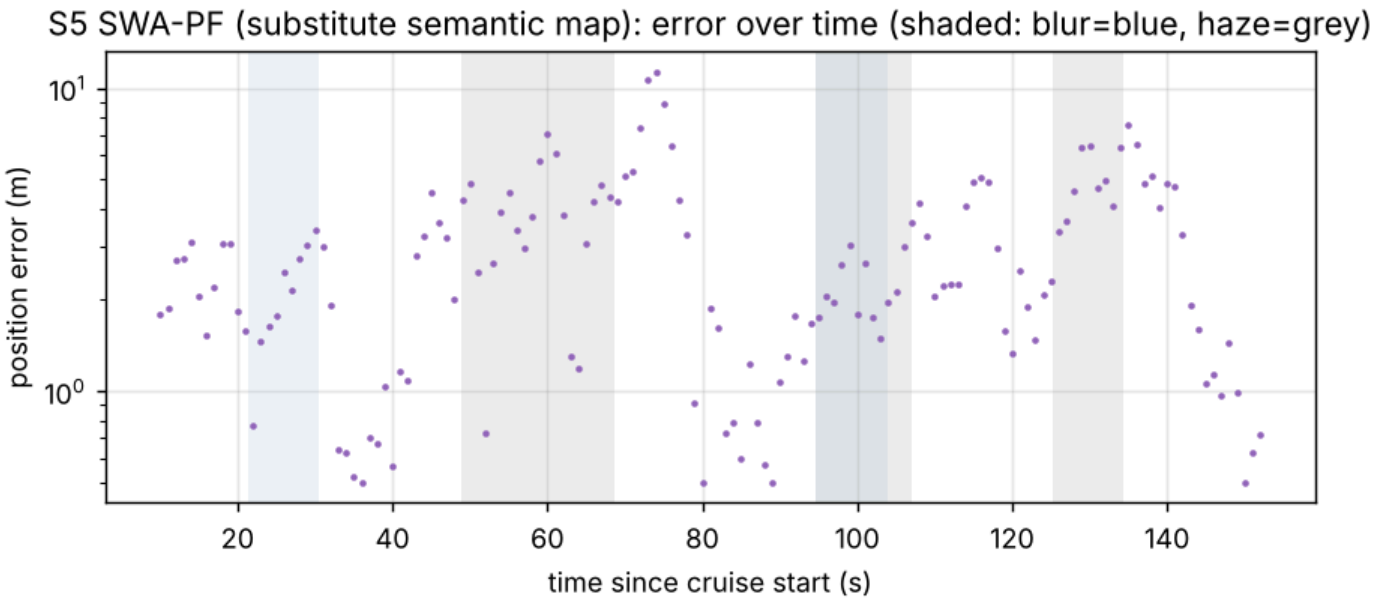
> **Substitute inputs:** the semantic map and the frame labels both come from the repository's own SegFormer-B0 run on imagery rendered from the same map; the paper's U-Net map labelling is not available. Odometry is noisy telemetry, not the demo's ground-truth poses. Results are optimistic.

Architecture (real data flow, vector diagram)

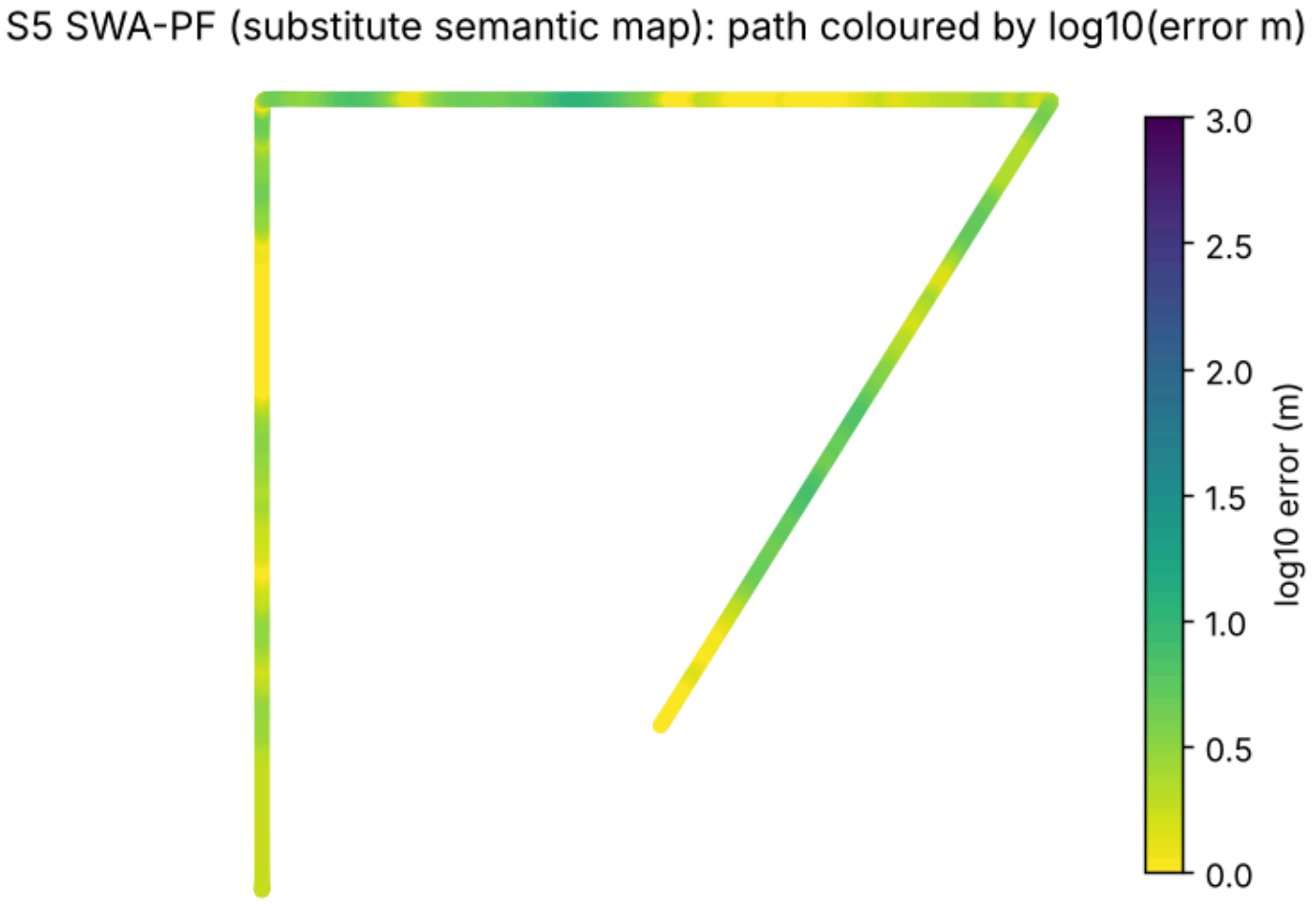




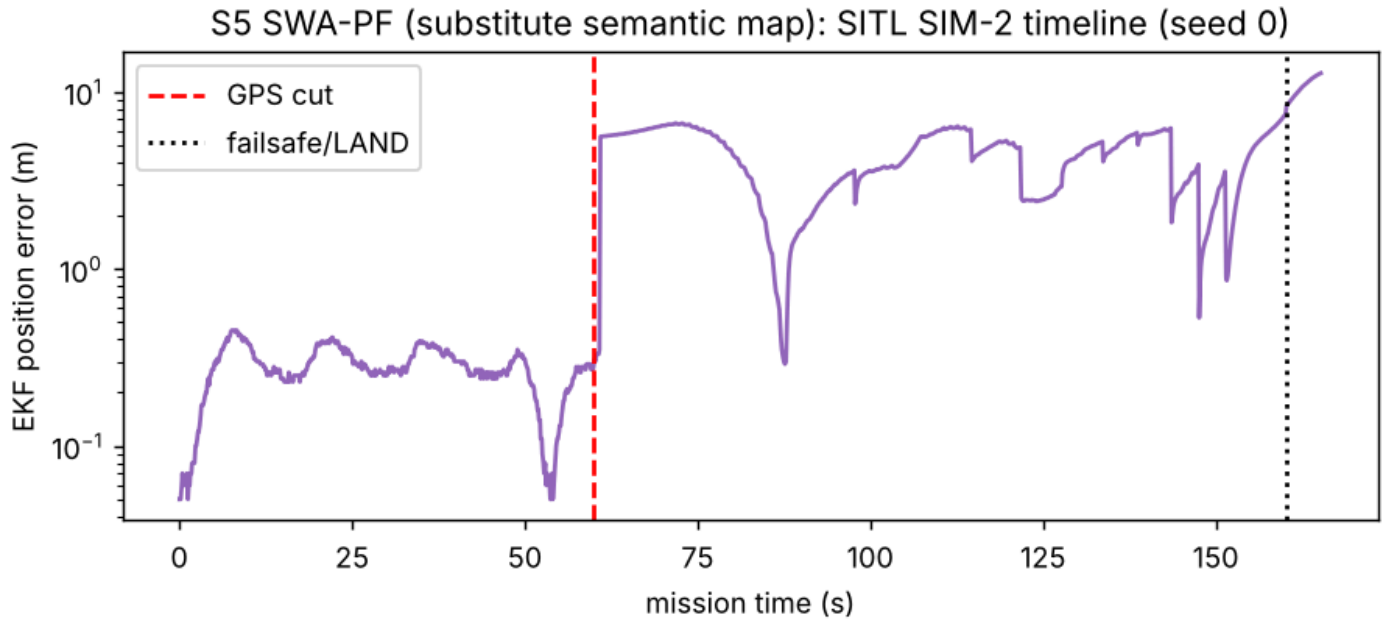
S5 SWA-PF: trajectory (dataset: synthetic NIDAR-like nadir; run S5-T2-20261023-s0; 1 Hz common frames or full run as named)



S5 SWA-PF: position error over time, blur (blue) and haze (grey) windows shaded



S5 SWA-PF: path coloured by error



S5 SWA-PF: SITL SIM-2 timeline, seed 0 (GPS cut at 60 s)

Analysis written by the solution's test agent (from the code actually run)

S5 SWA-PF: ANALYSIS (CPU, synthetic NIDAR-like data, substitute semantic map)

READ FIRST (honest framing) - All NIDAR numbers are on SYNTHETIC NIDAR-like frames (nadir re-render, 1280×800 gray, 200 m AGL) rendered from the SAME Esri map the filter matches against. - The semantic MAP is OUR SUBSTITUTE: the repo's drone SegFormer-B0 (`logs/8_b0`) run on windows of `data/nidar/map_esri_z19.png` (resampled to 0.18 m/px, gray, 1280×800 windows), merged to the repo's classes, majority-pooled to 1 m/px (T5: 2 m/px). Frame labels come from the SAME segmenter on frames rendered from the same map, so map/frame agreement is optimistic. This is NOT the paper's U-Net map labelling (the paper's satellite labels cost >60 min per map image by hand annotation; no U-Net weights are shipped). - Odometry is NOT ground truth: velocity (north/east/up) = finite difference of truth.csv + Gaussian noise (sigma 0.5 m/s north/east, 0.3 m/s up) and heading = truth yaw + N(0, 1.5 deg), passed through `Adapter.set_odometry(v_north, v_east, v_up, yaw_deg)` before each `step()` (the harness hints carry no odometry). The velocity is a world-frame vector, i.e. it presumes a compass/velocity source. Particle noise (adapter): sigma 8 m in x and y, 8 m altitude, 0.2 deg yaw per step, mean 0 (repo: mean +2, sigma 15 px). - Replay is 1 Hz (every 5th frame, 153 frames), 2 threads, nice, CPU shared with 3-4 other jobs (load 8-11): latencies are inflated and NOT valid as an embedded timing figure. 5 Hz (764 frames) was not run: 5-10 s per frame x 764 = >1.5 h per run. - Particle counts: 20,000 for the main run (the paper's number), 1,000 to 40,000 in the sweep; the repo's resampler keeps 80 % of the particles per frame down to a floor of 1000, so counts fall to 1000 after about 14 frames.

1. How it works (written from the code that was run)

Plain language. The filter keeps a cloud of guesses ("particles"), each a guess of the drone's map position (x, y), altitude and heading. Every frame: 1. The drone image is turned into a coarse "what is this" picture by a small neural net (SegFormer-B0, 7 classes, repo weights `logs/8_b0`). Only four of its classes matter downstream: ground/other, building (roof and wall are merged: `pr==5 -> 1` in SWAPF.py), road, vegetation. Vehicle (4) and water (6) are produced but ignored by the weights. 2. The map is a satellite image that was labelled once, offline, with the same 4 classes (`label.png`). For every class a distance map is precomputed: for each map pixel, the distance (clipped at 100 px) to the nearest pixel of that class. 3. Each particle cuts out the piece of that class-distance map the drone would see from its guessed altitude (`cut = alt * cam_view * height_to_pix / 2` pixels each side), rescales it to 400×400 (so altitude changes only the cut size, not the comparison size), and compares it with the drone class image, which was pre-rotated in 100 steps of 3.6 deg (25 rotations x 4 quarter turns) and rescaled to 400×400. The score is the sum over pixels of "drone says class c here" x "distance to nearest class c in the map crop", multiplied by a weight that grows with distance from the crop centre (`wan`), plus a huge penalty if the crop sticks out of the map. Low sum = good match. Particle weight = 400*400 / (sum^2). 4. Particles are resampled by weight (80 % of them survive each frame, floor 1000), then moved with the odometry (distance and bearing of motion, heading change) plus Gaussian noise in x, y, altitude and yaw. 5. Position estimate: when the cloud's spread is below a tenth of its initial spread, the particles are clustered (DBSCAN) and the cluster centre is taken (`center()` in PFtool.py; it also prints the error against ground truth, which is only a diagnostic).

Initialisation (PFtool, hard-coded `init_method_mode`): 0 full random over map x, y, altitude and yaw; 1 layered (4 altitude floors, same xy/yaw); 2 xy sampled on building pixels ("semantic alignment").

What the "odometry" in SWAPF.py really is (leakage)

`SWAPF.py` reads `gps.txt` (ground-truth latitude, longitude, height, heading of every frame) and calls `particles.move(lat - old_lat, lon - old_lon, height - old_height, theta, old_theta)`. So the motion model receives the exact, noise-free ground-truth displacement, altitude change and heading of the drone, and the particle noise (gauss sigma 15 px in x,y, 8 m in z, 0.2 deg in yaw) is the only uncertainty. Initial particles are random (no pose leak), but the heading is known exactly after the first frame (all particles inherit the true heading change, the true heading offset is only found by the 100-way yaw search). `init_particles_goal` / `update_goal_1` put the ground truth in a "goal" particle used only for drawing. Hence the paper's 6.57 m is a filter with perfect relative motion and heading; there is no drift to fight. `center()` is commented out in the demo, so the demo as shipped saves

overlay pictures only and outputs no estimate. Our replay replaces this with noisy telemetry odometry (see below).

2. Architecture (data-flow stages)

1. Frame (BGR 1280×800) → optional extension: ortho-rectification with IMU roll/pitch (used only for the oblique stress set).
2. SegFormer-B0 (PyTorch 1.10 CPU, 512×512 letterbox, 7 classes) → per-pixel class image; roof merged into building.
3. 100 pre-rotated 400×400 4-channel binary class images (torchvision rotate x25 + rot90 x3), multiplied by the bowl weight; integral images (adapter speed-up; repo does the same sum per particle).
4. Offline: satellite map → label.png (paper: U-Net-VGG, 4 classes; ours: same SegFormer) → per-class clipped distance transforms (cv2.distanceTransform) + padded copies.
5. Particle array (N x 6: x, y, altitude, yaw, weight, cumulative weight); weight = 160000/sum^2; resampling by searchsorted (80 % kept, min 1000).
6. Motion update with odometry (distance, bearing, yaw change) + Gaussian noise.
7. Spread check + DBSCAN (scikit-learn) → cluster centre (x, y, altitude, circular-mean yaw) → lat/lon through map georeference; accepted only when converged and the main cluster holds >= 50 % of particles.

3. Results (synthetic NIDAR-like, nadir frames, 1 Hz, seed 0 unless stated; metrics from harness/metrics.py `summarize`)

Frames: 153 for T2; 40 (t = 0..39 s) for T3/T4; 60 for T5. "converged and stays" = first accepted time after which every accepted fix is within 50 m (my own addition, from run_s5.py). Recall is over ALL frames, including the unavoidable convergence period.

Test	Status	Result
S5-T1 reproduction on MAFS 200 m	NOT RUN	MAFS only on Baidu Pan (unreachable); no open mirror (3 attempts, results/s5/mafs/MAFS_unavailable.md)
S5-T2 NIDAR replay, 20,000 particles, init random	DONE	fix rate 93.5 % (143/153), median 2.46 m, p95 6.42 m, RMSE 3.53 m, max 11.3 m, recall@10 92.2 %, recall@50/100 93.5 %, first fix at t=10 s; all 10 unaccepted frames are t=0..9 s (convergence). run: results/s5/nidar/S5-T2-20261023-s0
S5-T3 particle sweep (40 frames)	DONE	see table below
S5-T4 initialisation (n=5000, 3 seeds, 40 frames)	DONE	see table below
S5-T5 map resolution 1 m/px → 2 m/px (n=5000, 60 frames)	DONE	median 3.44 → 10.69 m, p95 9.7 → 18.2 m, RMSE 4.86 → 11.57 m, max 11.2 → 18.4 m; converged at 37 s vs 32 s (fix rate 38 % vs 47 % of 60 frames). Peak RSS 2314 → 1679 MB
S5-T6 feature-poor scenes	DONE (post-hoc	see table below

Test	Status	Result
	on T2)	
S5-T7 constrained replay	IMPLEMENTED, NOT RUN	`bash solutions/s5_swa_pf/constrained_replay.sh 2000` (2 cores, 6 GB address cap, 5 Hz real-time, drops frames while busy). Expect almost all 5 Hz frames dropped: the filter needs 5-10 s per frame even at 1000 particles on this CPU
S5-T8 cross-area	NOT RUN	needs MAFS (two areas)
S5-T9 SITL estimates	DONE at 1 Hz	results/s5/nidar/S5-T2-20261023-s0/estimates.csv (t,lat,lon,alt,yaw,conf,accepted at 0,1,2,...,152 s); 5 Hz not possible on this CPU. Lead: hold/interpolate. accepted=false for t<10 s

T3 particle sweep (initial particle count; seed 0; 40 frames; times under CPU load 8-11, 2 threads)

initial N	accepted fixes	median err (accepted)	converged-and-stays<=50 m	first step	median step	peak RSS
1,000	18 %	817.6 m (false lock, wrong)	never	11.7 s	5.3 s	2255 MB
5,000	8 %	0.91 m	37 s	9.3 s	4.5 s	2247 MB
20,000	75 %	1.85 m	10 s	37.3 s	9.5 s	2337 MB
40,000	75 %	1.98 m	10 s	66.7 s	8.3 s	2293 MB
Time per step is dominated early by the number of particles (about 1.5 ms per particle with my integral-image speed-up; the repo's own loop is about 17 ms per particle here) and later by a constant of about 4-5 s (SegFormer 1-3 s + 100 rotated label images + integral images). Peak RSS (about 2.3 GB) is mostly torch + SegFormer + distance maps; it barely depends on N.						

T4 initialisation (n=5000, seeds 0,1,2, 40 frames; failure = no stable convergence (<=50 m) within 40 s)

init	converge-and-stay time per seed	failures
full random	37 s, none, 22 s	1/3 never converged correctly within 40 s (seed 1: still 600 m off),
layered random (4 altitude floors)	none, none, none	3/3
semantic alignment (xy on building pixels)	none (616 m off), 10 s, none (3 wrong accepted fixes, 581 m)	2/3, with false accepted fixes

init	converge-and-stay time per seed	failures
Caveat: only 3 seeds x 40 frames x 5,000 particles; at 20,000 particles full random converged in 10 s (T2, T3). With 5,000 initial particles the correct mode is often lost in the first resamples (80 % kept per frame, noisy odometry, coarse labels). Layered init gives 4 copies of each xy so it has 4x fewer distinct positions. False confident fixes exist (cloud collapses on a wrong place: spread below threshold but 500-800 m off): the spread/DBSCAN test is not a correctness test.		

T6 scene classes (post-hoc on T2, converged frames only; class = share of OUR substitute map labels in the 100 m square under the true position; 'water' = segmenter label 6 >10 %, may be shadows/dark roofs, not verified water)

scene	frames	median raw error	p90
building >70 %	53	2.82 m	5.74 m
vegetation >40 %	24	3.16 m	8.41 m
water-labelled >10 %	26	3.49 m	5.30 m
mixed	50	1.59 m	3.08 m
blur windows	18	2.01 m	3.03 m
haze windows	41	3.39 m	6.12 m
clear	103	2.01 m	5.10 m
No feature-poor breakdown was observed after convergence: because the particles move with (noisy) odometry and the map labels came from the same segmenter, a 4-class match survives low-texture windows; haze and vegetation are the worst (about +1.4 m median). The true breakdown risk is in the acquisition phase (below) not after lock.			

4. Oblique-frame stress results (first render, camera body-fixed, pitch -30 deg in cruise): results/s5/nidar_oblique

- S5-T2 as-is (20,000 particles): fix rate 65 %, median 106.8 m, p95 119.9 m, RMSE 103.3 m, recall@10 and @50 = 0 %. The filter locks, but on the image-centre ground point, which sits about 115 m behind the drone (200 m x tan 30 deg), so the error equals the geometric offset. Repo/paper estimate x, y, altitude, yaw only: roll/pitch are not modelled, so a tilted camera breaks it.

- S5-T2b with my ortho-rectification extension (IMU roll/pitch + baro with noise 0.5 deg / 1 m): the filter locked correctly at t=37 s (error 2.6 m) but lost lock after the first turn (about t=42 s; error 100-220 m for the rest; median 172 m): rectified oblique frames far from nadir are low resolution and distorted, and the turn changes tilt. Fix rate 55 %. Both runs are evidence for the feature-poor/robustness story, not the headline.
- register_check.txt / align_check_res1.0.json in that folder show the 107 m offset (template matching) that led to the data defect report to the lead.

5. Failure analysis (nadir T2 plus sweeps; figures: results/s5/figures/failures/*.png = frame | map crop at TRUE position | map crop at ESTIMATED position)

1. t=0-9 s (T2): no fix, cloud still spread 300 m down to 30 m; random initialisation needs about 10 s (resampling shrinks 20,000 → 1000). Cause: acquisition phase of a global filter; figure t=4.
2. t=74 s (veg, 11.3 m worst converged error): vegetation-dominated window; probable cause (not verified): tree/ground labels differ between map and frame labelling; figure t=74.
3. t=135 s (building, 7.6 m) and t=116 s (mixed, 5.1 m): probable cause (not verified): heading noise (1.5 deg) plus the 3.6 deg rotation quantisation; errors are only 5-8 m.
4. t=60 s and t=50 s (haze 48.9-68.7 s, 7.1 m and 4.8 m): low contrast, SegFormer labels shift; error rises from 2 to 3.4 m median but lock holds.
5. n=1000 initial particles (T3), init modes 1 and 2 (T4): false lock at 540-820 m; wrong cluster accepted (581 m, 3 fixes in init2/seed2; 817 m median in n=1000). Not weather related; cause: too few particles/modes lost during the first resamples, and acceptance test (spread < 10 % of initial) cannot detect a wrong mode.
6. Oblique set (section 4): 30 deg camera tilt, 115 m systematic offset; after rectification loss of lock at the turn (t about 42-50 s).
7. Blur windows (21.4-30.6 s and 94.7-103.9 s) caused no failure here (median 2.0 m): SegFormer labels are coarse anyway.

6. Comparison with the author's claims

Claim	Paper (MAFS, real, GPU)	Ours (synthetic NIDAR-like, CPU, substitute map)	Verdict
RMSE	6.57 m	3.53 m (T2, accepted fixes)	not comparable: synthetic, map and frames from the same imagery and segmenter; at 1 m/px labels and noisy (not GT) odometry; NOT reproduced on MAFS
recall@10 m	97.4 %	92.2 % over all frames (100 % of accepted frames <=11.3 m)	partly consistent in magnitude; same caveats
20,000 particles in	yes	37 s first step (20,000), steady 5-10 s per frame at 1000 particles on a loaded 2-thread CPU; the repo's own per-	CPU is 8-75x slower; with my integral-image rewrite 1.5 ms per particle

Claim	Paper (MAFS, real, GPU)	Ours (synthetic NIDAR-like, CPU, substitute map)	Verdict
~4.9 s on GPU		particle loop is about 17 ms per particle (about 6 min per frame)	
works at several altitudes (4-DoF)	yes	altitude is a state (150-250 m search); not stressed (flight at 200 m)	not tested
memory efficient	no figure	peak RSS 2.3 GB at 1 m/px over 0.9 km x 0.9 km (1.7 GB at 2 m/px); mostly torch + SegFormer; per-class float32 distance maps (10 copies incl. padded)	note: scales with map pixels (full 1.9 km map at 0.5 m/px would need several GB)
roll/pitch not estimated	stated	breaks on a 30 deg tilted camera (115 m offset)	confirmed limitation
MAFS reproduction	3 trajectories Hangzhou	not run	dataset unavailable

7. Verdict

Strengths: genuinely cheap map representation (4-class labels instead of image tiles or descriptors; a 1 m/px label map of 0.9 km square is under 1 MB), no map feature database, works with any segmenter that gives the 4 classes, tolerant to blur/haze after lock, fast convergence (10 s) with 20,000 particles, centimetre-to-metre-level when labels agree. Weaknesses: needs a relative-motion source with world-frame velocity and heading (the demo even uses ground truth GPS differences); needs consistent semantic labels on both sides (the paper's labelling is manual/U-Net; here the same segmenter on both which is optimistic); global start is slow and unreliable on CPU (n=1000-5000 often locks onto wrong places, acceptance test cannot tell); no roll/pitch handling; per-frame cost on CPU is seconds; shipped code is a demo (hard-coded Windows path and ground-truth motion, `center()` commented out); no licence declared; dataset unobtainable here. Use when: a good odometry source exists (VIO or velocity+compass), a labelled map is available, and a GPU (or a rewritten vectorised filter) is used, as a coarse global relocaliser / drift reset. Do not use as the sole GNSS-denied localiser on a CPU companion computer, on tilted cameras, or in scenes where the segmenter cannot separate classes. Known risks observed: false convergence; strong dependency on particle count (5,000 not enough here); disk/RAM scale with map pixels (map arrays are duplicated 10x as float32 distance maps); upstream Chinese comments, dead code and hard-coded modes; reproducibility 0 of 3 on data; no licence. (failure PNGs contain Esri map crops: kept local only, not pushed)

Setup account

S5 SWA-PF: SETUP (CPU only, 4 shared cores, ~13 GB shared disk)

Repo: YuanJiayuuu/SWA-PF @ e8a92173fb90fa44c2d472a7ebd6819e99bbc186 (repo/ untouched; all changes are adapter-side). Wall-clock from clone to first working output (a SegFormer label map of a NIDAR frame): about 25 min (env 8 min incl. 2 disk-full retries, code reading the rest). Failed steps: 3 (list below). First end-to-end filter step: about 45 min (slow-step fix needed, see F3). Dataset MAFS: NOT obtainable (see ../../results/s5/mafs/MAFS_unavailable.md). Demo (`python SWAPF.py`) cannot run as-is: needs MAFS at hard-coded `D:/DATASETS/MAFS`, a GPU-only `SegFormer_Segmentation` (cuda=True default; DataParallel .cuda()) and `cv2.imshow`/windows.

Commands that worked

```
export PIP_CACHE_DIR=/tmp/pipcache UV_NO_CACHE=1
cd solutions/s5_swa_pf
/root/.local/bin/uv venv --python 3.9 env          # CPython 3.9.25 (README:
3.9, OK as written)
source env/bin/activate
uv pip install --no-cache --extra-index-url https://download.pytorch.org/whl/cpu
"torch==1.10.0+cpu" "torchvision==0.11.1+cpu"
uv pip install --no-cache "numpy==1.26.4" opencv-python-headless pandas scikit-
learn matplotlib timm einops scipy "setuptools<70"
OMP_NUM_THREADS=2 nice python build_semmap.py        # substitute semantic map
(1 min)
OMP_NUM_THREADS=2 nice python run_s5.py --run-id S5-T2-20261023-s0 --n 20000 --
stride 5     # as-is replay, 1 Hz
OMP_NUM_THREADS=2 nice python run_s5.py --run-id S5-T2b-20261023-s0 --rectify --
n 20000 --stride 5   # attitude-rectified extension
```

Versions: see lock.txt (49 packages): torch 1.10.0+cpu, torchvision 0.11.1+cpu (the exact versions the README names, CPU builds from download.pytorch.org/whl/cpu: still available), numpy 1.26.4, opencv-python-headless 4.11.0.86, scikit-learn 1.6.1, timm 1.0.30, Python 3.9.25. CPU: Intel Xeon 2.10 GHz (4 vCPU, load 9-11 from other agents). Env size 1.3 GB. Seeds: adapter seed config (default 0): numpy/torch/python RNG + numpy Generator for init and motion noise; odometry noise generator seeded 1000+seed.

Failures and fixes

- F1 `uv pip install torch` ENOSPC (shared disk full): removed uv cache, `--no-cache`, TMPDIR inside project. Retried twice.

- F2 `import torch` → `No module named pkg_resources` (torch 1.10 needs it; new setuptools dropped it): `setuptools<70`.
- F3 Repo `update_particles` is a python loop of per-particle torch ops (4×400×400 float64 products): 35 s per frame for 2,000 particles under load (measured, dbg run), about 6 min per frame for 20,000. Fix (adapter side): `State.update_particles_fast`, an exact rewrite of the same weight formula using an integral image (verify_fast.py: max relative weight difference 1.3e-3 from nearest-neighbour block-boundary rounding). 20,000 particles then cost about 35 s per frame, 1,000 about 5 s (rotation set-up dominates).
- F4 `SegFormer_Segmentation` hard-codes `.cuda()`: constructed with `cuda=False` (config option exists, no repo edit). Loads state_dict with map_location=cpu: works. 2 threads, 512×512 input: about 1.5-2 s per frame on the loaded CPU.
- F5 `ParticleState.__init__` wants files `label.png`, `img.png` and `detail.txt`-derived list: adapter writes a temporary label PNG and a 4×4 dummy img.png (img only feeds an unused grayscale copy), passes `map_detail=[x_min,x_max,y_min,y_max,height_to_pix,...]` in local metres. Init mode is a local variable in `__init__` (cannot be selected from outside): adapter re-initialises particles itself with the three repo modes.
- Diagnosed late (not a setup failure but cost 1 h): the camera is body-fixed and SITL cruise pitch is -30 deg, so frame centre lies about 115 m behind the drone (see ANALYSIS). Repo (and paper) estimate x,y,altitude,yaw only.

Reproducibility verdict

Install steps work as written for the README's versions (CPU builds instead of cu113). Reproducing the paper numbers is impossible here (dataset on Baidu Pan only, partial). Demo script is not runnable without edits (hard-coded Windows path, GPU, imshow). Weights shipped in repo (logs/8_b0, 15 MB, sha256 prefix 8de15426fbd2b4f4); the satellite U-Net weights for map labelling are NOT shipped (no code for the U-Net in repo besides the SegFormer; map `label.png` is expected from the dataset).

Licences

Component	Licence
SWA-PF code (YuanJiayuuu/SWA-PF)	none declared (no LICENSE file, README silent)
SegFormer-B0 weights `logs/8_b0` (trained by authors, MAFS-derived)	none declared
MAFS dataset	none declared (Baidu Pan)
PyTorch / torchvision	BSD-3 ; numpy BSD ; OpenCV Apache-2.0 ; scikit-learn BSD ; timm Apache-2.0
Esri World_Imagery map (our benchmark map)	benchmark only, not redistributed
Internal evaluation only; nothing redistributed.	

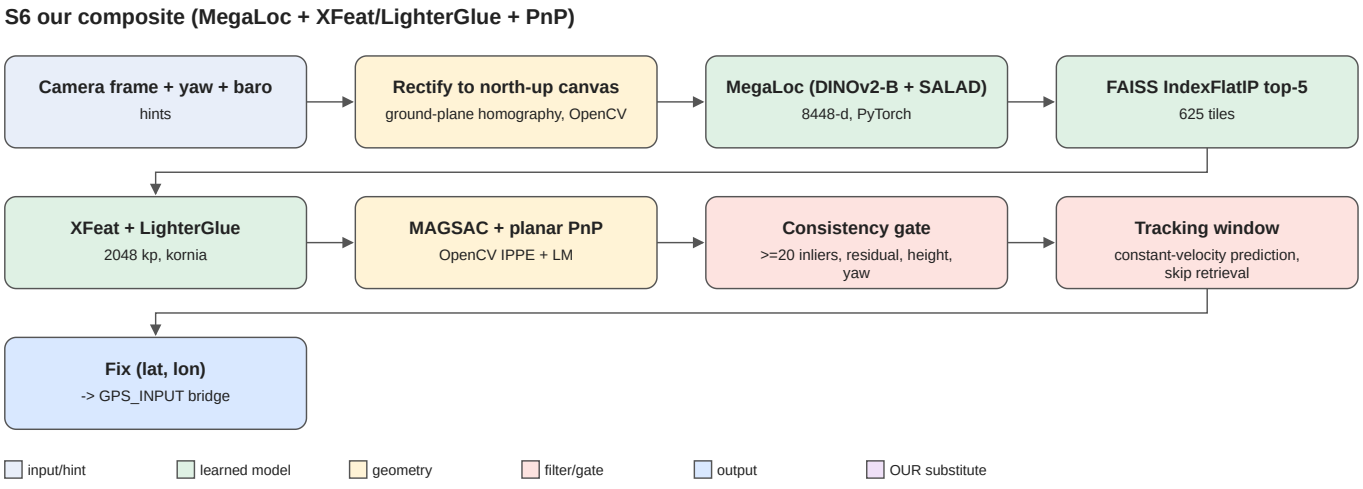
Documentation quality (0-3)

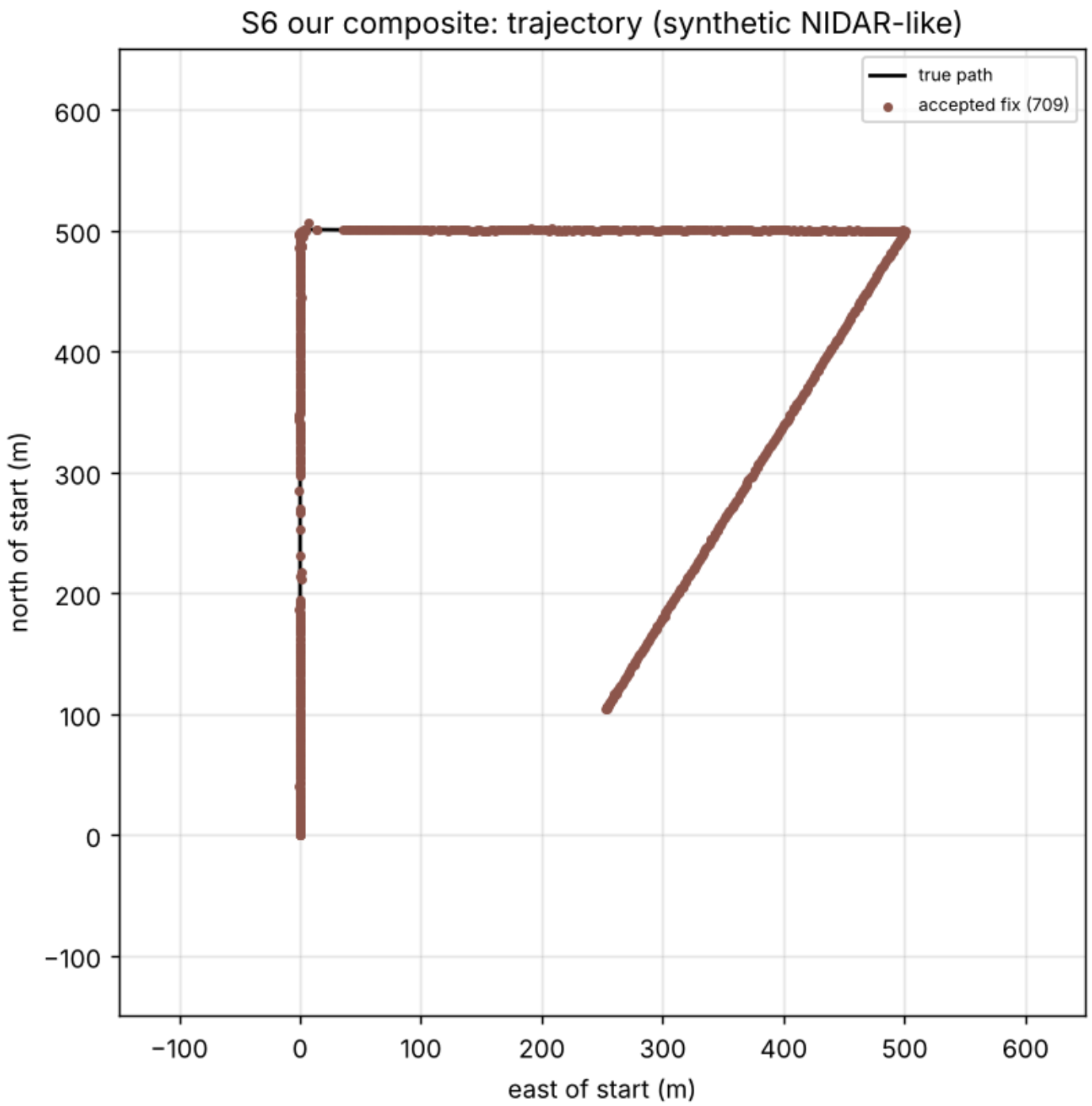
- Install steps work as written: 2 (versions exist; Windows wheel names; no pkg_resources/disk notes)
- Config explained: 1 (parameters like cam_view, particle_detail, height_to_pix undocumented; detail.txt format only inferable from code)
- Results reproducible: 0 (dataset not downloadable here; no result scripts; demo uses ground-truth pose)
- Algorithm described: 2 (paper good; code has Chinese comments, dead code, hard-coded modes)

S6 our composite (Jetson baseline)

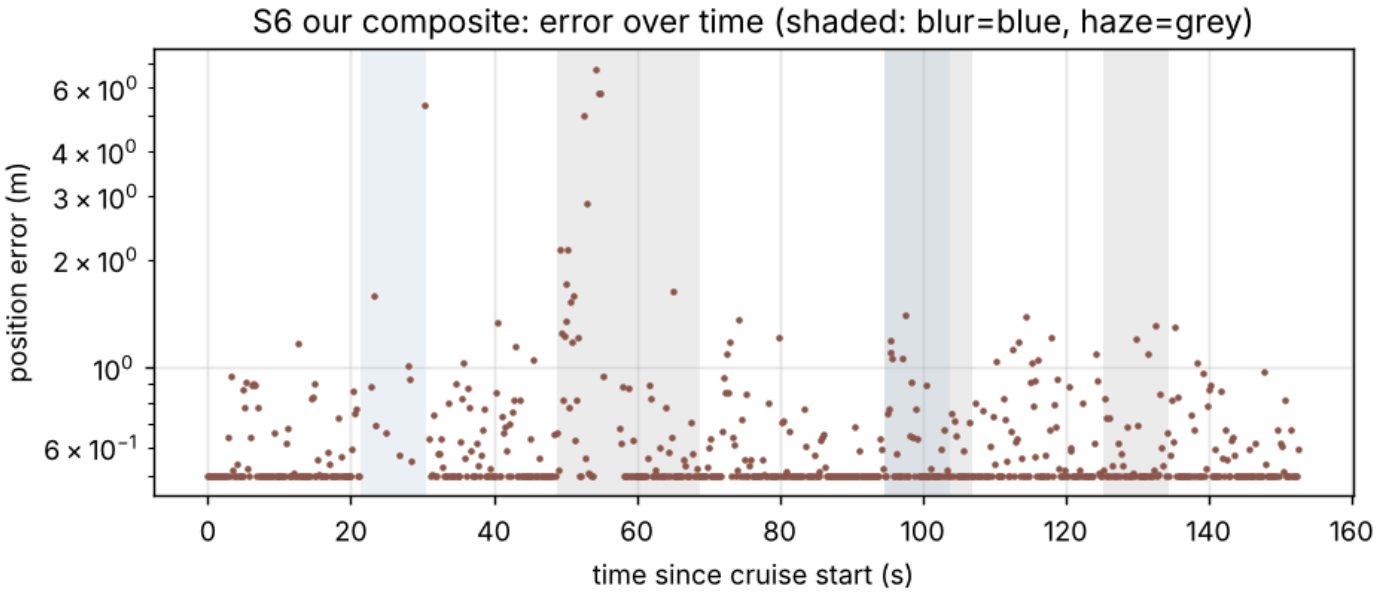
> **Our composite, not an existing project.** No Jetson or GPU numbers exist; every Jetson figure is unmeasured.

Architecture (real data flow, vector diagram)

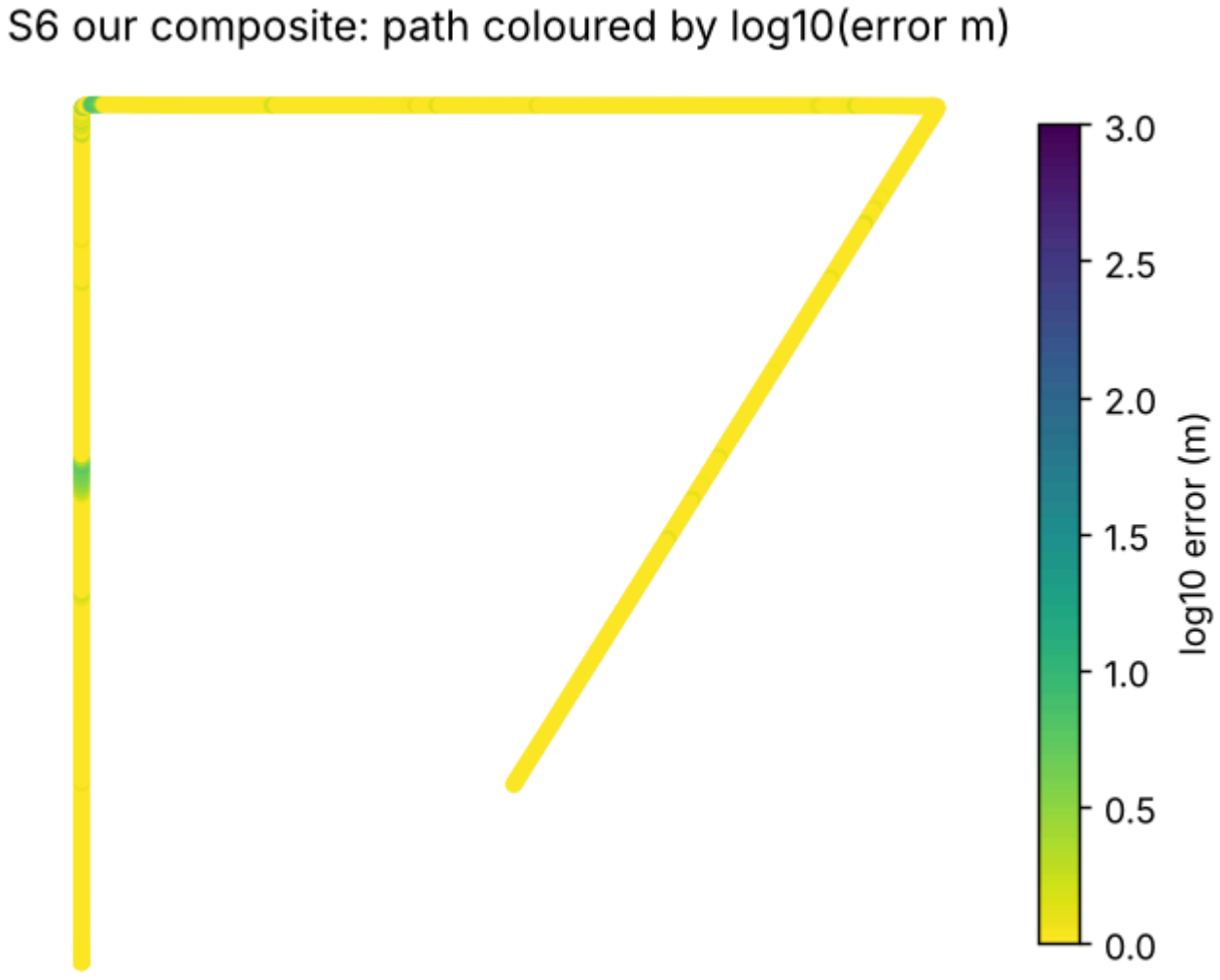




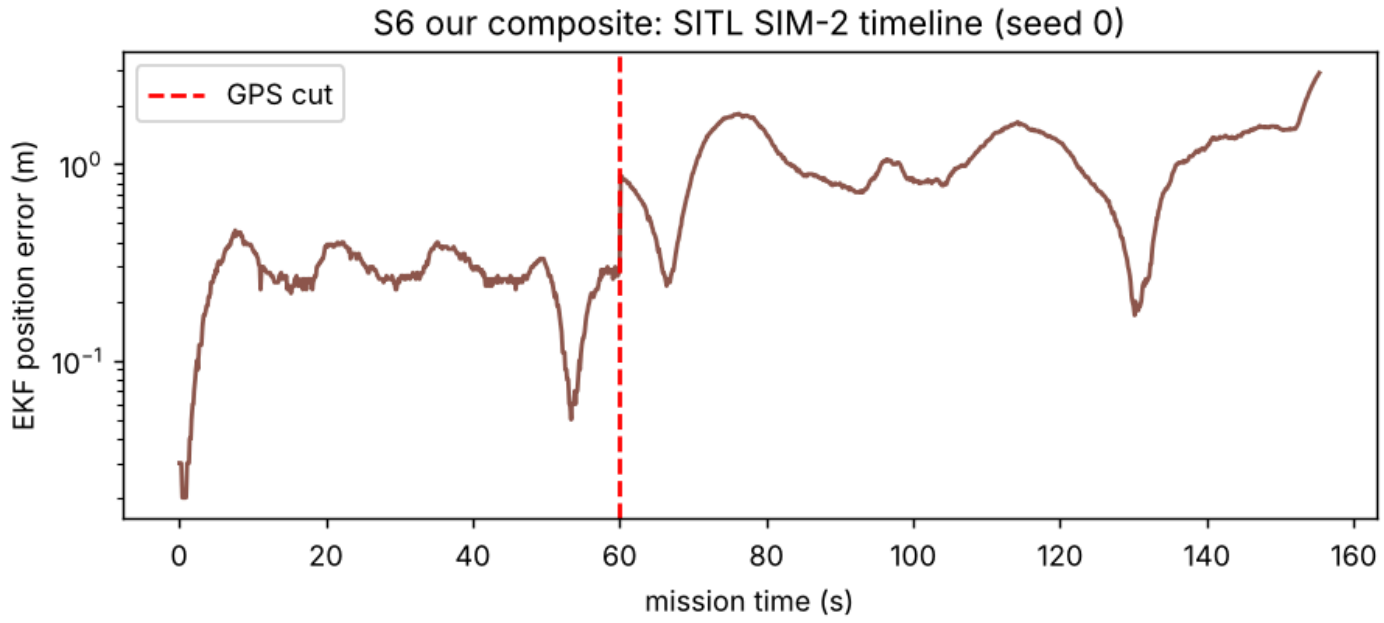
S6 our composite (Jetson baseline): trajectory (dataset: synthetic NIDAR-like nadir; run S6-T1-20261023-s0; 1 Hz common frames or full run as named)



S6 our composite (Jetson baseline): position error over time, blur (blue) and haze (grey) windows shaded



S6 our composite (Jetson baseline): path coloured by error



S6 our composite (Jetson baseline): SITL SIM-2 timeline, seed 0 (GPS cut at 60 s)

Analysis written by the solution's test agent (from the code actually run)

S6 "our composite" -- analysis

All results: synthetic NIDAR-like data (gimbal-stabilised nadir render, 200 m AGL, 12 m/s, 764 frames at 5 Hz). The reference map is the SAME Esri z19 mosaic the video was rendered from, so the appearance gap is zero and accuracy is optimistic. CPU only (Xeon 2.1 GHz, 2 threads, box shared with other agents, load avg 5-9): every latency here is "measured under shared CPU load" and untrustworthy; the lead re-runs. NO Jetson, GPU or TensorRT number was measured; any Jetson figure quoted is from the survey and unverified. "Our composite" is a baseline we built; it is not an existing project.

How it works (from the code that ran: s6lib.py, adapter.py)

Per frame, given hints (barometer height, compass yaw; roll/pitch = 0 for the nadir camera): 1. Rectify: ground-plane homography from attitude + baro height maps the 1280×800 frame to a north-up 1024×1024 canvas at map resolution (0.283 m/px; the camera GSD is 0.18 m/px so the frame is downsampled by 0.637; ratio recorded). Slight Gaussian blur before downsampling. 2. Retrieve: centre 384-px crop of the canvas, scaled 0.5 (192 px), MegaLoc descriptor (8448-d) → FAISS IndexFlatIP over 625 map tiles (512 px, stride 256) → top-5 tiles. 3. Match: XFeat (top 2048 keypoints, keypoints near the canvas black border dropped) vs XFeat features of a 1280-px map patch centred on the tile (cached) → LighterGlue matches. 4. Geometry: USAC-MAGSAC homography RANSAC for inliers, then planar PnP (IPPE + LM refinement, camera K from camera.json, map pixels in metres on z=0) → camera position (the fix), height, tilt, yaw. 5. Consistency gate: >=20 inliers, median reprojection

residual <=3 px, |PnP height - baro| <= max(12 m, 10 %), tilt <= 50 deg, |PnP yaw - compass| <= 15 deg (tilt-vs-IMU check only when attitude is trusted; off here). First candidate that passes with >=70 inliers stops the loop, otherwise the best passing candidate is used; else accepted=False (never a guess). 6. Tracking window (T4 config `tracking=true`): after a lock, retrieval is skipped; a map patch is cut around the constant-velocity prediction and matched directly (single match, extra gate: jump <= 30 m/s*dt+40 m, >=15 inliers). 3 consecutive misses → back to global retrieval. Data-flow stages: frame → [attitude/baro rectify, OpenCV] → [MegaLoc ViT-B/14 DINOv2 backbone + SALAD aggregator, PyTorch] → [FAISS flat IP] → [XFeat, PyTorch] → [LighterGlue, kornia] → [MAGSAC homography, OpenCV] → [IPPE planar PnP, OpenCV] → [gate] → [tracking state] -> lat/lon via map georef. Structure follows hloc (retrieve-match-PnP); the gate/tracking ideas are from S1 reimplemented, no code copied.

Setup facts

Tile index: 625 tiles x 8448 float32 = 21.1 MB; build 1088 s on shared CPU (1.7 s/tile, first run, 2 threads under load 9); cached afterwards. DINOv2-small index: 625 × 768 = 1.9 MB, 61 s. MegaLoc 228.6 M parameters (0.87 GB weights). Peak RSS ~2.1 GB (XFeat+LG) / 2.5 GB (SP+LG).

Results (synthetic NIDAR-like, nadir; metrics from harness/metrics.py)

Test	Run id	frames	fix rate	median err m	p95 m	max m	precision@100 m	note
S6-T1 offline, global retrieval every frame, 5 Hz	S6-T1-20261023-s0	764	92.8 %	0.41	1.09	6.76	100 %	estimates.csv for SITL
S6-T4 (offline accuracy run with tracking window)	S6-T4off-20261023-s0	764	95.3 %	0.35	1.01	5.36	100 %	736 track / 28 global frames; real-time run NOT done (lead)
S6-T3 XFeat+LighterGlue, 1 Hz	S6-T3-xfeatlg-20261023-s0	153	92.2 %	0.39	1.01	2.87	100 %	median inliers 270; latency p50 0.94 s
S6-T3 SuperPoint+LightGlue, 1 Hz	S6-T3-splg-20261023-s0	153	100 %	0.14	0.42	2.50	100 %	median inliers 507; latency p50 3.7 s
S6-T2 MegaLoc (1 Hz)	S6-T2-megaloc-	153	recall@1 84.3					embed p50 273 ms

Test	Run id	frames	fix rate	median err m	p95 m	max m	precision@100 m	note
	20261023-s0		%, @5 95.4 %					
S6-T2 DINOv2-small CLS+mean-patch (1 Hz)	S6-T2-dinov2s-20261023-s0	153	recall@1 39.9 %, @5 63.4 %					embed p50 59 ms
S6-T5 variants (CPU only)	S6-T5-20261023-s0							see below
T2 criterion: a tile (512 px) containing the true footprint centre. Time to first fix 0 s (first frame accepted). Full-run latency p50/p95 (shared load): T1 1.39/7.0 s per frame, T4off 1.77/8.2 s (the tracking path re-extracts the reference patch uncached each frame, so it is not faster on this CPU; the saving is the avoided retrieval and extra candidates). Not comparable to a Jetson.								
Fix rate is mostly bounded by our own gate (see failures): of 55 rejected frames in T1, 39 had a raw (rejected) pose within 5 m of truth, 5 were wrong by >50 m (haze), so the gate trades fix rate for 100 % precision.								

T3 caveat: SP+LG is more accurate and robust on this data but 4x slower on CPU, and the SuperPoint weights are research-licence only. T2 caveat: our DINOv2-small baseline is untrained for VPR (CLS + mean patch tokens); the gap to MegaLoc is partly that.

T5 variants (CPU, 2 threads, shared load, single process; "measured under shared CPU load")

Variant	p50 ms	note
MegaLoc PyTorch 196×196	265	
MegaLoc ONNX Runtime CPU	228	cosine vs PyTorch 1.000 (random input), 915 MB onnx
XFeat backbone PyTorch 1024×1024	200	
XFeat backbone ONNX Runtime CPU	64	max abs diff 1.2e-4
XFeat full extract PyTorch 1280×800	228	
LighterGlue PyTorch (2048 kpts)	278	
LighterGlue ONNX	export FAILED (torch.onnx IndexError in transpose; kornia LightGlue has data-dependent pruning)	1 attempt on that exporter, no workaround tried
TensorRT FP16 (any)	NOT RUN: no GPU/driver; `pip install tensorrt` fails (3 attempts, see SETUP.md)	
Jetson Orin (any)	NOT MEASURED	survey claims 15 Hz MegaLoc, 30+ Hz XFeat+LighterGlue are unverified here
Accuracy change of ONNX variants was checked on descriptor/feature outputs only, not end-to-end replay.		

Failure analysis (T1; figures: results/s6/figures/failures/*.png = frame | expected map crop at truth | map crop at the pipeline's rejected estimate)

Fix rate by condition (T1): clear 99.6 %, haze 90 %, blur-only 19.6 %, blur+haze 100 % (blur+haze windows overlap at 95-104 s where gaussian blur was probably weaker than in the pure blur window 21-31 s). 1. t=21.4-30 s (blur window peak sigma 3 px): ~40 frames rejected by the residual gate (median residual > 3 px) although 200-280 inliers and raw error < 1 m (t=21.4: raw err 0.4 m). Cause: our gate threshold, not the geometry → a loose gate would give ~+5 % fix rate; kept as run (no post-hoc tuning). 2. t=30.2 (end of blur): 24 inliers, raw error 23.6 m, correctly rejected (few inliers + residual). 3. t=52.4 (haze 49-69 s, strength 0.6): retrieval returned wrong tiles, 6 inliers, raw error 986 m, rejected by 4 gate checks. Haze lowers contrast so top-5 tiles missed. 4. t=56.0 and 57.2 (haze): same pattern, raw errors 1044 m / 1049 m, rejected (inliers 6). 5. t=55.0 (haze): 17 inliers, 20 m raw error, rejected (few_inliers). Also 57.4 s: 7 inliers, 175 m. The tracking run bridges these windows (95.3 % fix, no wrong accepted fix) because the constant-velocity window supplies the location prior; global retrieval is what fails in haze. No accepted fix exceeded 6.8 m error in T1 (precision@100 m = 100 %); accepted-fix precision is the strong point, fix availability under blur/haze the weak one.

Comparison with claims

- Survey "MegaLoc ~15 Hz and XFeat+LighterGlue 30+ Hz on Jetson Orin": NOT reproduced and not testable (no Jetson). On this 2-thread shared CPU: MegaLoc 3.8 Hz alone (265 ms), XFeat extract 4.4 Hz, LighterGlue match 3.6 Hz; end-to-end frame p50 1.4 s. Hardware classes differ by orders of magnitude, so no inference about Orin is drawn.
- No author accuracy claim exists for the composite itself (it is our design).

Verdict

Strengths: sub-metre median error on synthetic data, 100 % precision of accepted fixes, absolute (no drift), permissive licences for the main path (MIT/Apache; LighterGlue weight licence unconfirmed), modular (retriever/matcher swappable, ONNX path for XFeat/MegaLoc works). Weaknesses: heavy for CPU (MegaLoc 228 M params, 0.9 GB weights), global retrieval fails in haze, strict residual gate rejects blurred-but-correct fixes, needs height and heading (and tilt) hints, optimistic data (same map). Use: as an absolute-position corrector with an onboard prior (tracking mode) on GPU-class hardware; do not rely on global retrieval alone in low-contrast scenes or on CPU-only boards. Known risks observed: (a) the first (oblique, body-fixed camera) dataset made the nadir assumption fail completely (north-up NCC 0.1-0.2); attitude had to be known to rectify (probe of 7 oblique frames with true attitude: 427-574 inliers, <0.5 m: results/s6/nidar_oblique/probe_7frames.txt, stress result

"body-fixed 30-deg-oblique camera"); (b) hint yaw units (deg) and string-sorted frame names in the harness; (c) truth60.csv still carries the old -30 deg pitch; (d) T6 fallback dataset: not run (dataset unreachable, no open mirror attempted beyond the lead's note); (e) T7 SITL: not run by S6 (lead uses results/s6/nidar/S6-T1-20261023-s0/estimates.csv).

Commands for the lead (run ALONE, one at a time, from solutions/s6_jetson_composite with `source env/bin/activate; export OMP_NUM_THREADS=2`)

- T4 real-time replay with tracking: `python driver.py --run-id S6-T4-rt-<date>-s0 --mode realtime --cfg '{"tracking":true}'`
- Constrained (2 cores): `taskset -c 0,1 python driver.py --run-id S6-T4-c2-<date>-s0 --mode realtime --cfg '{"tracking":true}'` (add `systemd-run` /cgroup memory limit as desired)
- Clean latency (alone): `python driver.py --run-id S6-T1lat-<date>-s0 --stride 5`
- T5: `python t5_variants.py --run-id S6-T5-<date>-s0` Extra per-frame stage timings are in extras.csv of each run.

Setup account

S6 "our composite" -- setup log

"Our composite" is NOT an existing project. It is a Jetson-oriented baseline assembled by us from permissive parts: MegaLoc (retrieval) + FAISS tile index + XFeat/LighterGlue (matching) + planar PnP + consistency gate + tracking window. All data below is synthetic NIDAR-like. The map used for retrieval is the SAME map the video was rendered from (optimistic appearance gap).

Host: Intel Xeon 2.10 GHz, 4 cores shared with other agents (load avg 7-9 most of the time), 15 GB RAM, no GPU, Python 3.12 (uv venv; system python is 3.13).

Commands that worked (from `solutions/s6_jetson_composite`)

```
export PIP_CACHE_DIR=/tmp/pipcache
uv venv --python 3.12 env && source env/bin/activate
uv pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu      # torch 2.14.1+cpu
uv pip install numpy opencv-python-headless faiss-cpu pycolmap onnx onnxruntime
timm huggingface_hub pandas scipy kornia einops tqdm pillow safetensors
mkdir repo && cd repo
git clone --depth 1 https://github.com/verlab/accelerated_features   # XFeat +
```

```
LighterGlue weights live in the repo (weights/)
git clone --depth 1 https://github.com/gmberton/MegaLoc
git clone --depth 1 https://github.com/cvg/LightGlue && uv pip install -e
LightGlue
uv pip uninstall opencv-python opencv-python-headless && uv pip install opencv-
python-headless   # see failure 3
# MegaLoc weights (HF gberton/MegaLoc, model.safetensors, 0.87 GB) download on
first load into repo/hf
cd .. && python driver.py --run-id S6-T1-20261023-s0 --stride 1
```

Pins: COMMIT.txt (MegaLoc 5fe0dd6, accelerated_features e92685f, LightGlue eb42fee), lock.txt (pip freeze). Wall clock from clone to first working output: about 95 min (of which ~18 min MegaLoc index build under CPU contention, ~45 min tracking down the camera-geometry problem below). Failed steps: 6.

Failures and fixes

1. `torch.hub.load("gmberton/MegaLoc", ...)` → `HTTP Error 400: Bad Request` (torch.hub fetches the GitHub zip/branch via api.github.com, not reachable through the proxy). Fix: `git clone` (works) and `torch.hub.load(<local clone>, "get_trained_model", source="local")`; this still uses MegaLoc's own hubconf and downloads weights from Hugging Face (reachable). Equivalent code path, different source.
2. `pip install lightglue` → not on PyPI under that name. Fix: install from the git clone (cvg/LightGlue).
3. Installing LightGlue pulled `opencv-python` which shadows `opencv-python-headless` (twice, also after moving the clone). Fix: uninstall both, reinstall headless.
4. `uv pip install -e` pointed at a path I later moved (/tmp → repo/); reinstalled from the final path.
5. TensorRT: `pip install tensorrt` → sdist build aborts, "nvidia-smi command not found". `tensorrt-cu12==10.3.0` → same build error (needs NVIDIA index + driver). `--no-deps tensorrt-cu12-libs` → wheel extraction `No space left on device` (disk was shared and full). Three attempts, all failed; removed. No FP16 engine can be built without a GPU anyway.
6. Disk: uv cache reached 4 GB; purged (`uv cache clean`) after lead's notice. Env 1.6 GB, repo (incl. 0.87 GB weights) 1 GB.
7. Own mistakes: a `pkill -f driver.py` killed my own shell; deleting the symlink frame dir of a running job broke it (restarted).

Data defects found (reported to lead)

- Original frames were rendered with a body-fixed camera while the quad pitches -30 deg in cruise (truth60 pitch -0.52 rad) → strongly oblique. Re-rendered by lead as gimbal nadir. NOTE: truth60.csv still contains the old pitch; our final runs use `att_source=hints` (roll=pitch=0) and ignore truth60.
- Harness hint yaw is in degrees (adapter converts). run_replay sorts file names as strings; our driver symlinks frames with zero-padded names.

Licences

Component	Licence	Notes
MegaLoc code + weights (gberton, HF gberton/MegaLoc)	MIT (repo LICENSE); weights trained on public VPR datasets - weight licence not separately stated	
XFeat code (verlab/accelerated_features)	Apache-2.0	
XFeat weights `xfeat.pt`	repo says Apache-2.0 (single repo licence)	
LighterGlue weights `xfeat-lighterglue.pt`	NOT separately confirmed; repo licence Apache-2.0, but derives from LightGlue (Apache-2.0 code; SuperPoint-trained LightGlue weights carry non-commercial terms) → treat as unconfirmed	record
LightGlue (cvg) / SuperPoint weights (used only in T3 swap)	LightGlue Apache-2.0; SuperPoint weights (MagicLeap) research-only licence	not for product
DINOv2-small (timm, T2 swap)	Apache-2.0	
hloc (structure only, no code used)	BSD-3	
S1 gnss-denied-nav (ideas only)	MIT	
FAISS	MIT; PyTorch BSD; OpenCV Apache-2.0; kornia Apache-2.0; onnxruntime MIT	

Reproducibility verdict

Reproducible given the commit hashes, lock.txt and seeds (no stochastic components apart from OpenCV RANSAC, which is not seeded by cv2 → bit-exact reproduction not guaranteed; seed arg is recorded). Index build is cached in repo/cache (not committed).

Documentation-quality rubric (0-3)

- Install steps work as written (ours, composite of 3 READMEs): 2 (torch.hub path fails behind this proxy; opencv conflict)
- Config explained: 3 (s6lib.DEFAULT is commented, every key used)
- Results reproducible: 2 (RANSAC unseeded, latency depends on shared load)
- Algorithm described: 3 (ANALYSIS.md written from the code)

S7. Paper-only and special-purpose items (not ranked)

S7 review: paper-only and special-purpose items

Compared on documentation, feasibility and relevance. NOT ranked against S1-S6; no shared accuracy numbers. All claimed numbers are author claims, unverified, unless stated. Date 2026-10-23. CPU: shared 4-core VM (lscpu model not exposed to agent), 2 threads, CPU only.

1. YFS90 terrain-weighted geolocalisation (documentation review only)

- Sources: repo https://github.com/YFS90/GNSS-Denied-UAV-Geolocalization (HEAD 968eaf8, 2024-11-28, single commit line, branch `main` only, no tags); paper Yao, Lan, Wang, Wan, Gao, Wei, "GNSS-denied geolocalization of UAVs using terrain-weighted constraint optimization", Int. J. Applied Earth Observation and Geoinformation 135, 104277 (Dec 2024), doi 10.1016/j.jag.2024.104277. Full text was not retrievable here (ScienceDirect 403); the summary below uses the repo README and the published abstract, so it is high level.
- Plain-language method: the UAV image stream is (a) matched to publicly available satellite map tiles (the map is cut into chunks for memory management), (b) tracked frame to frame by visual odometry, and (c) a 2D-3D geo-registration step ("bidirectional homologous point search") lifts matched points to 3D "virtual control points" using public DEM terrain heights. A sliding-window optimisation then fuses matches and odometry, with each constraint weighted by terrain (so flat or featureless terrain is trusted less). Claimed to need no altimeter or IMU, no loop closure, no error accumulation, and to re-localise after a failure; also claimed to tolerate tilt, rotation, altitude change, season and light change.
- Code and weights: NONE. Verified: repo tree contains only README.md and Img/ (images, GIF, 20 result figures with bilibili/YouTube video links); `git ls-remote` shows only refs/heads/main; no tags or releases; README links no code, data or weights. The author's other public repo `Real-time-orthophoto` is a different paper (GPS-free orthophoto SLAM), not this system. We did not reimplement.
- Licence: no LICENSE file; all rights reserved by default, so reuse is not permitted. Paper is journal-published (check ScienceDirect access terms).
- Claimed data/results (unverified): 20 scenarios (plains, hills, urban, rural, multiple viewpoints, seasons, thermal infrared night); abstract claims mean absolute error under 7 m; README claims accuracy "comparable to GPS". No per-scenario table seen by us.

- Feasibility on Jetson Orin class: unknown; no runtime reported in the material we could read, and without code it cannot be tested; a reimplementation would be a multi-month project (VO + DEM registration + windowed optimisation).
- Relevance to NIDAR (200 m nadir, 1280×800 grey, 0.18 m/px): conceptually highly relevant (satellite matching plus VO plus no IMU need, absolute fixes with no drift), and DEM terrain weighting matters little over the flat Pune site. Currently unusable: nothing to run.
- Documentation rubric 0-3: install steps 0 (no code); config explained 0; results reproducible 0 (videos only); algorithm described 1 (README is a marketing summary; detail lives in the paywalled paper).

2. Bearing-UAV (CVPR 2026)

- Sources: paper "Beyond Matching to Tiles: Bridging Unaligned Aerial and Satellite Views for Vision-Only UAV Navigation", Liu et al., arXiv 2603.22153 (CVPR 2026); code https://github.com/liukejia121/bearinguav (HEAD d16e558, 2026-05-01); weights https://huggingface.co/HaoyZhou/bearinguav (Bearing_UAV.zip, 131 MB, ungated, no HF licence tag); dataset Bearing-UAV-90K on HF (single 6.3 GB zip, NOT downloaded; we read only the 225-row mini metadata and 3 sample blocks via HTTP range to learn the input format).
- Plain-language method: it is not a matcher. A VGG-16 backbone (frozen, ImageNet) plus a non-local/clustering head turns each of five 256×256 images into a descriptor: four satellite tiles forming a 2×2 block and one UAV patch. A relative-coordinate encoder, patch-similarity prior and cross-attention fuse them, and two regression heads output the UAV position inside the block and heading as (cos, sin). A "Bearing-Naver" scheme steps along waypoints using it. Needs a known starting block (prior), no map-wide search.
- Code/weights: both released. Licence Apache-2.0 (code); dataset/weights licence not stated. Weights run on CPU (about 133 ms per step in the paper on GPU; here median 4.4 s per sample on a loaded shared 4-core CPU at 2 threads).
- Claimed results (unverified, author claims, own Google-Earth-3D-rendered data): mean localisation error 8.61 m (7.48 m with weather aug), mean heading error 12.90 deg, Recall@1 83.2%, LSR@15 89.4%, HSR@15 77.2%; waypoint navigation success only about 50% (SR@20), navigation error 275 m. Model about 68 MB, 10.5 GFLOPs at 256×256.
- Our run (labelled SYNTHETIC NIDAR-like, out of distribution; results/s7/bearing/S7-B-20261023-s0/): driver `solutions/s7_paper_only/bearing_driver.py`. Four 64 m tiles cut from the Esri reference map (same map the video was rendered from, so optimistic) around an oracle block centre placed at truth plus a seeded random offset (+-0.9 units = +-29 m); UAV patch = central 64 m (356 px) crop of our frame resized to 256. 153

frames (1 Hz subset, seed 0). Metrics via harness/metrics.py summarize(): median position error 28.2 m, p95 48.8 m, RMSE 30.2 m, max 59.7 m, 100% "fix" rate (the model always answers). Trivial baseline "predict block centre" (the oracle prior itself): median 23.1 m, mean 22.2 m. So the model is NOT better than the prior here. Heading: median error 93.6 deg, mean 96.2 deg, i.e. chance level (random is about 90 deg); heading convention is assumed (see caveats), and the nadir 64 m crop of a straight-down 1280×800 image does not look like the paper's Google-Earth 3D rendered views.

- Sanity check of the code path (results/s7/sanity/dataset_samples.json): on 3 samples from the paper's own mini dataset the released weights give 3.9, 1.5, 1.6 m and 2.4, 11.2, 2.7 deg errors (3 samples only; may be training data; shows weights and driver load correctly).
- Caveats: tile geometry assumed contiguous 64 m tiles at p1..p4 = (-1,-1),(-1,+1),(+1,-1),(+1,+1) in (x east, y south) units of 32 m; the sample tiles looked non-contiguous so the real layout may differ; heading frame (image-up = heading) assumed; scale 0.25 m/px per 256 px assumed from README/config; the 64 m footprint is much narrower than our true 230×144 m footprint; sat tiles from different imagery source than training (Google Earth). The negative result therefore means "no evidence of use out of the box", not "the method cannot work".
- Feasibility on Jetson Orin class: yes, small VGG-16 style network (68 MB, 10.5 GFLOPs), likely tens of ms on Orin GPU; but needs retraining for our camera and a known-start block.
- Relevance to NIDAR: low-to-medium; designed for oblique/3D-rendered urban views with a known starting block, only about 50% waypoint success in its own paper, regression gives no confidence/inlier check; fine-tuning on NIDAR-like data would be needed (not done: instructed not to train).
- Documentation rubric 0-3: install steps 1 (README lists CUDA 11.8 / Python 3.9 pins; Python 3.13 and torch 2.1.2 not installable, we used py3.11 CPU torch 2.1.2; imgaug/albumentations imported at module load although unused for inference; ImageNet VGG16 weights silently downloaded at model build, 528 MB); config explained 2 (JSON config shipped with weights); results reproducible 1 (test script and weights exist but need the 6.3 GB zip and GPU-style paths, no inference-on-your-own-images example); algorithm described 2 (paper plus architecture tree; heading/tile conventions undocumented).
- Setup effort: about 45 min wall clock, 4 failed steps: (1) gh API access to repo denied (used add_repo then git clone), (2) pip torch 2.1.2 not available for Python 3.13 (used 3.11 venv), (3) two installs failed with "No space left on device" (shared disk dipped to 185 MB free; retried after space returned), (4) imgaug missing (stubbed). Env 1.4 GB, weights 262 MB (git-ignored).

3. STHN thermal homography

- Sources: repo https://github.com/arplaboratory/STHN (HEAD 0b44a84, 2026-05-16); paper Xiao, Zhang, Tortei, Loianno, IEEE RA-L 9(10) 2024, arXiv 2405.20470 (v3 Sep 2025); weights on HF: xjh19972/STHN (one_stage 6.5 MB, two_stages 12.3 MB safetensors) and xjh19972/STHN-one-stage; dataset (122 GB) NOT downloaded, as instructed.
- Plain-language method: given a thermal image from straight-down UAV camera and a larger satellite crop around the last known position, a CNN predicts how the four corners of the thermal image move to land on the satellite image (homography via DLT). Coarse stage: both images resized to 256 px, iterative correlation-based updates (6 iterations). Optional fine stage: crop the satellite image around the coarse result and refine. A pix2pix-style "thermal generative module" synthesises fake thermal images from satellite tiles to enlarge training data. The location is the image centre mapped into the satellite frame. Works within a 512 m search radius of the last known position, even with the thermal image only 11% the satellite size.
- Code/weights: both released. Licence MIT (code, LICENSE file "Copyright 2024 ARPL"); data/weights licence not stated. Demo script STHN_demo.py runs on one sample pair (needs conda env.yml, torch, kornia).
- Claimed results (unverified): MACE 12.70 m (two-stage, 512 m offset, 0% failures), 16.42 m (one-stage), localisation error 12.12 m at 63.9 ms on an RTX 2080 Ti; keypoint baselines fail 83-99.7%.
- Run: not run. Not applicable: models are trained for 8-bit thermal (Boson, about 1 m/px, night) and our payload is visible grayscale at 0.18 m/px, so a thermal-trained model would be out of domain; running it would not produce meaningful evidence. Weights existence verified by the HF listing only.
- Feasibility on Jetson Orin class: yes; tiny models (6.5-12 MB) and paper reports 64 ms on a desktop GPU.
- Relevance to NIDAR: not applicable to the visible payload; useful only if a thermal camera is ever added (night operation). The homography-regression idea (bounded search around last position, no dense tile search) is conceptually transferable, but would need retraining on visible-to-satellite pairs.
- Documentation rubric 0-3: install 2 (env.yml plus demo that pulls weights automatically); config explained 2 (scripts per setting, HF config.json); results reproducible 2 (weights, 122 GB data and train/eval scripts; slurm-oriented); algorithm described 3 (paper, README architecture notes, code links).

Summary table

Item	Source	Procedure	Run?	Result	Licence
YFS90 terrain-	github YFS90/GNSS-Denied-UAV-	doc review only	no (no code exists)	claims <7 m MAE over 20 scenarios (unverified)	none (all rights reserved)

Item	Source	Procedure	Run?	Result	Licence
weighted geoloc.	Geolocalization; IJAEOG 2024				
Bearing-UAV	github liukejia121/bearinguav; arXiv 2603.22153; HF HaoyZhou/bearinguav	inference with released weights on synthetic NIDAR-like frames	yes, 153 frames (1 Hz), synthetic, oracle block prior	median 28.2 m vs 23.1 m for block-centre baseline; heading 93.6 deg median (chance); own-data sanity 1.5-3.9 m on 3 samples	Apache-2.0 code; weights/data unstated
STHN	github arplaboratory/STHN; RA-L 2024; HF xjh19972/STHN	review, weights check	no (thermal only, not applicable)	claims 12.7 m MACE at 512 m offset on thermal (unverified)	MIT code; weights/data unstated

Reproduction: `cd solutions/s7_paper_only; env/bin/python -I bearing_driver.py nidar --out ../../results/s7/bearing/<run-id> --step 5 --seed 0` (needs weights/Bearing_UAV and clones/bearinguav, both git-ignored; ImageNet VGG16 weights auto-download).

3. Cross-solution comparison

3.1 Master table: every solution on the same 153 frames (1 Hz, t = 0..152 s), metric code `harness/metrics.py`

Recall is over all 153 frames (a frame without a fix counts as a miss). Median and p95 are over accepted fixes only, so read them together with the fix rate and recall.

Solution	frames	fix rate %	accepted-fix median m	p95 m	recall@10 %	recall@50 %	recall@100 %	accepted precision@100 %	time to first fix s
S1 koksalan/gnss-denied-nav	153	22.2	1.40	7.0	22.2	22.2	22.2	100.0	0.0
S2 uav-geolocalization (CAEVL + VIO)	153	100.0	804.96	1256.2	17.0	17.0	17.0	17.0	0.0
S3 tile-matching (Shuhaieva)	153	34.0	0.07	452.0	29.4	29.4	29.4	86.5	0.0
S4 hmakelin/gisnav	–	not run	–	–	–	–	–	–	–
S5 SWA-PF	153	93.5	2.46	6.4	92.2	93.5	93.5	100.0	10.0
S6 our composite (Jetson baseline)	153	92.2	0.39	1.0	92.2	92.2	92.2	100.0	0.0

3.2 Full-rate runs (frames at the rate each solution actually ran; see run id)

Solution	run	frames	fix rate %	median m	p95 m	RMSE m	max m
S1 koksalan/gnss-denied-nav	`S1-T1-20261023-s0`	153	22.2	1.40	7.0	3.1	7.6
S2 uav-geolocalization (CAEVL + VIO)	`S2-T3-20261023-s0`	764	100.0	804.83	1260.6	797.6	1344.8

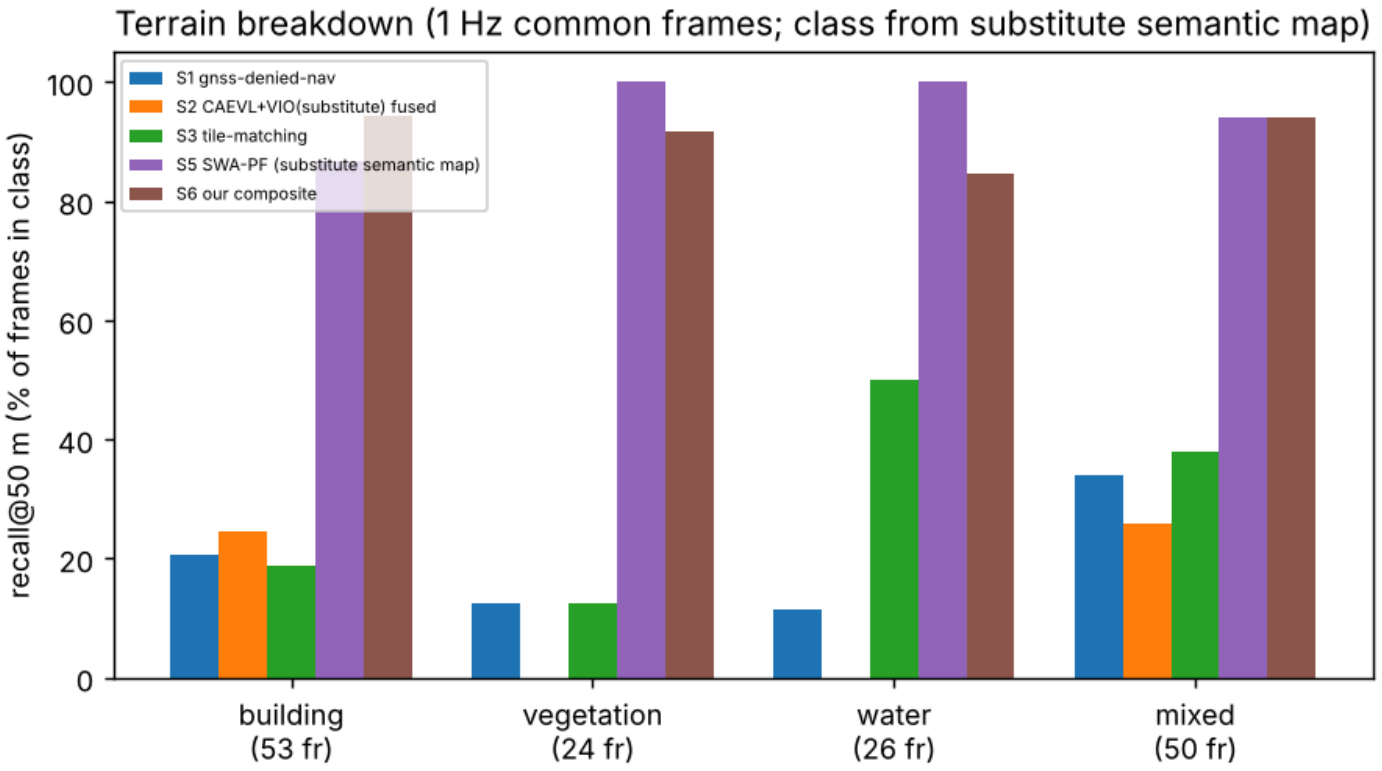
Solution	run	frames	fix rate %	median m	p95 m	RMSE m	max m
S3 tile-matching (Shuhaieva)	`S3-T1-20261023-s0`	153	34.0	0.07	452.0	200.6	989.6
S5 SWA-PF	`S5-T2-20261023-s0`	153	93.5	2.46	6.4	3.5	11.3
S6 our composite (Jetson baseline)	`S6-T1-20261023-s0`	764	92.8	0.41	1.1	0.7	6.8

3.3 Weather: blur and haze windows against clear flight

Solution	frames in blur/haze windows	recall@100 % (windows)	fix rate % (windows)	frames clear	recall@100 % (clear)	fix rate % (clear)
S1 koksalan/gnss-denied-nav	63	7.9	7.9	90	32.2	32.2
S2 uav-geolocalization (CAEVL + VIO)	63	9.5	100.0	90	22.2	100.0
S3 tile-matching (Shuhaieva)	63	22.2	25.4	90	34.4	40.0
S5 SWA-PF	63	100.0	100.0	90	88.9	88.9
S6 our composite (Jetson baseline)	63	81.0	81.0	90	100.0	100.0

3.4 Terrain breakdown (class under the true position from the substitute semantic map, not ground-truth land cover)

Solution	building (53 fr) recall@50 %	vegetation (24 fr) recall@50 %	water (26 fr) recall@50 %	mixed (50 fr) recall@50 %
S1 koksalan/gnss-denied-nav	20.8	12.5	11.5	34.0
S2 uav-geolocalization (CAEVL + VIO)	24.5	0.0	0.0	26.0
S3 tile-matching (Shuhaieva)	18.9	12.5	50.0	38.0
S5 SWA-PF	86.8	100.0	100.0	94.0
S6 our composite (Jetson baseline)	94.3	91.7	84.6	94.0

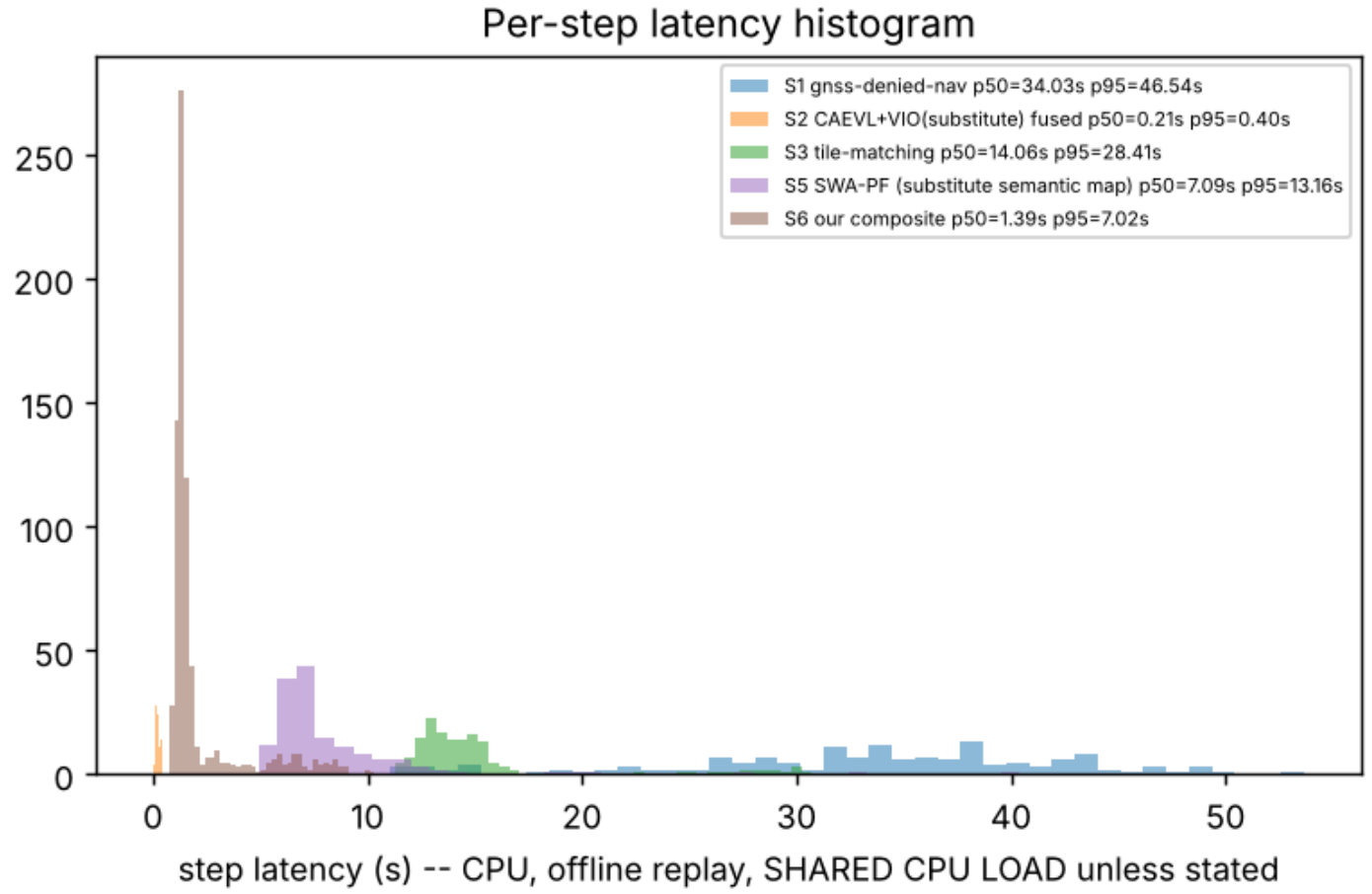


Terrain breakdown: recall@50 m per class (1 Hz common frames; substitute semantic map)

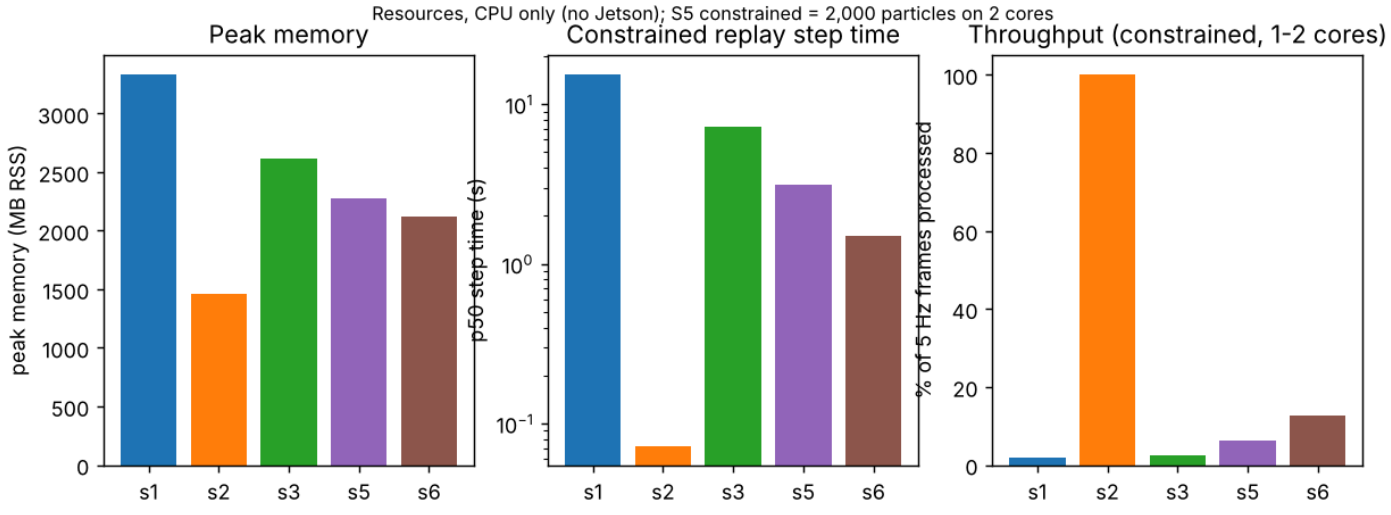
3.5 Speed and resources (CPU only, serial runs on an otherwise idle machine; NO Jetson)

Solution	clean offline p50 / p95 step (s)	real-time 5 Hz: frames processed %	constrained (1-2 cores) p50 step s	constrained frames processed %	peak RSS MB	startup / index build s	notes
S1 koksalan/gnss-denied-nav	8.79 / 11.24	3.0	15.47	1.8	3331	30	tracking config, 1,024 query keypoints, DINOv2-base; tile index cached (first build 15 min); constrained = 1 core
S2 uav-geolocalization (CAEVL + VIO)	0.06 / 0.08	100.0	0.07	100.0	1462	6	OUR VIO substitute + CAEVL on 120 m tiles; steps are cheap, the fix quality is not; constrained = 1 core
S3 tile-matching (Shuhaieva)	5.61 / 11.73	3.3	7.24	2.5	2613	3	embedding index cached (first build 700 s); constrained = 1 core
S4 hmakelin/gisnav	not run	–	–	–	–	–	–
S5 SWA-PF	15.87 / 31.61	6.3	3.13	6.3	2272	5	offline row = first 8 frames at 20,000 particles (first step dominates); constrained row = 2,000 particles on 2 cores (the particle count is reduced; not comparable to the others); real-time column uses the constrained run

Solution	clean offline p50 / p95 step (s)	real-time 5 Hz: frames processed %	constrained (1-2 cores) p50 step s	constrained frames processed %	peak RSS MB	startup / index build s	notes
S6 our composite (Jetson baseline)	0.90 / 5.02	20.7	1.49	12.6	2117	5	tracking config; constrained = 1 core

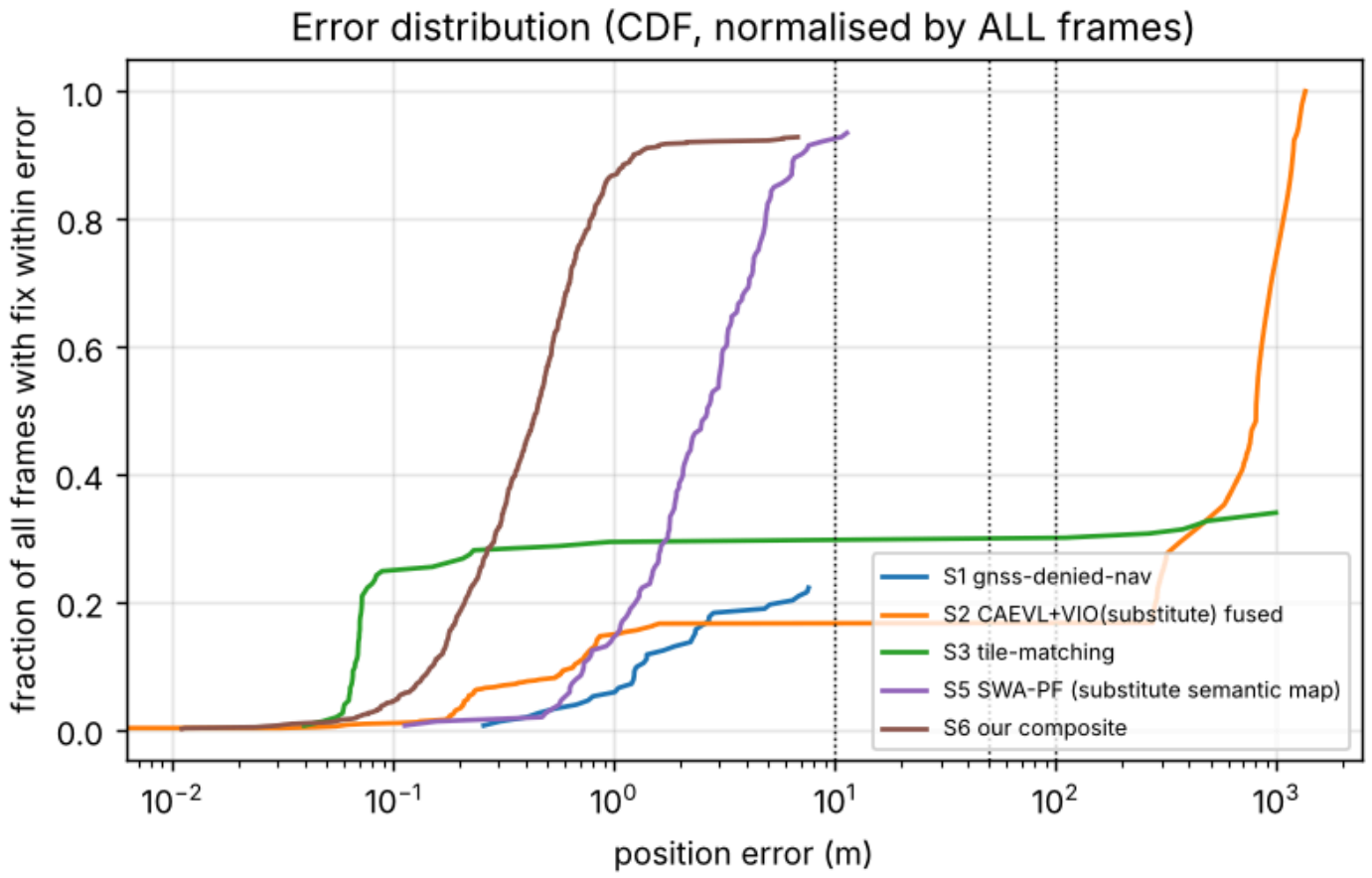


Per-step latency histograms (offline replay; shared-CPU load unless the table says otherwise)

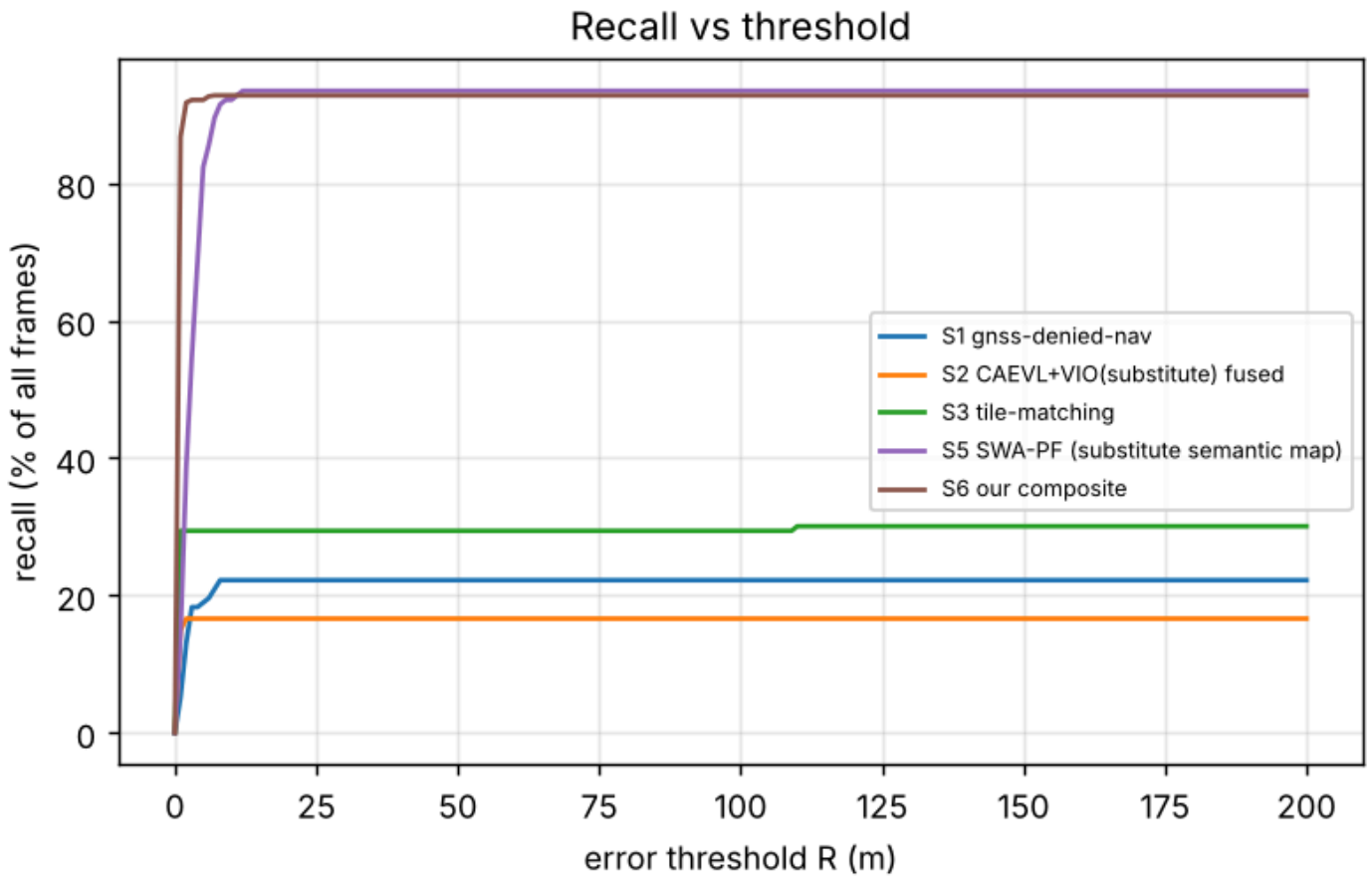


Peak memory, constrained step time and throughput of every solution that ran (CPU only, no Jetson)

3.6 Accuracy figures



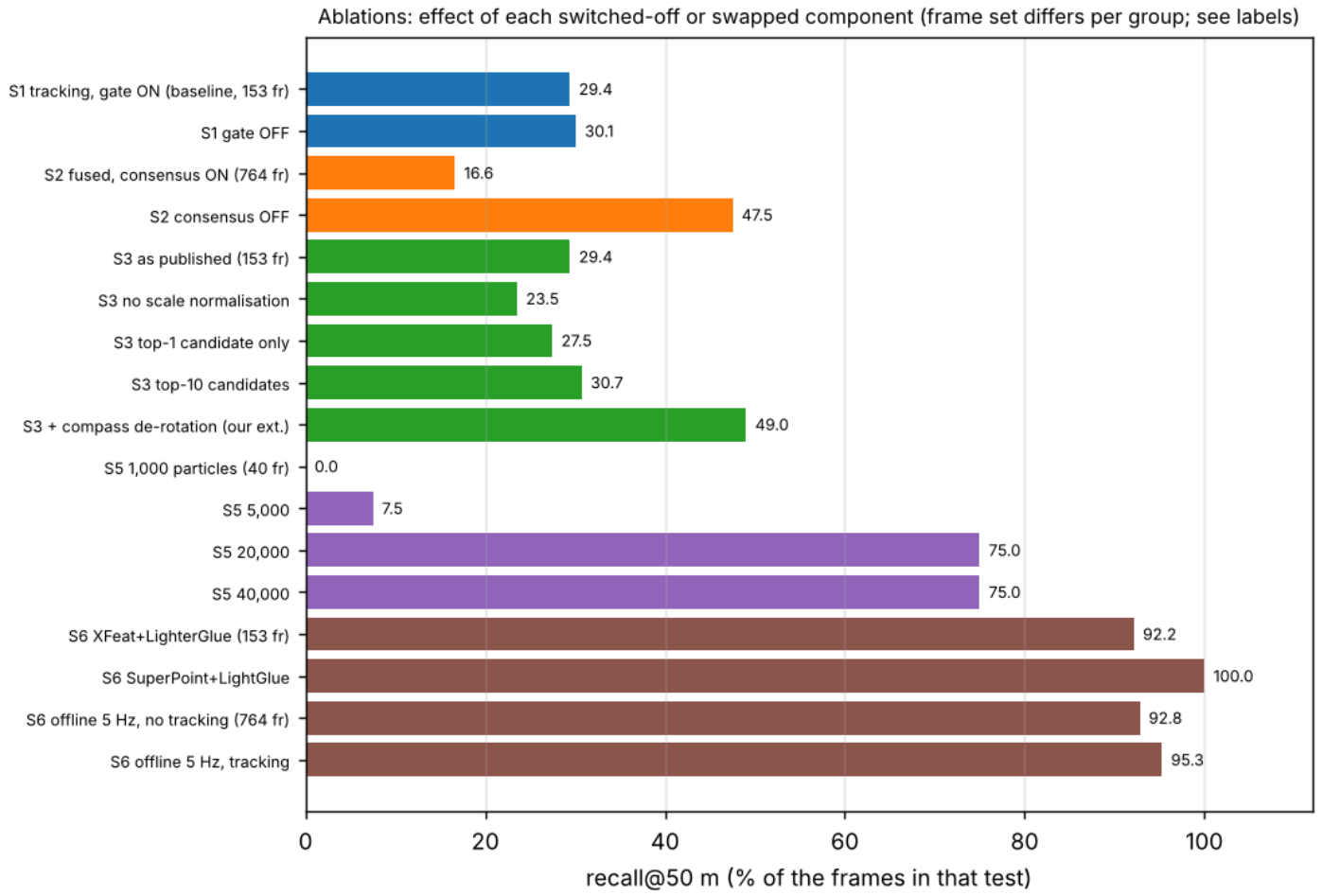
Error distribution of all solutions (dataset: synthetic NIDAR-like; primary runs; normalised by all frames of each run)



Recall against error threshold, all solutions

3.6b Ablations

Each group compares a baseline with the switched-off or swapped component; frame counts differ per group because the CPU cost forced 1 Hz subsets for some tests.



Ablation chart: recall@50 m for each test variant (synthetic NIDAR-like; labels give frame counts)

3.7 Documentation, licence and setup effort side by side

Solution	README / docs quality (0-12)	install / config / reproducible / algorithm	setup minutes (failed steps)	licence class	licence detail	maturity (0-10)
S1 koksalan/gnss-denied-nav	7	1 / 2 / 1 / 3	25 (6)	non-commercial	MIT code; SuperPoint weights research-only (non-commercial); author's fine-tuned DINOv2 not published	3 (13 commits, single author, thresholds tuned on one flight, no Jetson path)
S2 uav-geolocalization (CAEVL + VIO)	7	1 / 2 / 1 / 3	95 (7)	none	no LICENSE in uav-geolocalization or CAEVL fork (treated as all rights reserved); OpenVINS GPLv3 (copyleft)	3 (19 commits; OpenVINS last release v2.7 (May 2023))
S3 tile-matching (Shuhaieva)	6	1 / 1 / 1 / 3	45 (3)	none	no licence stated; notebook only; pretrained DINOv2/ALIKED/LightGlue components are Apache-2.0/BSD	2 (single Colab notebook, no tests, no config file)
S4 hmakelin/gisnav	5	1 / 2 / 1 / 1	n/a (never ran) (3)	permissive	MIT code; PX4/Gazebo stack heavy; ArduPilot support marked unmaintained	7 (2,793 commits, 92 stars, one main contributor; sim only, warns against real flights)
S5 SWA-PF	5	2 / 1 / 0 / 2	25 (3)	none	no licence for code, weights or MAFS	3 (research code; demo leaks ground-truth pose; dataset behind Baidu Pan)

Solution	README / docs quality (0-12)	install / config / reproducible / algorithm	setup minutes (failed steps)	licence class	licence detail	maturity (0-10)
S6 our composite (Jetson baseline)	10	2 / 3 / 2 / 3	95 (6)	permissive	MegaLoc MIT, XFeat Apache-2.0, hloc BSD; LighterGlue weight licence unconfirmed; SuperPoint swap is non-commercial	5 (our composite of maintained parts; built here, no field history)

3.8 Weighted ranking and sensitivity

Scores are min-max scaled to 0-10 across the solutions that ran each test; a solution that could not run a test gets 0 for it and is shown as *not run*. Weights are the owner's (30/20/20/15/15).

- **Accuracy** = mean of median error (accepted fixes), p95 error and recall@50 m, on the common 153 frames.
- **Robustness** = mean of accepted-fix precision@100 m, recall@100 m inside blur/haze windows, and the worst terrain-class recall@50 m. (The failure-injection tests differ between solutions, so the shared weather windows are the common basis.)
- **Real-time and edge fit** = mean of constrained-replay p50 latency, real-time processed-frame share and peak memory. CPU only; no Jetson number exists.
- **SITL outcome** = mean of mission-completion share and (log10) maximum EKF error over SIM-2 to SIM-7, 3 seeds each.
- **Integration, documentation and licence** = mean of setup effort (minutes + 10 per failed step), documentation rubric, licence class (permissive 10, copyleft 5, non-commercial 3, none declared 1) and maturity (a judgement from the repository facts, 0-10).

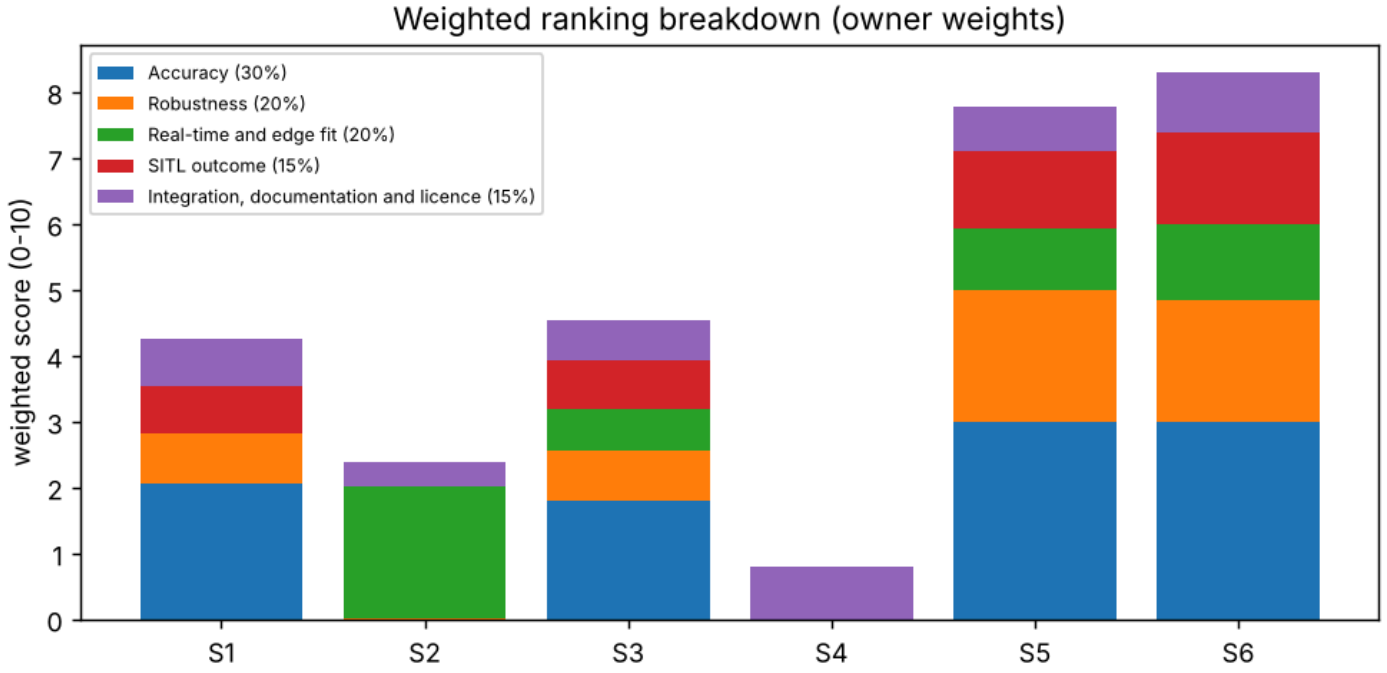
Rank	Solution	Weighted score (0-10)	Accuracy (30%)	Robustness (20%)	Real-time and edge fit (20%)	SITL outcome (15%)	Integration, documentation and licence (15%)
1	S6 our composite (Jetson baseline)	**8.29**	9.9	9.2	5.8	9.3	6.1
2	S5 SWA-PF	**7.78**	10.0	10.0	4.7	7.8	4.5
3	S3 tile-matching (Shuhaieva)	**4.53**	6.0	3.8	3.1	5.0	4.0
4	S1 koksalan/gnss-denied-nav	**4.25**	6.9	3.8	0.0	4.8	4.8
5	S2 uav-geolocalization (CAEVL + VIO)	**2.38**	0.0	0.1	10.0	0.0	2.5
6	S4 hmakelin/gisnav	**0.79**	0.0	0.0	0.0	0.0	5.3

Weight raised by 10 points	winner	order
Accuracy	S6	S6 8.53 > S5 8.10 > S3 4.74 > S1 4.61 > S2 2.07 > S4 0.66
Robustness	S6	S6 8.44 > S5 8.11 > S3 4.46 > S1 4.22 > S2 2.07 > S4 0.66
Real-time and edge fit	S6	S6 8.01 > S5 7.44 > S3 4.37 > S1 3.75 > S2 3.32 > S4 0.66

Weight raised by 10 points	winner	order
SITL outcome	S6	S6 8.45 > S5 7.83 > S3 4.61 > S1 4.35 > S2 2.07 > S4 0.66
Integration, documentation and licence	S6	S6 8.04 > S5 7.42 > S3 4.49 > S1 4.35 > S2 2.37 > S4 1.32

Per-criterion scores:

Solution	Accuracy	Robustness	Real-time and edge fit	SITL outcome	Integration, documentation and licence
S1 koksalan/gnss-denied-nav	6.87	3.78	0.00	4.80	4.78
S2 uav-geolocalization (CAEVL + VIO)	0.00	0.06	10.00	0.00	2.46
S3 tile-matching (Shuhaieva)	6.01	3.79	3.07	5.00	4.05
S4 hmakelin/gisnav	0.00	0.00	0.00	0.00	5.29
S5 SWA-PF	9.98	10.00	4.67	7.81	4.54
S6 our composite (Jetson baseline)	9.94	9.23	5.80	9.31	6.06



Weighted score by criterion for every solution; the sensitivity table lists the order under each weight change

4. SITL results (ArduCopter 4.5.7, scenarios SIM-0 to SIM-7, 3 seeds)

Method. ArduCopter 4.5.7 SITL at 8x speed. Take-off to 200 m, then the A→B→C→D mission in GUIDED at 12 m/s. A second GPS (`GPS_TYPE2 = 14`, MAVLink) receives `GPS_INPUT` messages from `harness/sitl_bridge.py` (accepted fixes only; velocity, altitude and accuracy-of-speed fields ignored; horizontal accuracy from the solution's confidence; each

fix projected forward to the send time with the EKF velocity). EKF3 is set to position-only GPS (`EK3_SRC1_POSXY=3`, `VELXY=0`, `POSZ=1` baro), `EK3_GPS_CHECK=0`. The real GPS is cut with `SIM_GPS_DISABLE=1` at mission time t = 60 s (t = 0 in SIM-6) and the second GPS becomes primary. A run **fails** when the EKF raises a failsafe that ends the mission (LAND) or the vehicle leaves the planned corridor by more than 200 m. Final parameter files, bridge logs, EKF-vs-truth traces and dataflash logs (seed 0, git-ignored) are in `results/<row>/sitl/<scenario>-s<seed>/`.

ID	Scenario
SIM-0	GPS stays on; solution only logged (fixes not sent)
SIM-1	GPS cut at 60 s, no vision: dead-reckoning control
SIM-2	GPS cut at 60 s, solution feeds `GPS_INPUT` (main test)
SIM-3	as SIM-2, solution output dropped in 20 s windows (80-100 s and 125-145 s)
SIM-4a/4b	as SIM-2 with 200 ms / 500 ms added latency
SIM-5	as SIM-2 with 5 % of fixes moved by 100-300 m
SIM-6	GPS cut at the start of cruise (cold start; take-off cut not possible, see deviations)
SIM-7	full mission with GPS cut at 60 s, then return-to-launch with no vision on the way back (the recorded video ends at D)

Rows other than S1-S6: a **control** (ground truth plus 5 m noise as fixes) shows what the EKF achieves with ideal fixes; "S2 VIO-only" is the S2 odometry track without map fixes; "S3 + compass de-rotation" is our extension. These rows are not ranked.

Maximum EKF position error after GPS loss (m, mean ± sd over seeds)

Row	SIM-0	SIM-1	SIM-2	SIM-3	SIM-4a	SIM-4b	SIM-5	SIM-6	SIM-7
Control: truth + 5 m noise fixes	0 ± 0	3 ± 0	12 ± 0	12 ± 0	12 ± 0	12 ± 0	12 ± 0	4 ± 0	18 ± 0
S1	0 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	18 ± 0	3 ± 0
S2 (fused)	0 ± 0	3 ± 0	1340 ± 0	1341 ± 0	1341 ± 0	1341 ± 0	1341 ± 0	909 ± 1	1341 ± 0
S2 VIO-only track (no map fixes)	0 ± 0	3 ± 0	6 ± 0	5 ± 0	7 ± 0	7 ± 0	30 ± 8	19 ± 0	6 ± 0
S3 (as published)	0 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	3 ± 0	4 ± 0	3 ± 0
S3 + compass de-rotation (our extension)	0 ± 0	3 ± 0	447 ± 217	554 ± 0	102 ± 53	216 ± 236	103 ± 56	104 ± 50	66 ± 54
S5	0 ± 0	3 ± 0	13 ± 0	7 ± 0	14 ± 1	13 ± 1	117 ± 121	2 ± 0	14 ± 1
S6	0 ± 0	3 ± 0	3 ± 0	5 ± 0	3 ± 0	3 ± 0	28 ± 5	6 ± 0	20 ± 0

Mission completed (share of seeds whose true position came within 30 m of waypoint D before any EKF failsafe/LAND)

Row	SIM-0	SIM-1	SIM-2	SIM-3	SIM-4a	SIM-4b	SIM-5	SIM-6	SIM-7
Control: truth + 5 m noise fixes	100%	0%	100%	0%	100%	100%	100%	100%	100%
S1	100%	0%	0%	0%	0%	0%	0%	0%	0%
S2 (fused)	100%	0%	0%	0%	0%	0%	0%	0%	0%
S2 VIO-only track (no map fixes)	100%	0%	0%	0%	0%	0%	0%	0%	0%
S3 (as published)	100%	0%	0%	0%	0%	0%	0%	0%	0%
S3 + compass de-rotation (our extension)	100%	0%	0%	0%	0%	0%	0%	0%	0%

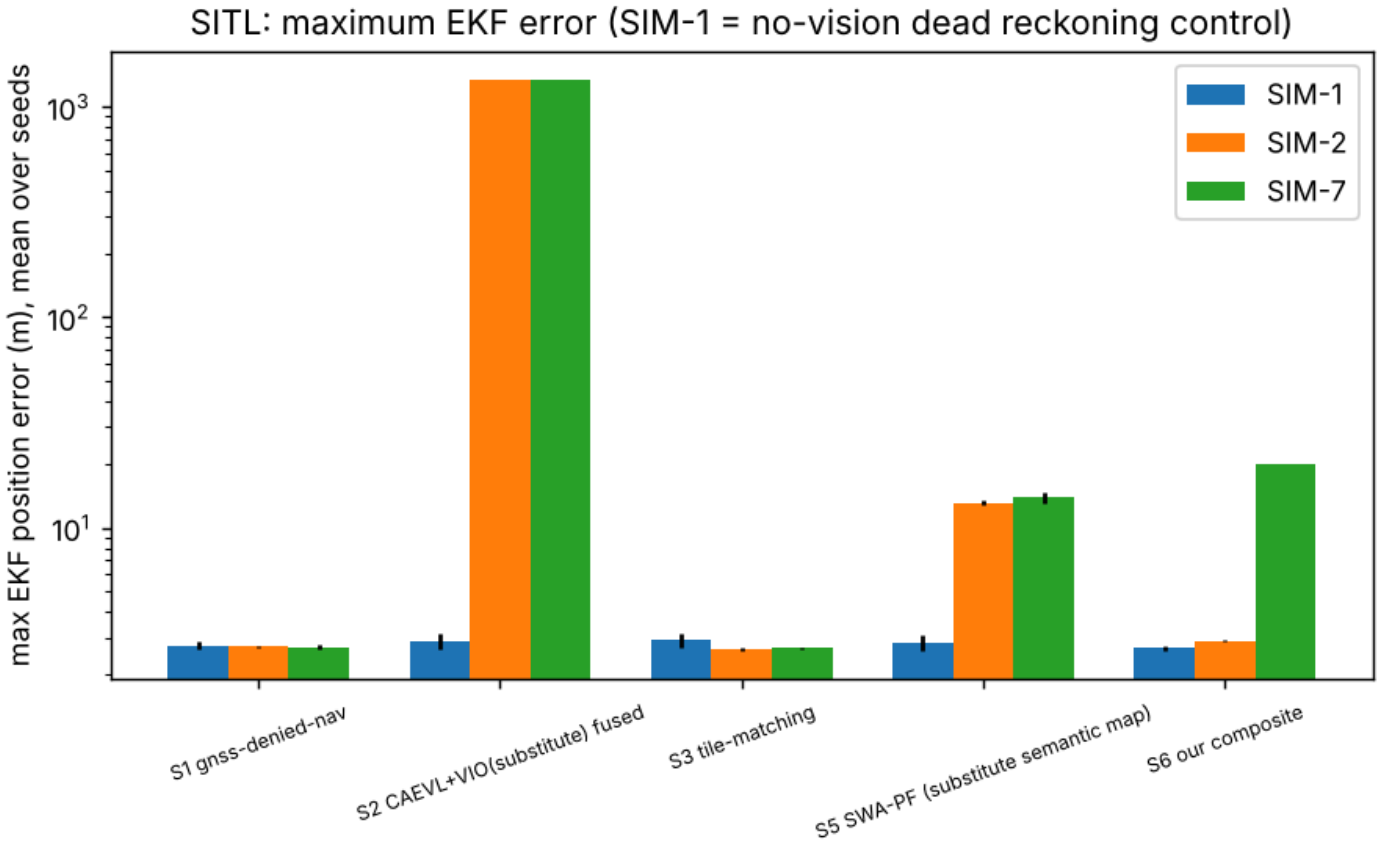
Row	SIM-0	SIM-1	SIM-2	SIM-3	SIM-4a	SIM-4b	SIM-5	SIM-6	SIM-7
S5	100%	0%	100%	0%	100%	100%	33%	0%	100%
S6	100%	0%	100%	0%	100%	100%	33%	100%	100%

EKF failsafe / LAND triggered (share of seeds)

Row	SIM-0	SIM-1	SIM-2	SIM-3	SIM-4a	SIM-4b	SIM-5	SIM-6	SIM-7
Control: truth + 5 m noise fixes	0%	100%	0%	100%	0%	0%	0%	0%	0%
S1	0%	100%	100%	100%	100%	100%	100%	100%	100%
S2 (fused)	0%	100%	100%	100%	100%	100%	100%	100%	100%
S2 VIO-only track (no map fixes)	0%	100%	100%	100%	100%	100%	100%	100%	100%
S3 (as published)	0%	100%	100%	100%	100%	100%	100%	100%	100%
S3 + compass de-rotation (our extension)	0%	100%	100%	100%	100%	100%	100%	100%	100%
S5	0%	100%	100%	100%	100%	100%	100%	100%	100%
S6	0%	100%	0%	100%	0%	0%	67%	0%	0%

Distance from home at the end of the run (m, SIM-7 = after return-to-launch)

Row	SIM-0	SIM-1	SIM-2	SIM-3	SIM-4a	SIM-4b	SIM-5	SIM-6	SIM-7
Control: truth + 5 m noise fixes	269	528	270	628	271	270	270	270	206
S1	269	529	528	528	528	528	528	513	528
S2 (fused)	269	529	511	512	511	512	511	485	511
S2 VIO-only track (no map fixes)	269	529	705	625	706	706	704	495	705
S3 (as published)	269	529	528	528	528	528	528	524	528
S3 + compass de-rotation (our extension)	269	529	651	596	714	690	687	714	707
S5	269	529	262	626	261	261	430	79	262
S6	269	529	266	627	266	265	561	266	199



SITL: maximum EKF error per solution for SIM-1 (no vision), SIM-2 and SIM-7, 3 seeds

5. Findings and next steps

Adopt

- **Retrieve → match → PnP → gate → track (S6 structure).** Evidence: S6-T1 (764 frames, 92.8 % fixed, 0.41 m median, 100 % precision), S6-T4 with tracking (95.3 % fixed, bridges the blur/haze gaps), SITL SIM-2/4/6/7 completed in all seeds. Retrieval quality is the lever: MegaLoc recall@1 84.3 % against 39.9 % for DINOv2-small CLS+mean-patch (S6-T2), and XFeat+LighterGlue loses only 8 points of fix rate against SuperPoint+LightGlue while running 4x faster (S6-T3).
- **A consistency gate with barometer and compass hints.** S1 and S6 produced no wrong fix above 8 m (S1) or none over 100 m (S6); S3 without any gate accepted 7 wrong fixes of 52 (precision@100 m 86.5 %), and S2's consensus step accepted 84 fixes of which 7 % were right (the wrong fixes agreed with each other).
- **Orienting the frame to the map with the compass.** De-rotating S3's frames raised its fix rate from 34 % to 56 % with no other change.

Drop or do not rely on

- **S2 as tested.** Retrieval near chance (4.6 % recall@100 m) with our 50-minute CPU-trained CAEVL; fused RMSE 798 m against 5 m for the VIO alone. Fusion does help a drifting VIO if fixes are good (synthetic sanity test: 50-67 % lower RMSE with 20-40 % good fixes) but never helped here, so the author's 36 % claim is **not reproduced and not refuted** (different encoder, VIO, data).
- **S3 as a standalone localiser** (no rotation handling, no gate, 6-14 s per step on CPU).
- **S1 with pretrained weights.** The author's fine-tuned DINOv2 is not published; with DINOv2-base the true tile ranks 11-212 of 1,225 under blur and haze, and fixes exist only for the first 44 s.
- **S4** cannot be judged: the 6.9 GB image, PX4/Gazebo build, GPU and X11 requirements did not fit here. Its matcher needs a prior position and heading, so it cannot localise from a cold start, and ArduPilot support is marked unmaintained.

Methodological findings

- **The video must be nadir.** Our first render used a body-fixed camera on a quadcopter that pitches −30° in cruise: every solution failed or needed rectification (S5 106.8 m median error without rectification; S6's nadir assumption failed completely). Any real NIDAR payload needs a gimbal or an attitude-aware rectification stage; S5's and S6's rectification extensions recover it only with accurate attitude.
- **Same-map imagery inflates accuracy.** Median errors of 0.07-0.4 m for S3 and S6 are a property of the test, not of the methods.
- **The EKF side matters as much as the vision side.** With a position-only second GPS, the EKF3 failsafe lands the vehicle about 68 s into the flight with no vision (SIM-1) and within about 8 s of the start of a 20 s fix drop-out (SIM-3 lands at about 88 s, even for ideal 5 m-noise fixes). 5 % outliers of 100-300 m (SIM-5) cost S5 and S6 two thirds of their missions: the fix quality gate needs to run on the autopilot side as well.
- **Latency.** On this CPU the steps cost 0.06 s (S2), 0.9 s (S6), 3-16 s (S5), 5.6 s (S3), 8.8 s (S1). Only S2 (poor fixes) and S6 (partly) approach a 5 Hz stream; the others need a GPU or a much lighter model before they can run at the frame rate.

Next three tests (in order)

1. **S6 on real NIDAR video with a reference map of a different date**, with the thresholds frozen beforehand and a held-out flight. This removes the three optimism sources (same-map imagery, tuning on the test frames, noise-free hints).
2. **S6 on a Jetson Orin with TensorRT FP16**, measuring the whole pipeline (MegaLoc, XFeat, LighterGlue, PnP) at 5 Hz and its power mode; LighterGlue's ONNX export failed here and must be solved first.

3. **Closed-loop SITL (Gazebo-rendered camera) with a velocity/odometry aid in the EKF**, comparing S6 and S5 on feature-poor terrain (forest, water, fields) and with an injected fix outage and outliers, to fix the position-only EKF limits seen in SIM-3 and SIM-5.

Appendices

A. Reproduction commands

```
# data (synthetic NIDAR-like), from the repository root gpsdenied-eval/
python tools/fetch_esri_map.py --lat 18.5204 --lon 73.8567 --half_m 900 --z 19 -
-out data/nidar/map_esri_z19.png
python tools/sitl_mission.py --out data/nidar/flight_sitl            #
ArduCopter SITL A->B->C->D at 200 m AGL
python tools/render_video.py --flight data/nidar/flight_sitl --map
data/nidar/map_esri_z19.png --out data/nidar --stabilised
# harness tests
cd harness && python -m pytest test_harness.py
# SITL scenarios for one solution (3 seeds x 9 scenario rows)
python harness/run_sitl_all.py --solution s6 --estimates results/s6/nidar/S6-T1-
<date>-s0/estimates.csv
# serial latency / real-time / constrained runs
bash report/serial_latency.sh
# report
python harness/collect.py --runs report/runs.json --out
report/metrics_master.json
python harness/terrain.py --runs report/runs.json --out report/terrain.json --
fig report/figures/terrain_breakdown.png
python harness/plots.py --runs report/runs.json --out report/figures
python harness/arch_svg.py report/arch
python harness/report_build.py
```

Per-solution replay commands are in each `SETUP.md` / `ANALYSIS.md`.

B. Where the evidence is

What	Path
Raw replay logs, estimates, metrics	`results/<solution>/nidar/<run-id>/` (`estimates.csv`, `run.json`, `metrics.json`, `log.txt`)

What	Path
Oblique-camera stress runs (first, defective render)	`results/<solution>/nidar_oblique/`
SITL runs (summary, EKF-vs-truth trace, bridge log, final parameter file `params.json`; dataflash logs for seed 0 are local, git-ignored)	`results/<row>/sitl/<scenario>-s<seed>/`
Environment lockfiles and commit hashes	`solutions/<solution>/lock.txt`, `solutions/<solution>/COMMIT.txt`
Command logs / status log with every action and outcome	`STATUS.md`
Failure logs (setup failures with the exact error)	`solutions/<solution>/SETUP.md`, `results/s4/nidar/S4-T1-*/log.txt`
Harness code and tests	`harness/` (`metrics.py` is the single metric implementation)
Data generator	`tools/`

C. Licence list (every component recorded in the solution `SETUP.md`)

S1: MIT code; SuperPoint weights research-only. S2: no licence for the repository or the CAEVL fork; OpenVINS GPLv3 (isolated in `solutions/s2_uav_geolocalization/openvins_gpl/`). S3: no licence. S4: MIT. S5: none declared for code, weights or MAFS. S6: MIT / Apache-2.0 / BSD parts, LighterGlue weight licence unconfirmed. S7: YFS90 none; Bearing-UAV Apache-2.0 code; STHN MIT code. Esri World_Imagery tiles: benchmark use only, not redistributed (the report contains no imagery; imagery-derived files are git-ignored).